



AVEVA Plant (12 Series)

Programmable Macro Language

TRAINING GUIDE

TM-1401

Revision Log

Date	Revision	Description of Revision	Author	Reviewed	Approved
16/01/2009	0.1	Issued for Review	BT		
26/01/2009	0.2	Reviewed	BT	EJW	
29/01/2009	1.0	Approved for Training 12.0.SP3	BT	EJW	RP
18/02/2009	1.1	Issued for Review	EJW		
19/02/2009	1.2	Reviewed	EJW	BT	
19/02/2009	2.0	Approved for Training 12.0.SP3	EJW	BT	RP
22/07/2009	3.0	Approved for Training 12.0.SP4	EJW	BT	RP

Updates

All headings containing updated or new material will be highlighted.

Suggestion / Problems

If you have a suggestion about this manual or the system to which it refers please report it to the AVEVA Group Solutions Centre at gsc@aveva.com

This manual provides documentation relating to products to which you may not have access or which may not be licensed to you. For further information on which products are licensed to you please refer to your licence conditions.

Visit our website at <http://www.aveva.com>

Disclaimer

Information of a technical nature, and particulars of the product and its use, is given by AVEVA Solutions Ltd and its subsidiaries without warranty. AVEVA Solutions Ltd. and its subsidiaries disclaim any and all warranties and conditions, expressed or implied, to the fullest extent permitted by law.

Neither the author nor AVEVA Solutions Ltd or any of its subsidiaries shall be liable to any person or entity for any actions, claims, loss or damage arising from the use or possession of any information, particulars or errors in this publication, or any incorrect use of the product, whatsoever.

Trademarks

AVEVA and Tribon are registered trademarks of AVEVA Solutions Ltd or its subsidiaries. Unauthorised use of the AVEVA or Tribon trademarks is strictly forbidden.

AVEVA product names are trademarks or registered trademarks of AVEVA Solutions Ltd or its subsidiaries, registered in the UK, Europe and other countries (worldwide).

The copyright, trademark rights or other intellectual property rights in any other product, its name or logo belongs to its respective owner.

Copyright

Copyright and all other intellectual property rights in this manual and the associated software, and every part of it (including source code, object code, any data contained in it, the manual and any other documentation supplied with it) belongs to AVEVA Solutions Ltd. or its subsidiaries.

All other rights are reserved to AVEVA Solutions Ltd and its subsidiaries. The information contained in this document is commercially sensitive, and shall not be copied, reproduced, stored in a retrieval system, or transmitted without the prior written permission of AVEVA Solutions Limited. Where such permission is granted, it expressly requires that this Disclaimer and Copyright notice is prominently displayed at the beginning of every copy that is made.

The manual and associated documentation may not be adapted, reproduced, or copied in any material or electronic form without the prior written permission of AVEVA Solutions Ltd. The user may also not reverse engineer, decompile, copy or adapt the associated software. Neither the whole nor part of the product described in this publication may be incorporated into any third-party software, product, machine or system without the prior written permission of AVEVA Solutions Limited or save as permitted by law. Any such unauthorised action is strictly prohibited and may give rise to civil liabilities and criminal prosecution.

The AVEVA products described in this guide are to be installed and operated strictly in accordance with the terms and conditions of the respective licence agreements, and in accordance with the relevant User Documentation. Unauthorised or unlicensed use of the product is strictly prohibited.

Printed by AVEVA Solutions on 18 August 2009

© AVEVA Solutions and its subsidiaries 2001 – 2007

AVEVA Solutions Ltd, High Cross, Madingley Road, Cambridge, CB3 0HB, United Kingdom.

1	Introduction	9
1.1	Aim	9
1.2	Objectives	9
1.3	Prerequisites	9
1.4	Course Structure.....	9
1.5	Using this guide	9
2	PML Overview	11
2.1	PML 1 – String Based Command Syntax.....	11
2.1.1	Example of a simple command syntax macro	11
2.1.2	Examples of command syntax	11
2.1.3	Syntax graphs	12
2.2	PML 2 – Object-orientated programming	13
2.2.1	Features of PML 2.....	13
2.2.2	Examples of object-orientated PML	13
2.2.3	Software Customisation Reference Manual.....	13
2.3	PML Objects	14
2.3.1	Creating variables (instances of objects)	14
2.3.2	Naming conventions.....	15
2.3.3	Using the members of an object	15
2.3.4	Special objects used in PDMS	15
2.4	PML Functions and Methods	16
2.4.1	Arguments of type ANY.....	16
2.5	PML Forms.....	17
2.6	PDMSUI environment variable.....	18
2.7	PMLLIB environment variable	18
2.8	Modifications to the PDMSUI and PMLLIB	19
	Exercise 1 – Updating the environment variables	19
2.9	General Notes on PML.....	20
3	Macros, Synonyms and Control Logic	21
3.1	A simple Macro.....	21
3.2	Finding examples of command syntax.....	21
3.3	Communicating with AVEVA Products in PML.....	21
3.4	Parameterised Macros.....	22
3.5	Synonyms	22
3.6	Defining variables	23
3.6.1	Numbered variables	23
3.6.2	PML 1 style variables	23
3.6.3	PML 2 style variables	23
3.7	Expressions.....	24
3.7.1	Expression operators	24
3.7.2	Operator Precedence.....	24
3.7.3	PML 2 expressions.....	24
3.8	Arrays	25
3.9	Concatenation Operator	25
3.10	DO loop	25
3.10.1	DO loops with BREAK.....	26
3.10.2	DO loops with SKIP	26
3.10.3	DO INDEX and DO VALUES	26
3.11	IF Statements.....	27
3.11.1	IF, ELSEIF and ELSE Statements	27
3.12	Branching	27
3.12.1	Conditional Branching	28
3.13	Error handling	28
3.13.1	Error codes.....	28
3.13.2	Error Handling using the HANDLE syntax	28
3.14	Alert Objects.....	29
3.14.1	Alert Objects with no return value	29
3.14.2	Alert Objects that return value.....	29

3.14.3	Input Alerts	29
Exercise 2 - The centre of a Handwheel.....		30
Exercise 3 - The full Handwheel.....		31
4	PML Functions	33
4.1	Functions instead of macros	33
4.2	Creating a PML function.....	33
4.3	PML procedures	34
4.4	Making use of methods on PML objects	34
4.4.1	Method information	35
4.4.2	Method Concatenation	35
4.5	Using the !!CE Object.....	36
Exercise 4 – Convert the HoseReel macro into a PML procedure		37
5	Forms	39
5.1	Forms Are Global Objects.....	39
5.2	Dynamic loading of Objects, Forms and Functions.....	39
5.3	Defining a form.....	40
5.3.1	Using the .net framework	40
5.3.2	Showing and hiding Forms.....	40
5.3.3	Built-in methods for Forms	41
5.4	Callbacks	41
5.5	Form Gadgets.....	42
5.5.1	Built-in members and methods for Gadgets	42
5.5.2	Gadget positioning	43
5.5.3	Docking and Anchoring Gadgets	44
5.6	Paragraph Gadgets.....	45
5.7	Button Gadgets	45
5.8	Text Entry Gadgets	46
5.9	Format Object.....	46
5.10	List Gadgets	47
5.11	Frame Gadgets	48
5.12	Textpane Gadgets.....	49
5.13	Option Gadgets	50
5.14	Toggle Gadgets.....	50
5.15	Radio Gadgets.....	51
5.16	User-defined form members	52
5.17	Tooltips	52
Exercise 5 - Working with a form		53
5.18	Progress bars and interrupting methods	55
5.19	Open Callbacks	56
5.20	Menus.....	57
5.20.1	Defining a menu object on a form	57
5.20.2	Defining a bar menu on a form.....	57
5.20.3	Defining a popup menu on a form.....	58
5.20.4	Adding menus objects to a form.....	58
Exercise 6 – Extending the form		59
6	PML Objects	61
6.1	Built in PML object types	61
6.2	Methods available to all PML objects	61
6.3	The FILE Object.....	62
6.3.1	Using FILE objects	62
6.3.2	Opening a FILE object in Notepad	62
6.3.3	Using the standard file browser.....	63
6.4	User-Defined Objects	64
Exercise 7 – User-defined objects		65
7	Collections.....	67
7.1	COLLECT command syntax (PML 1 style)	67
7.2	EVALUATE command syntax (PML 1 style).....	67
7.3	COLLECTION object (PML 2 style).....	67
7.4	Evaluating the results from a COLLECTION object	68
Exercise 8 – Equipment Collections.....		69
8	View Gadgets	71

8.1	Alpha Views	71
8.2	Plot View Example	71
8.3	Volume View example	72
Exercise 9 – Adding a Volume View		74
9	Event Driven Graphics (EDG)	77
9.1	A simple EDG event	77
9.2	Using EDG	78
Exercise 10 – Adding EDG to forms		79
10	Miscellaneous	81
10.1	Recursive Pml2 Functions	81
10.2	Undo and Redo	81
10.2.1	Marking the database	81
10.2.2	Undo and Redo Database Commands	81
10.3	Error tracing	82
10.4	PML Publisher	82
11	Menu Additions	83
11.1	Miscellaneous Notes	83
11.2	Modules that use the new Addins functionality	83
11.3	Adding a Menu/Toolbar	83
11.4	Application Design	83
11.5	Form and Toolbar Control	84
11.6	Converting Existing Applications	84
11.6.1	DBAR and Add-in Object	84
11.6.2	Bar Menu and Toolbar	85
11.6.3	Define toolbar	85
11.6.4	Adding to Menus	85
11.6.5	Adding new menus to form	85
11.7	Example object including toolbars, bar menus and menus	86
Worked Example – Creating a toolbar with Design		88
Exercise 11 – Add a Utility and Toolbar menu		90
Appendix A – PDMS Primitives		91
Appendix B – Example code		97
Appendix B1 - Example ex2.mac		97
Appendix B2 - Example ex3.mac		97
Appendix B3 - Example ex4.mac (fixed)		97
Appendix B4 - Example ex4.pmlfnc		99
Appendix B5 - Example ex5.pmlfrm		99
Appendix B6 - Example ex6.pmlfrm		101
Appendix B7 - Example cuboid.pmlobj		103
Appendix B8 - Example datastore.pmlobj		103
Appendix B9 - Example ex8.pmlfrm		103
Appendix B10 - Example ex9.pmlfrm		107
Appendix B11 - Example ex10.pmlfrm – part 1		113
Appendix B12 - Example ex10.pmlfrm – part 2		113
Appendix B13 - Example ex10.pmlfrm – part 3		114
Appendix B14 - Example apppml.obj		115

1 Introduction

This manual is designed to give an introduction to the AVEVA Plant Programming Macro Language. There is no intention to teach software programming but only provide instruction on how to customise PDMS using Programmable Macro Language (PML) in AVEVA Plant.



This training guide is supported by the reference manuals available within the products installation folder. References will be made to these manuals throughout the guide.

1.1 Aim

The following points need to be understood by the trainees:

- Understand how PML can be used to customise AVEVA Plant
- Understand how to create Functions, Forms and Objects.
- Understand how to use the built-in features of AVEVA Plant
- Understand the use of Addins to customise the environment.

1.2 Objectives

At the end of this training, you will have a:

- Broad overview of Programmable Macro Language (PML)
- Basic coding practices and conventions
- How PML can interact with the Design model
- How Forms and Menus can be defined with PML

1.3 Prerequisites

The participants must have completed an AVEVA Basic Design Course and have a familiarity with PDMS. Previous experience of programming is of benefit –but not necessary

1.4 Course Structure

Training will consist of oral and visual presentations, demonstrations, worked examples and set exercises. Each trainee will be provided with some example files to support this guide. Each workstation will have a training project, populated with model objects. This will be used by the trainees to practice their methods, and complete the set exercises.

1.5 Using this guide

Certain text styles are used to indicate special situations throughout this document, here is a summary;

Menu pull downs and button press actions are indicated by **bold dark turquoise text**.

Information the user has to Key-in '**will be red and BOLD**'



Additional information will be highlighted



Reference to other documentation will be separate

System prompts should be bold and italic in inverted commas i.e. '***Choose function***'

Example files or inputs will be in the courier new font with colours and styles used as before.

2 PML Overview

Programmable Macro Language (PML) is the customisation language used by AVEVA Plant and it provides a mechanism for users to add their own functionality to the AVEVA Plant software family. This functionality could be as simple as a re-naming macro, or as complex as a complete user-defined application (and everything in between).

PML is a coding language specific to AVEVA products based on the command syntax that is used to drive PDMS. As the products are developed, PML is also improved providing new functionality and bringing it closer to other object-orientated programming languages, while still retaining the command syntax. Although it is one language, there are three distinct parts to it:

- PML 1 the first version of PML based on command syntax. String based, but provides IF statement, loops, variables, error handling
- PML 2 object oriented language extending the ability of PML. Use of functions, objects and methods to process information
- PML .NET provides the platform in PML to display and use objects created in other .NET languages (covered in the PML .NET course)

2.1 PML 1 – String Based Command Syntax

When PML is written as command syntax, PDMS processes it as individual command lines and runs them in sequence - as if they had been typed directly into the command window.

A simple macro is likely to be written completely in command syntax and allows users to re-run popular commands. Saved as an ASCII file, the macro can be run in PDMS through the command window (by typing **\$M/FILENAME** or by dragging and dropping).

2.1.1 Example of a simple command syntax macro

The following is an example of a simple macro that can be used to create an Equipment element. As each line are run in order, the same piece of equipment will be created each time the macro is run.

```
NEW EQUIP /ABCD
NEW BOX
XLEN 300 YLEN 400 ZLEN 600
NEW CYL DIA 400 HEI 600
CONN P1 TO P2 OF PREV
```

Macros can be extended to include IF statements, DO loops, variables and error handling (these are explained further in section 3.10) If additional information is typed onto the same line as the call for the macro, then these become input parameters and are available for use.

i.e. **\$M/BUILDBOX 100 200 300**

This means that the extra 3 values after the macro name are treated as parameters 1, 2 and 3. When entered in this way, these parameters are then available for use in the macro instead of fixed values.

2.1.2 Examples of command syntax

The following are examples of command syntax that can be typed directly into the command window (or used within a macro):

Working with Elements and attributes:

- To find out attribute information about the current element, type **Q ATT**
- To create a new element (for example, BOX), type **NEW BOX**
- To find out a specific attribute (for example NAME), type **Q NAME**

- To set a value to an attribute on the current element (for example XLEN), type **XLEN 300**

Manipulating the drawlist in DESIGN:

- To add the current element to the drawlist, type **ADD CE**
- To add a specific element below a piece of equipment, type **ADD ONLY /E1301-S1**
- To remove all the elements from the drawlist, type **REM ALL**

Annotating elements in DESIGN:

- To label the current element with its name, type **MARK CE**
- To label the element /PIPE with its name, type **MARK /100-B-1-B1**
- To remove the label from the current element, type **UNMARK CE**
- To remove all labels, type **UNMARK ALL**
- To mark all branch elements with their names, type **MARK WITH (NAME) ALL BRAN**
- To mark all large valve elements with their specification reference, type **MARK WITH (NAME OF SPREF) ALL VALV WHERE CPAR[1] GT 100**

Using Aid graphics:

- To draw an unnumbered graphic aid line, type **AID LINE E0N0U0 TO E0N1000U1000**
- To draw a sphere aid (aid number 1000), type **AID SPHERE NUM 1000 E0N0U0 DIAM 200**
- To remove all unnumbered graphical aid lines, type **CLEAR AID LINE UNN**
- To remove all number 1000 sphere aids, type **CLEAR AID SPHERE 1000**
- To remove all graphical aids, type **CLEAR AID ALL**

Adding colour to the 3D model:

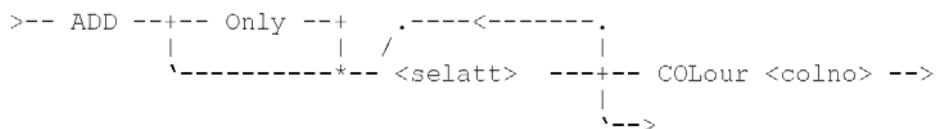
- To apply the standard enhance colour to the current element, type **ENHANCE CE**
- To apply colour 10 to element /PIPE, type **ENHANCE /PIPE COL 10**
- To remove all colour from the 3D model, type **UNENHANCE ALL**

Position and orientation of elements:

- To move the current element 1000mm in a N45E direction, type **MOVE N45E DIST 1000**
- To move the current element E, until it is inline with /BOX, type **MOVE E THRU /BOX**
- To rotate the current element around it's origin (pointing up) by 45, type **ROTATE BY 45 ABOUT U**
- To relatively move the current element east by 1000mm, type **BY E 1000**
- To explicitly position the current element, type **AT E0 N1000 U2000**
- To position & orientate the current element based on /EQUIP-N1, type **CONNECT P2 TO P0 OF /EQUIP-N1**
- To position & orientate the current element based on the previous, type **CONNECT P3 TO P1 OF PREV**

2.1.3 Syntax graphs

Many examples of command syntax are provided in the various reference manuals within the installation folder of AVEVA Plant. For each a syntax graph is provided to show the various combinations of syntax available:



The above syntax graph is for the **ADD** syntax (used to add elements to the drawlist). From the graph it can be seen that only two words are required (e.g **ADD /PIPE**) but others can be included (e.g. **ADD ONLY /PIPE1 /PIPE2 COL 3**).

 More examples of command syntax and the supporting syntax graphs can be found in the relevant reference manuals provided with AVEVA Plant.

2.2 PML 2 – Object-orientated programming

PML 2 is almost an object oriented language and provides most features of other Object-Orientated Programming (OOP) languages. Operators and methods are polymorphic and overloading of methods is supported. There is no inheritance and there is no concept of public or private variables.

PML 2 provides for classes of built-in, system-defined and user-defined object types. Objects have members (their own variables) and methods (their own functions). All PML variables are an instance of a built-in, system-defined or user-defined object type.

Through the use of method concatenation, it is possible to achieve multi operations in a single line of code. This means that PML 2 methods are typically shorter and easier to read than the PML 1 equivalent. While most PML 1 macros will still run within PDMS, PML 2 brings many new features that were previously unavailable. If used together with command syntax it must be expressed as a string before use.

2.2.1 Features of PML 2

The main features of PML 2 are:

- Available Variable Types - STRING, REAL, BOOLEAN, ARRAY
- Built in Methods for commonly used actions
- Global Functions supersede old style macros
- User Defined Object Types
- PML Search Path (PMLLIB)
- Dynamic Loading of Forms, Functions and Objects

2.2.2 Examples of object-orientated PML

Declaration of objects (variables):

- To declare a local variable as a REAL 3, type **!realVariable = 3**
- To declare an global BOOLEAN variable as false, type **!!booleanVariable = FALSE**
- To set the third value in an ARRAY as Fred, type **!arrayVariable[3] = |Fred|**
- To declare a variable as an empty position object, type **!position = object position()**

Finding out information about objects (variables)

- To find out the value of an object, type **q var !exampleObject**
- To find out the type of an object, type **q var !exampleObject.objectType()**
- To find out what members an object has, type **q var !exampleObject.attributes()**

Methods available on objects:

- To find out what methods are available on an object, type **q var !exampleObject.methods()**
- To find out is an object has been given a value, type **q var !exampleObject.set()**
- To find out how large an ARRAY variable is, type **q var !arrayVariable.size()**
- To find out how long a STRING variable is, type **q var !stringVariable.length()**

Clearing an object

- To empty an object, type **!exampleObject.clear()**
- To delete an object that is no longer needed, type **!exampleObject.delete()**

2.2.3 Software Customisation Reference Manual

Many examples of the major objects available in AVEVA Plant are provided in the Software Customisation Reference Manual with the products installation folder. For each object, the **members** and **methods** are listed and explained.

- For each of the members the name, type and purpose is provided
- For each method the name, argument types, returned object type and purpose are provided.

 *Refer to the Software Customisation Reference Manual for more information about the major objects.*

2.3 PML Objects

Every variable defined with PML has an object type. This type is set when the variable is defined and is fixed for the life of the variable. When assigning a value to a variable, the object type needs to be considered, otherwise an error may occur. There are three groups of object types available:

- Built-in (e.g. String, Real, Boolean and Array)
- System-defined (e.g. Position, Orientation)
- User-defined



Refer to the Software Customisation Reference Manual for more examples of System-defined objects

When a variable is declared as a specific object type, it is given all the **members** (attributes) and **methods** of the object definition. This means standard groupings can be setup and that code repetition can be avoided.

A user-defined object provides an opportunity to group object types together for a specific purpose. Once grouped as an object, it can be assigned to a variable and used as any other object. The following are examples of two user-defined objects:

```
define object FACTORY
  member .name is STRING
  member .workers is REAL
  member .output is REAL
endobject

define object PRODUCT
  member .productCode is STRING
  member .total is REAL
  member .site is FACTORY
endobject
```

These objects would be defined as two separate object files (.pmlobj) and loaded into PDMS. You will notice that the object **PRODUCT** is able to have a member which is another user-defined object. This means that the **PRODUCT** object has access to all the members and methods of a **FACTORY** object. For explanation is provided in chapter 6

2.3.1 Creating variables (instances of objects)

There are two types of variables in PML (1) **Local** and (2) **Global**

The difference between the two is that global variables last for the whole PDMS session and can be referenced directly from other PML routines. Local variables are only available for use within the routine which defined them. The variables a declared with a single '!' for local and a double '!!' for global

- **!localVariable** – a local variable
- **!!globalVariable** – a global variable

A variable object-type can be implied from the assigned value:

!name = Fred 	\$* To create a LOCAL, STRING variable
!!answer = 42	\$* To create a GLOBAL, REAL variable
!!flag = TRUE	\$* To create a GLOBAL, BOOLEAN variable

It is also possible to define a variable as an object-type without an initial value. The value is therefore **UNSET**

!name = STRING()	\$* To create a LOCAL, UNSET STRING variable
!!answer = REAL()	\$* To create a GLOBAL, UNSET REAL variable
!array = BOOLEAN()	\$* To create a LOCAL, UNSET ARRAY variable

2.3.2 Naming conventions

It is common practice to follow a naming convention when defining objects and variables. Use upper case for the name of the object type and mixed upper and lower case for the variable.

For example, the type might be **WORKERS** while the name of the variable might be **numberOf Workers**

① Notice that to make the variable name more meaningful, full words are used and upper case letters are used to indicate the words that make up the variable.

Variable names can be 16 alpha-numeric characters long. They should not start with a number or contain any spaces or full stops (full stops are used to indicate methods and members – explained later)

AVEVA uses a CD prefix on most of its global variables. New functionality does not use a prefix so all new PML must be checked for name clashes. Using your own prefix could help avoid this.

2.3.3 Using the members of an object

When a variable is declared as an object-type, it is also given all the objects members and methods. For example:

```
!newPlant = object FACTORY()
```

After being declared as above (using the **OBJECT** keyword and ending with a double bracket) the local variable !newPlant is now a **FACTORY** object and has the same members as the **FACTORY** object. These members are available to store information and can be assigned values in the following way:

```
!newPlant.name = |ProcessA|  
!newPlant.workers = 451  
!newPlant.output = 2000
```

① Notice the use of a dot between the variable and its member. This methods works providing the word after the dot is a valid member of a the variable object-type

Once assigned a value, this value is available for use. For example:

```
!numberOfWorkers = !newPlant.workers
```

This creates a new real local variable and assigns it the value 451

2.3.4 Special objects used in PDMS

In a standard PDMS DESIGN session, there are number of specialised objects which are loaded and used by standard product. These objects should not be deleted or overwritten, but are available for use. The particularly useful objects are:

- | | |
|-----------------------|--|
| ▪ !!CE | - a global DBREF object which tracks and represents the current element |
| ▪ !!ERROR | - a global ERROR object which holds information about the last error |
| ▪ !!PML | - used to obtain file path strings through the .getPathName() method |
| ▪ !!ALERT | - used to provide popup feedback to users |
| ▪ !!AIDNUMBERS | - used to manage aid graphics |
| ▪ !!APPDESMAN | - the form which represents the main DESIGN interface |

2.4 PML Functions and Methods

Functions and methods provide the actions of PML. When called, a function or method will run through the lines of PML it contains in order – just like a PML 1 style macro. Functions and methods may be passed arguments and even return values. A function which does not return a value is typically referred to as a PML Procedure.

A function and a method are written in the same style, the only difference is where the definition is stored and how it is called. A function is a global method (stored in its own file) and can be called directly on the command line (e.g. **call !!exampleFunction()**) while a method is local to the object it is defined within (e.g. **!exampleObject.exampleMethod()**)

Arguments become local variables within the function/method is the object-type needs to be declared within the definition. The returned object-type is also defined. For example:

```
define function !!area( !length is REAL, !width is REAL) is REAL
  !area = !length * !width
  return !area
endfunction
```

In this example, the function **!!area** is expecting two real arguments. The two arguments are expressed as local variables which are multiplied together to calculate the local variable **!area**. Using the return keyword, the variable **!area** is then returned. If you called the function in following way, you would expect the variable **!area** to have a value of 2400:

```
!area = !!area(12, 200)
```

Functions will be explained further with more examples in chapter 4 and methods on objects will be covered further in chapters 5&6

 *It is now common practice to write all macros as a functions or procedures*

2.4.1 Arguments of type ANY

When defining an argument within a method or function, you may declare it as an **ANY** object. This means that the argument can be of any object-type. When **ANY** is used, no argument check is carried out meaning that there may be a subsequent error in the code. For this reason, **ANY** should be used with special consideration:

```
define function !!argumentType(!argument is ANY)
  !type = !argument.objecttype()
  return !type
endfunction
```

2.5 PML Forms

For users of PDMS, the concept of a form should already be familiar. In terms of PML, a form is a global variable of the form name (for example **!!exampleForm**). A form is capable of owning members and methods (like any other object) but there are a set of predefined members which can be used to put gadgets on the form (buttons, lists, options etc). Some of these gadgets can be given callbacks, which can activate a standard command or call a function or object method (including the methods defined within the form)

```
setup form !!nameCE
  !this.formTitle = |Name CE|
  button .button |Print Name Of CE| call |!this.print()|
exit

define method .print()
  $p !!ce.flnn
endmethod
```



The above example is the definition of a form called nameCE. Saved within one file (.pmlfrm), it defines two form members (a predefined member for the title and a new button gadget) and a form method.

The gadgets defined on a form are objects-types as well. This means that when a button is defined, all the members and methods of a button are available.

!this is a special local variable and using it replaces the need to reference the owning object directly. For example, to call the method **.print()** from within the form, type **!this.print()** and to call the method from outside the form, type **!!nameCE.print()**

2.6 PDMSUI environment variable

All PML1 Macros are in a directory structure pointed at by the PDMS environment variable **PDMSUI**. The PDMSUI environment variable is set in the .bat file that is used to load PDMS (typically within evars.bat). To set the environment variable, the following line is needed in the .bat file:

```
set PDMSUI=C:\AVEVA\plant\PDMS12.0.3\pdmsui
```

The purpose of an environment variable is to reduce the length of the command used to call a macro. This means that **\$M/%PDMSUI%/DES/PIPE/PIPE** can be typed instead of the full file path.

In standard product, this process is shortened further as all PML1 macros and forms are called using synonyms. For example, the macros associated with piping are called using the synonym **CALLP**:

```
$S CALLP=$M/%PDMSUI%/DES/PIPE/$s1  
CALLP MPIPE
```

 *If all synonyms are killed then PDMS will cease to function as normal*

2.7 PMLLIB environment variable

All PML 2 objects and functions are in a directory structure pointed at by the PDMS environment variable **PMLLIB**. As with the PDMSUI variable, this is set in the .bat file which runs PDMS. To set the environment variable, the following line is needed in the .bat file:

```
set PMLLIB= C:\AVEVA\plant\PDMS12.0.3\pmllib
```

The PMLLIB environment variable differs because it can be searched dynamically. This means that the individual files do not need to be referenced directly and can be called by name. This is possible because PDMS compiles a **pml.index** file which sits in the PMLLIB folder and provides the path to all suitable files within it. For example, to load and show a form the command is **show !!exampleForm** (no need to reference the file path at all)

If a new file is created or the PMLLIB variable changed then there is a need to update the pml.index file. This can be done by typing **PML REHASH** onto the command window. If there are multiple paths that need updating, type **PML REHASH ALL**

If a pml object has already been loaded into PDMS, but the file definition has changed then the object needs to be reloaded before the changes can be seen. This can be done by typing either **pml reload form !!exampleForm** or **pml reload object EXAMPLEOBJECT**

Although not necessary, it is good practice to organise the files below the PMLLIB folder. A standard PDMS installation organises the files based on application and then on forms, functions, objects. This is normally a good starting point for organising customisation.

2.8 Modifications to the PDMSUI and PMLLIB

As PML is developed, it is normal practice to store it in a parallel file structure outside the standard installation directory. This parallel file structure can then be reference by the .bat files. There are a couple of reasons why this is a good idea:

- It keeps the customisation separate from the standard install, so as subsequent versions are installed there will not be a need to move the customised files.
- If any standard files are modified then the originals are still available if required
- If the customisation fails, the standard installation is available to go back to
- Many local PDMS installations can reference the same customisation from a network address

 *Any changes to AVEVA standard product may cause PDMS to function inappropriately*

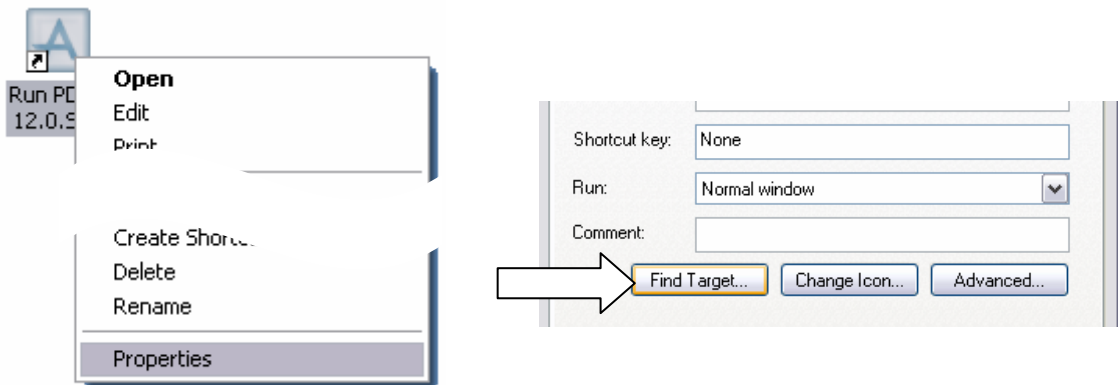
It is possible to get PDMS to look in different places for PML and this is done by setting the environment variables to multiple paths. This allows the standard install to be kept separate from user and company customisation. This is done by updating the variable to include another path:

Set PDMSUI=c:\temp\pdmsui %pdmsui%

This will put the additional file path in front of the standard (which would have already been defined in the .bat file). This change can also be checked in a PDMS session by typing **q evar PDMSUI** or **q evar PMLLIB** onto the command window.

Exercise 1 – Updating the environment variables

- 1
 - Extract the provided files into the folder **C:\temp**
 - Right-click on the icon that opens PDMS. **Choose Properties...** then **Find Target**



- 2
 - Open the file in a suitable text editor
 - Add the following lines to pdms.bat (after the line that calls evars.bat)

set PDMSUI=c:\temp\pdmsui %pdmsui%
set PMLLIB=c:\temp\pmllib %pmllib%

 - Save the .bat as a **new file** to the computers desktop. This is now the icon you will use to enter PDMS.
- 3
 - Load PDMS by using the new .bat and enter the Design module
- 4
 - Confirm that the search paths have been correctly updated. Type **q evar PDMSUI** or **q evar PMLLIB** into the command window.

2.9 General Notes on PML

- PML functions and methods run through their definitions in line order (control logic can be used to alter this order)
- Functions should be written in preference to macros
- Variables are instances of object definitions
- The return command can be used to exit a function/method as well as return a value
- PML files are **ASCII** and can be created/edited in any basic text editor
- PML 1 files are saved under **PDMSUI** folder and PML 2 under the **PMLLIB** folder
- PML 2 objects have specific file extensions (.pmlfnc, .pmlobj and .pmlfrm)
- If new created, PML can be found by typeing **pml rehash all**
- Once loaded, objects can be reloaded by typing **pml object reload OBJECT**
- Once loaded, forms can be reloaded by typing **pml object reload FORM**
- When declaring a string, text delimiters must be used. Either '**single quotes**' or **|vertical bars|**
- File paths of files can be obtained by using **!!pml.getPathName(|form.pmlfrm|)**
- The **\$** is used as an escape charater and can be used to expand a variable before it is read as command syntax. If a \$ symbol is actually required, then two must be entered
- To provide comments in statements with PML, one of the following can be used:
 - For a comment line, start the line with two hyphens --
 - For a comment at the end of a line, start the comment with **\$***
 - For a block comment, start with a **\$(** and end with a **\$)**
- Variable names are not case sensitive
- String comparisions are case sensitive
- Variable names can be 16 characters long and should not start with a number or contain a dot
- It is good practice to name variables in lower case, using upper case to separate words e.g. **!stringLength**
- It is possible to abbreviate some command syntax. The required part is shown in upper case with the syntax graphs e.g. **WIDTH** means that **WID** is acceptable when declaring width

3 Macros, Synonyms and Control Logic

A macro is a group of PDMS commands written to a file (in sequence) that can be run together. This removes the need for user to have to enter every line of code separately and that repetitive processes can be run as a macro.

3.1 A simple Macro

The following example is a macro which creates an EQUI element which owns a BOX and CYLI. To test the example drag and drop the provided file into the command window.

When running this macro in, ensure the Current Element (CE) is a ZONE or below

```
NEW EQUIP /ABCD
NEW BOX
XLEN 300 YLEN 400 ZLEN 600
NEW CYL DIA 400 HEI 600
CONN P1 TO P2 OF PREV
```

You have created your first macro. To run the macro into PDMS, either drag and drop the file into the command window or type **\$M/** followed by the full file path of the saved macro.

e.g. **\$M/%PDMSUI%\examples\simpleMac.mac**

3.2 Finding examples of command syntax

While developing a macro there are four main techniques of deriving the command syntax needed:

- DB listing utility create the required elements in PDMS using the standard appware and use the DB Listing utility to output the information (**Utilities>DB Listing...**)
- \$Q syntax if part of a command is known, the syntax which follows it can be found by typing **\$Q** after the command e.g **NEW \$Q**
- Using standard product standard PDMS is supplied with numerous PML files which can be searched for keywords and used for inspiration
- Reference Manuals the reference manuals supplied with the product focus on command syntax and supply syntax graphs for each case.

As macros develop in complexity and use, it is likely that a mixture of the above will be used.

3.3 Communicating with AVEVA Products in PML

All commands need to be supplied to the command processor as **STRINGS**. This is important when working with variables as they may have another object-type. If a **\$** symbol is put in front of a variable, PDMS will expand the contents of the variable as a string and then read the line. For example:

```
!componentType = |BOX|
!xLength = 5600
NEW $!componentType XLEN $!xLength
```

This is the equivalent of writing **NEW BOX XLEN 5600**

3.4 Parameterised Macros

Macros can be parameterised. This means instead of hard coding the values in the macro, arguments can be passed to it and used instead. This allows the values to be varied, making the macro flexible. As an example, simpleMac.mac can be parameterised in the following way:

```
NEW EQUIP /$1
NEW BOX
XLEN $2 YLEN $3 ZLEN $4
NEW CYL DIA $3 HEI $4
CONN P1 TO P2 OF PREV
```

To test the example, find the provided example. To run this Macro, parameters need to be passed to it. The four parameters are defined by including the values of the parameters are the macro call:

e.g. **\$M/%PDMSUI%\examples\parameterMac.mac ABCDE 300 400 600**

If this macro is dragged and dropped into the command window, the parameters would be undefined and the macro will error. To avoid this, default parameter values can be set at the top of the macro using the **\$d=** syntax. For example:

```
$d1=ABCDEF
$d2=300
$d3=400
$d4=600
```

 *The defined default values will only be used if no parameters are passed to the macro.*

Macros may have up to 9 parameters separated by space. In the below example, ABC, DEF & GHK are seen as separate strings and therefore different parameters:

e.g. **\$M/%PDMSUI%\examples\nineParameter.mac ABC DEF GHK 55 66 77 88 99 00**

If a text string is required as a single parameter, it can be entered by placing a **\$<** before and a **\$>** after the string. **\$< \$>** act as delimiters and anything in between is interpreted as a single parameter.

e.g. **\$M/%PDMSUI%\examples\sevenParameter.mac \$<ABC DEF GHK\$> 55 66 77 88 99 00**

3.5 Synonyms

Synonyms are abbreviations of longer commands. They are created by assigning a command to a synonym variable. To call the command held by the synonym, just type the name of the synonym

e.g. **\$SNewBox=NEW BOX XLEN 100 YLEN 200 ZLEN 300** called by typing **NewBox**

It is also possible to parameterise a synonym:

e.g. **\$SNewBox=NEW BOX XLEN \$S1 YLEN \$S2 ZLEN \$S3** called by typing **NewBox 100 200 300**

A synonym can call itself and if it uses the **\$/** syntax (return character) it can be used to loop through elements. For example, a new XLEN value can be applied to all elements for level in the hierarchy

e.g. **\$SXChange=XLEN 1000 \$/ NEXT \$/ XChange**

Synonyms can be turned off and on with the **\$S-** and **\$S+** syntax. To kill a synonym, type **\$SXXX=** & to kill all synonyms **\$sk**. Standard PDMS relies on synonyms to function. If a required synonym is killed, the product will no longer function properly (requiring a restart)

3.6 Defining variables

There are number of different methods for defining variables in PDMS:

3.6.1 Numbered variables

Numbered variables are set by typing a number then value after the command VAR. Examples of this are below:

```
var 1 name
var 2 |hello|
var 3 (99)
var 4 (99 * 3 / 6 + 0.5)
var 117 pos in site
var 118 (name of owner of owner)
var 119 'hello ' + 'world ' + 'how are you'
```

The value of variable 1 can be obtained by typing **q var 1** into the command window. The available variable numbers only go to **119** (there is no 120) and they are module dependant. For these reasons, this technique is no longer commonly used and has been included in this training guide for completeness.

3.6.2 PML 1 style variables

The following are some examples of setting variables using the PML 1 style syntax:

```
VAR !NAME NAME          $* Takes the current element's (CE) name attribute
VAR !POS POS IN WORLD    $* Takes CE position attribute relative to the world
VAR !x |NAME|            $* Sets the variable to the text string |NAME|
VAR !temp (23 * 1.8 + 32) $* Calculate a value using the expression
VAR !list COLL ALL ELBO FOR CE $* Makes a string array of database references
```

This style of defining variables is still valid and is useful for command syntax that returns a value. It has been included within this guide primarily for information for when old code is upgraded.

3.6.3 PML 2 style variables

The following are some examples of setting variables using the PML 2 style syntax:

```
!name = !!ce.name        $* Takes the current element's (CE) name attribute
!pos = !!ce.pos.wrt(/*) $* Takes CE position attribute relative to the world
!x = |NAME|               $* Sets the variable to the text string |NAME|
!temp = 23 * 1.8 + 32     $* Calculate a value using the expression
```

These examples show that there are equivalents to the PML 1 style, but because PML 2 is object-based this can be incorporated into the declaration of the variable. Objects will be explained further with examples in chapter 6. There is no single line PML 2 equivalent to the PML 1 COLL syntax. This will be discussed in chapter 7

3.7 Expressions

Expressions are calculations using PML variables. This can be done in a PML 1 or PML 2 style:

```
VAR !Z ( |$!X| + |$!Y| )      PML 1
!Z = !X + !Y                  PML 2
```

 In the PML1 example, !Z is set as a *STRING* variable. In the PML2 example, !Z is returned as a *REAL*, if !X and !Y are *REAL*

3.7.1 Expression operators

There are a number of expression operators which are available for use within PML. The numeric operators and functions are the same for both PML 1 and PML 2 styles. For comparison and logic operators there are new PML 2 methods available:

Numeric operators: + - / *
 Numeric functions: SIN COS TAN SQR POW NEGATE ASIN ACOS ATAN LOG ALOG ABS INT NINT

PML 1 style Comparison operators: LT GT EQ NE LE GE
 PML 1 style Logic operators: NOT AND OR

PML 2 style Comparison methods: .lt() .gt() .eq() .neq() .leq() .geq()
 PML 2 style Logic methods: .not() .and() .or()

The PML 2 comparison methods are available on any object-type variable, providing it is compared to a variable of the same type. The PML 2 logic methods are available on the **BOOLEAN** object

 For further examples, refer to *PDMS Software Customisation Reference Manual*:

Some examples of expressions in use:

```
!s = 30 * sin(45)
!t = pow(20,2)                (raise 20 to the power 2 (=400))
!f = (match(name of owner, |LPX|)gt 0)
```

3.7.2 Operator Precedence

When PDMS reads an expression, there is a precedence that applied to it. This should be considered when writing expressions. The order is as follows:

()	e.g.	60 * 2 / 3 + 5	= 45
* /		60 * (2 / (3 + 5))	= 15
+ -			
EQ NE GT LT GE LE			
NOT			
AND			
OR			

3.7.3 PML 2 expressions

PML 2 expressions may be of any complexity and may derive the required values from PML Functions and Methods and include Form gadget values, object members and methods. For example:

```
!newValue = !!myFunc(!OldVal) * !!form.gadget.val / !myArray.method()
```

3.8 Arrays

An **ARRAY** variable can contain many values, each of which is called an **ARRAY ELEMENT**.

An array is created by defining one of its array elements or it can be initialised as an empty array (i.e. **!x = ARRAY()**). If an **ARRAY ELEMENT** is itself an **ARRAY**, this will create a **Multi-dimensional ARRAY**. For example, type out the following onto the command window to define an array:

```
!x[1] = |ABCD|
!x[2] = |DEFG|
!y[1] = |1234|
!y[2] = |5678|
!z[1] = !x
!z[2] = !y
```

To query the information about !z, type **q var !z**. This will return the following information:

```
<ARRAY>
  [1] <ARRAY> 2 Elements
  [2] <ARRAY> 2 Elements
```

To find out more information about the elements within the Multi-dimensional array, type **q var !z[1]** or **q var !z[2][1]**

3.9 Concatenation Operator

Values to be concatenated are automatically converted to **STRING** by the '&' operator. Type the following onto the command window and compare this against the results of typing **!d = !a + !b**

```
!a = 64
!b = 32
!m = |mm|
!c = !a & !b & !m
q var !c
```

3.10 DO loop

A **DO** loop is a way of repeating PML macro, allowing pieces of code to be run more than once. This is useful as it allows code to be reused and reduces the overall number of lines. As an example, try the provided file **%PDMSUI%\examples\doLoop.mac**:

```
DO !loopCounter TO 10
  !value = !loopCounter * 2
  q var !loopCounter !value
ENDDO
```

In the above example, as the loop runs, values of !loopCounter and !value will be printed to the command line for the full range of the defined loop. The step of the loop can be altered by adding **FROM** and **BY** to the loop definition. For example:

```
DO !loopCounter FROM 5 TO 10 BY 2
  !value = !loopCounter * 2
  q var !loopCounter !value
ENDDO
```

3.10.1 DO loops with BREAK

If you need a loop to run until a certain condition is reached, a **BREAK** command will exit the current loop. This can be used in conjunction with an infinite loop or with a loop with a range. As an example, try the provided file %PDMSUI%\examples\doBreak.mac:

```
DO !n
    !value = POW(!n, 2)
    q var !value
    BREAK IF (!value GT 1000)
ENDDO
```

The loop in the example will run until the **BREAK** condition is met. If the condition is never reached, then the code will run **indefinitely!** A DO loop of 1 to 100000 could be used instead as it has an end and won't require PDMS to be crashed to exit the macro

The **BREAK** command can also be called from within a normal **IF** construct. This is typically done if multiple break conditions need to be considered. For example:

```
IF (!value GT 1000)
    BREAK
ENDIF
```

3.10.2 DO loops with SKIP

It is possible to skip part of the DO loop using the **SKIP** command. This could be useful if parts of a number sequence needs to be missed. As an example, try the provided file %PDMSUI%\examples\doSkip.mac:

```
DO !n FROM 1 TO 25
    SKIP IF (!n LE 5) OR (!n GT 15)
    q var !n
ENDDO
```

The SKIP command can also be called within a normal IF construct (as the **BREAK** command)

3.10.3 DO INDEX and DO VALUES

DO INDEX and **DO VALUES** are ways of looping through arrays. This is an effective method for controlling the values used for the loops. Typically values are collected into an ARRAY variable then looped through using the following:

```
DO !X VALUES !ARRAY    $* !X takes each ARRAY element
DO !X INDEX !ARRAY      $* !X takes a number from 1 to !ARRAY size
```

Try the provided file %PDMSUI%\examples\doArray.mac. The **COLLECT** syntax is discussed in chapter 7

```
VAR !zones COLL ALL ZONES FOR SITE
VAR !names EVAL NAME FOR ALL FROM !zones
q var !names

DO !x VALUES !names
    q var !x
ENDDO

DO !x INDEX !names
    q var !Names[!x]
ENDDO
```

3.11 IF Statements

An IF statement is a construct for the conditional execution of commands. The commands within the statement will only be run if the conditions are met. In the following example, the code within the IF construct is only run if the expression is **TRUE** (i.e. !number is real and less than 0):

```
IF ( !number LT 0 ) THEN
    !negative = TRUE
ENDIF
```

The expression can be written in any form, providing the answer is **BOOLEAN**. For example, the following is the same as above but written in a PML 2 style:

```
IF ( !number.lt(0) ) THEN
```

If the value itself is already **BOOLEAN**, a comparison does not need to be made. For example:

```
!booleanVariable = TRUE
IF ( !booleanVariable ) THEN
```

Or **IF (!!exampleFunction()) THEN** where !!exampleFunction() returns a **BOOLEAN** result

3.11.1 IF, ELSEIF and ELSE Statements

An IF construct can be extended by adding additional conditions. This is done by adding some **ELSEIF** or **ELSE** statements to it. When an **IF** construct is encountered, PML will evaluate its first condition. If the condition is **FALSE**, PDMS will look to the next **ELSEIF** condition.

Once a condition is found to be **TRUE**, that part of the code will be run and then the IF construct is complete. If an **ELSE** condition is added, this portion of code will only be run if no other conditions are met. This is a way of ensuring some code runs as part of the construct. As an example, try the provided file %PDMSUI\examples\numCheck.mac. Run the macro with one real parameter for the comparison.

```
IF ($1 EQ 0) THEN
    $p Your value is zero
ELSEIF ($1 LT 0) THEN
    $p Your value is less than zero
ELSE
    $p Your value is greater than zero
ENDIF
```

 *The ELSEIF and ELSE commands are optional, but there can only be one ELSE command in an IF construct.*

3.12 Branching

PML provides a way of jumping from one part of a macro to another using the **GOLABEL** syntax.

```
LABEL /FRED
...
Some PML code
...
GOLABEL /FRED
```

The next line to be executed after **GOLABEL /FRED** will be the line following **LABEL /FRED**, which could be before or after the **GOLABEL** command. The name of the label can be up to 16 characters (excluding the leading slash)

 *The use of this method should be limited as it can make code hard to read and therefore debug.*

3.12.1 Conditional Branching

It is possible to add a condition to the **GOLABEL** syntax such that the jump will only be made if the condition is **TRUE**. As an example, type out the following and save it as **c:\temp\pdmsuilconditional.mac**:

```
DO !A
$P Processing $!A
  DO !B TO 10
    !C = !A * !B
    GOLABEL /finished if (!C GT 100)
    $P Product $!C
  ENDDO
ENDDO
LABEL /finished
$P Finished with processing = $!A Product = $!C
```

If the expression **!C GT 100** is **TRUE** there will be a jump to label **/finished** and PML execution will continue with the **\$P** command. If the expression is **FALSE**, PML execution will continue with the command: **\$P Product \$!C** and go back through the DO loop.

3.13 Error handling

An error condition can occur when a command could not complete successfully. This is because of a mistake in the executed macro or function. An error normally has three effects:

- An Alert box appears which the user must acknowledge.
- An error message is outputted to the command line together with a trace back to the error source.
- Any current running PML macros and functions are abandoned.

3.13.1 Error codes

When an error occurs, an error code is returned to the user along with a message. The following example error is caused when trying to create an EQUI element in the wrong part of the hierarchy.

(41,8) ERROR – Cannot create an EQUI at this level.

Where 41 is the program section which identified the error and 8 is the error code itself.

3.13.2 Error Handling using the HANDLE syntax

If the input line which caused the error was part of a PML macro or function, the error may optionally be **HANDLED**. This allows the designer of the macro to limit the errors the user will experience.

As an example, try the provided file **%PDMSUI%\examples\errorTest.mac**. First run the macro at a **SITE** element, then at a **ZONE** element and then again at the same **ZONE**. Compare the return printed lines in the command window.

```
NEW EQUI /ABCD
HANDLE (41, 8)
  $p Need to be at a ZONE or below
ELSEHANDLE (41, 12)
  $p That name has already been used. Names must be unique
ELSEHANDLE ANY
  $p Another error has occurred
ELSEHANDLE NONE
  $p Everything OK. EQUI created
ENDHANDLE
```

Notice how the **HANDLE** syntax can differentiate between specific error codes and how it is possible to capture all errors or only run if no error occurs.

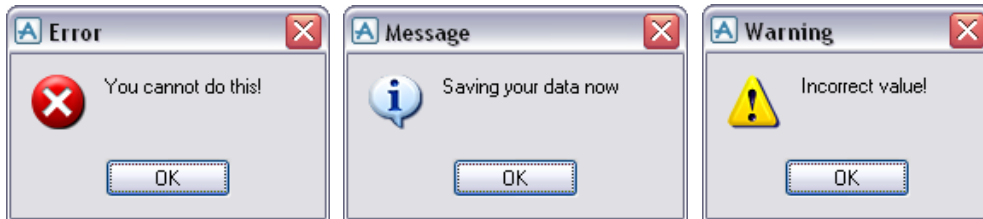
3.14 Alert Objects

Alert objects allow user-controlled pop-up forms to be used, providing feedback to the user. The choice on which form type is used and whether one is needed at all is down to the PML designer.

3.14.1 Alert Objects with no return value

There are 3 types of alert with no return value:

```
!!alert.error( |You cannot do this!| )  
!!alert.message( |Saving your data now| )  
!!alert.warning( |Incorrect value!| )
```



By default, all alert forms appear with the relevant button as near to the cursor as possible. To position an alert specifically, X and Y values can be specified as a proportion of the screen size.

```
!!alert.error( |You cannot do this!| , 0.25, 0.1)
```

3.14.2 Alert Objects that return value

There are three types of alert which return a value. This value can be subsequently used to as a indication of the decision the user has made (i.e. as part of a IF statement).

3.14.2.1 Confirm Alerts

A Confirm alert returns a string of 'YES' or 'NO'

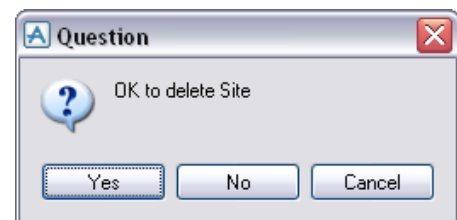
```
!answer = !!alert.confirm( |Are you sure!| )  
q var !answer
```



3.14.2.2 Question Alerts

An Answer alert returns a string of 'YES' or 'NO' or 'CANCEL'

```
!answer = !!alert.question( |OK to delete Site| )  
q var !answer
```



3.14.3 Input Alerts

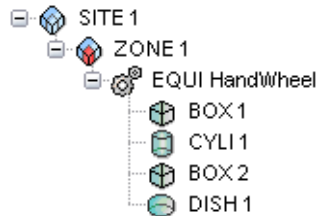
An Input alert returns the value entered by the user. If the user does not enter a value, then the default value (set at definition) is returned.

```
!answer = !!alert.input( |Enter Width of  
Floor|,|100| )  
q var !answer
```



Exercise 2 - The centre of a Handwheel

- 1
 - Write a macro that will produce the centre of a handwheel (shown to the right)
 - It shall be constructed from 4 primitives: 2 boxes, a cylinder and a dish and should create the following hierarchy:

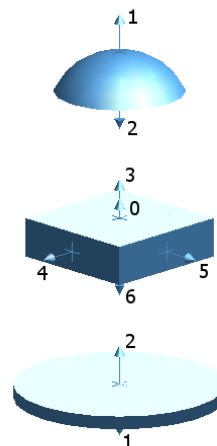


- 2
 - The macro shall create the primitive with the following fixed sizes:

First BOX	X Length = 100; Y Length = 100; Z Length = 100
CYLI	Diameter = 80; Height = 5
Second BOX	X Length = 50; Y Length = 50; Z Length = 15
DISH	Diameter = 50; Height = 15
 - The primitives should be unnamed, but sit below an EQUI called HandWheel

- 3
 - Let the first BOX be created without specifying a position or orientation.
 - Use the CONN syntax to position the next primitive to the first BOX. For example:

CONN P1 TO P1 OF PREV
 - This will connect PPoint 1 (P1) of the current primitive to P1 of the primitive created previously.
 - To help with connecting the primitives, refer to the adjacent diagram showing an exploded version of the equipment, with the PPoints identified.

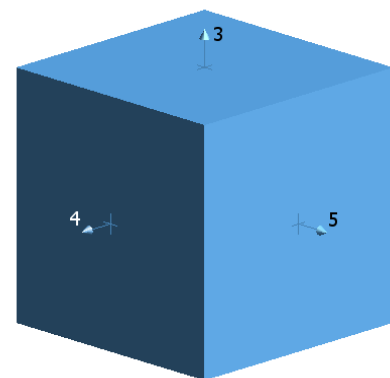


- 4
 - You may wish to add some other syntax to the macro:

To add the HandWheel to the drawlist
ADD /HandWheel

To set the limits of the view to the HandWheel
AUTO /HandWheel

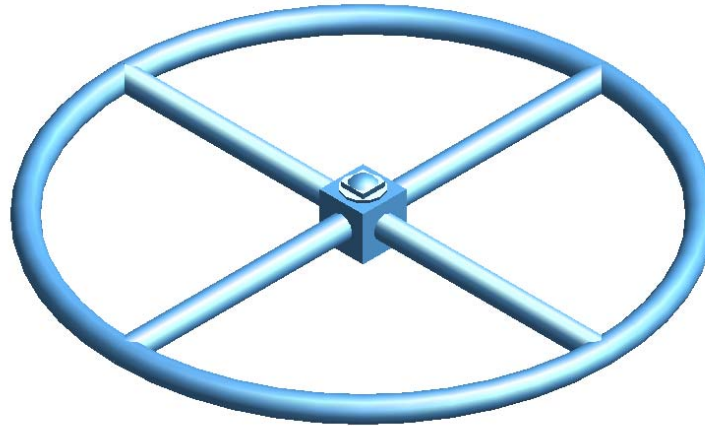
To completely clear the drawlist
REM ALL
 - Save the file as **c:\templ\pdmsuilex2.mac** and run it on the command line by typing **\$m/%PDMSUI%\ex2.mac** or by dragging and dropping the file into the command window



An example of the completed macro can be found in Appendix B

Exercise 3 - The full Handwheel

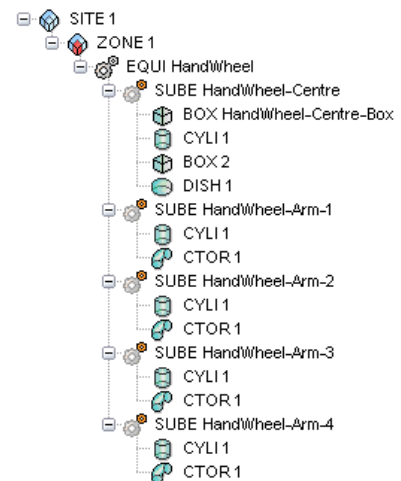
- 1
 - Extend the previous macro to build the remaining parts of the HandWheel (shown below).



- Information about primitives can be found in Appendix A

- 2
 - It is possible to build the remaining primitives individually, but as there is a rotational centre, a copy-rotate could be used.
 - To copy a BOX element (where /XXXX is another BOX. Note: the word PREV is also valid)
NEW BOX COPY /XXXX
 - To rotate the current element (where AAA is the angle and BBB the direction)
ROTATE BY AAA ABOUT BBB

- 3
 - To help with organization of the primitives of the EQUI, you may wish to add additional SUBE elements. This could help with the copy-rotate methods
 - An example hierarchy is shown to the right, but this will depend on your macro



- 4
 - Turn the macro into a parameterized macro. Pass it two parameters (1) the name of the EQUI and (2) the outside diameter of the HandWheel
 - Think about how the sizes of the primitives will relate to the outside diameter. You may need to do some calculations before the value can be used
 - As parameters are used, set the default values

- 5
 - If any errors occur in your macro, consider error handling or changing the macro so that the error cannot happen
 - Save the file as **c:\temp\pdmsuilex3.mac** and run it on the command line by typing **\$m/%PDMSUI%\ex3.mac** or by dragging and dropping the file into the command window



An example of the completed macro can be found in Appendix B

4 PML Functions

Functions are lines of PML grouped together in a file designed to do something. When called, the lines of the function are run and the intended action completed.

4.1 Functions instead of macros

Functions are designed to replace macros and can contain the same syntax that would have been placed inside a PML 1 style macro. It is intended that any future macros which are created in PML should be written as functions. The following are the reasons why:

- Functions are preloaded through the PMLLIB searchpath
- Functions can accept arguments of specific object-types
- Functions can return values of specific object-types

4.2 Creating a PML function

A function is saved as a **.pmlfnc** file under the **PMLLIB** search path. The file should be given the same name as the function. For example, if the function is called **!!exampleFunction** then the file should be called **exampleFunction.pmlfnc**.

A function will typically return a value, although arguments are optional. The object-type of the returned information needs to be defined, and is done so at the end of the define function line.

The following is an example of a simple function designed to return the full name of the current element. It is an example of a **RETURN** function with **NO ARGUMENTS**

```
define function !!nameCE() is STRING
  !ce = !!CE.flnn
  return !ce
endfunction
```

After a **PML REHASH ALL**, the function can be called by typing:

```
!name = !!nameCE()
```

After running, variable **!name** will be a string holding the full name of the current element.

If a function is defined with arguments, then these arguments will become local variables available within the function. These can then be used in expression, to control the function etc. The following is an example of a **RETURN** function with an **ARGUMENT**:

```
define function !!area(!radius is REAL) is REAL
  !circleArea = !radius.power(2) * 3.142
  return !circleArea
endfunction
```

After definition, the function can be used as part of an expression as it returns a real object. This means that common expressions/calculations can be written as a function and used as required.

```
!height = 64
!cylinderVolume = !!area(2.3) * !height
q var !CylinderVolume
```

4.3 PML procedures

A function that does not return a value is typically described as a PML procedure. This means that as there is no return, the object-type does not need to be defined. The following example of a **NON-RETURN** function with **NO ARGUMENTS**:

```
define function !!setNameOfCE(!name is STRING)
    !!CE.name = !name
endfunction
```

The following is an example of a **NON-RETURN** function with **ARGUMENTS**:

```
define function !!setBoxPrimitiveSize(!x is REAL, !y is REAL, !z is REAL)
    if !!ce.type.eq(|BOX|) then
        !!CE.xlen = !x
        !!CE.ylen = !y
        !!CE.zlen = !z
    endif
endfunction
```

 Notice how both the examples use the **!!CE** object (a global object which represents the current element). PML procedures are typically used to interact with defined global variables

4.4 Making use of methods on PML objects

After a variable has been defined as a specific object-type, the methods of the object will be available on the variable. Making use of these methods can help manipulate the information within a function to ensure the correct outcome.

From the example on the previous page, **!radius** is defined as a **REAL** object. Therefore it has access to all the methods of a real object. The example makes use of the **.power()** method, and method which takes a real argument and raises the !variable to the power of the argument. The method returns the answer as a real object.

 When working with built-in objects, refer to PDMS Software Customisation Reference Manual for the available methods and information about them

If you are working with different object-types, it is possible to switch between types. For example, there may be a need for a **REAL** to be seen as a **STRING** for use in an expression. The following would give an error:

```
!value = |56|
!result = !value * 2
```

As a **STRING** object cannot be multiplied by a **REAL**, an error will be returned. To avoid this error, the **.real()** methods can be used to return a **STRING** from a **REAL**. Note, the original variable remains a **STRING**, but it is seen as a **REAL** for the expression:

```
!value = |56|
!result = !value.real() * 2
```

 There are equivalent methods available for the other standard objects, refer to PDMS Software Customisation Reference Manual.

4.4.1 Method information

For each object-type in the Software Customisation Reference Manual, there is a table which lists the available methods and supporting information. For each method there will be:

NAME	The name of the method or member. For example, a REAL object has a method named Cosine. If there are any arguments, they are indicated in the brackets () after the name. For example, the REAL object has a method named BETWEEN which takes two REAL arguments.
RESULT	The type of value returned by the method. For example, the result of the method Cosine is a REAL value. Some methods do not return a value: these are shown as NO RESULT.
PURPOSE	This column tells you what the member or method does along with other information about the method or members.

① Note that for the system-defined PDMS object types, the members can be listed by using the .attributes() method and the methods can be listed using the .methods() method

The following are some examples of **ARRAY** object methods to explain the different styles:

!numberOfNames = !nameStrings.size()

This method returns the number of elements currently in the array. This is an example of a **RESULT** method with **NO-EFFECT** on the original object.

!nameStrings.clear()

This method deletes the contents of the array, but not the array. This is an example of a **NO RESULT** method which does **EFFECT** the original object.

!newNameArray = !nameStrings.removeFrom(5,10)

This method result removes 10 elements (starting at element 5) from the array. These elements are then returned by the method. This is an example of a **RESULT** method which does **EFFECT** the original object. If you need to remove part of the array and do not need it, then it does not need assigning to a variable (e.g. just type **!nameStrings.removeFrom(5,10)**)

4.4.2 Method Concatenation

One of the advantages of object-orientated programming is that methods can be built-up within a single line of code. This means that complex manipulations can be made in fewer lines of PML. This process will only work if the previous method returns a suitable object for the following method. For example:

```
!line = 'hello world how are you'
!newline = !line.upcase().split().sort()
q var !line !newline
```

```
<STRING> 'hello world how are you'
<ARRAY>
  [1] <STRING> 'ARE'
  [2] <STRING> 'HELLO'
  [3] <STRING> 'HOW'
  [4] <STRING> 'WORLD'
  [5] <STRING> 'YOU'
```

In this example, the first two methods are valid for **STRING** objects. The **.split()** method returns an **ARRAY** object so the following method has to be a valid method for an **ARRAY** object.

4.5 Using the !!CE Object

!!CE is a special GLOBAL PML variable that always points to the current PDMS element. It is a **DBREF** object and its members are the attributes of the current element. Type **q var !!CE** onto the command line and compare it against the elements attributes (**Query>Attributes...**). You will notice the returned attribute information is the same of the members list of the !!CE object.

This means that the !!CE object can be used to assign the values of attributes to !variables. For example:

```
!branchHeadBore = !!CE.hbore
```

This assigns the **HBORE** attribute (taken from the current **BRAN** element) to the variable **!BranchHeadBore** making it real.

 *It will be necessary to check that HBORE is a valid attribute of the current element before running this line. It may cause an error.*

If the !!CE object member is an object itself, that object could also have members so further information be obtained. For example, obtain the east coordinate of a head of a **BRAN** element, either of the two following can be used:

```
!headPosition = !!CE.hpos  
!headEasting = !headPosition.east
```

Or:

```
!headEasting = !!CE.hpos.east
```

If the !!CE object member is an object with built-in methods, then these methods can also be called:

```
!PosWRTValve = !!CE.hpos.wrt(ZONE)           returns a POSITION object w.r.t the owning ZONE
```

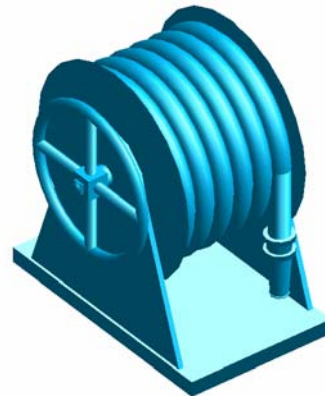
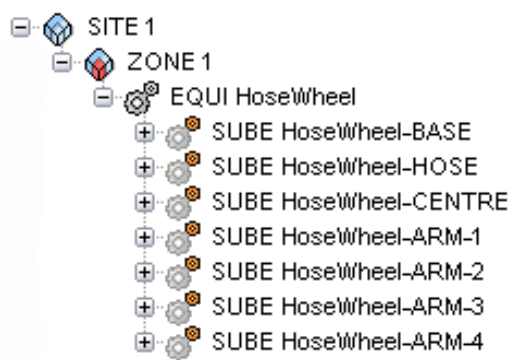
This process can also be reversed and values can be applied to the attributes of the !!CE. This means that it is possible to record the current value of an attribute, modify and reassign back to the CE. For example, type out the following onto the command line:

```
q POS  
!position = !!CE.pos  
!position.up = !position.up + 2000  
!!CE.pos = !position  
q POS
```

These lines will have moved the CE up by 2000. This same logic can be applied to other attributes, providing the object-type is considered.

Exercise 4 – Convert the HoseReel macro into a PML procedure

- 1
 - You have been provided with an old PML 1 style macro that builds a Hose Reel as a piece of equipment. The macro is old and has not been maintained. There are at least 5 problems with it.
 - Debug the existing macro and convert it into a new PML procedure with **NO ARGUMENTS**



- 2
 - Save the new file as **ex4.pmlfnc** under the PMLLIB searchpath.
 - You may wish to create a new folder to keep the exercises separate from the examples
 - Update the PMLLIB search path by typing **PML REHASH ALL**
 - Test the function in the command window by typing **!!ex4()**
- 3
 - Update the procedure and given it **TWO ARGUMENTS** which will represent the name of the EQUI and the diameter of the Hose Reel.
 - Check the updated procedure to make sure the Hose Reel is created as expected. Consider error handling while you check the procedure.



An example of the completed procedure can be found in Appendix B

5 Forms

5.1 Forms Are Global Objects

Forms are objects which are represented by **GLOBAL VARIABLES**. The gadgets on a form are objects that are **MEMBERS** of the form object. As it is a global variable, once defined, the information held within a form is available to the rest of the program.

To find out information about the form, it can be queried as if it was an object. For example, show the Graphics Settings form (**Settings>Graphics...**)

This form could also have been shown using **show !!gphsettings**. The **SHOW** syntax loads and displays a form. If you wish to load the form, but not see it, you could type **load !!gphsettings**

Type **q var !!gphsettings** onto the command line. The information is a list of the members of the form. Compare this list against some of the gadgets on the form. As these gadgets are objects as well, we can find out further information.

The first gadget listed is called **OK** (representing the OK button on the form). To find out information about it, type **q var !!gphsettings.ok**. Specific information about the gadget can be queried directly e.g.:

```
q var !!gphsettings.ok.tag
q var !!gphsettings.ok.val
q var !!gphsettings.ok.active
```

 To get the name of a shown form, type **show !!pmlforms**. This form can list shown forms

5.2 Dynamic loading of Objects, Forms and Functions

When a PML object is used for the first time, it is loaded automatically. This applies to both **FORMS** and **OBJECTS** e.g.:

```
!Person = object PRIMEMINISTER()
show !!MyInputForm
```

Once an object is loaded by PDMS, the definition is held by PDMS. This means that if the object definition is changed outside PDMS whilst it is loaded, the form will need to be reloaded. To reload a form type **pml reload form** or object followed by the object name e.g.:

```
pml reload form !!MyInputForm
pml reload object PRIMEMINISTER
```

If a new file is created whilst PDMS is open, the file will not be mapped (even if it is saved in an appropriate file path). To remap files type **pml rehash**. This will remap all the files within the first file path in the PMLLIB variable.

The remap all files in all the file paths, type **pml rehash all**

5.3 Defining a form

A form is defined within a .pmlfrm file and should be saved under the **PMLLIB** searchpath. The file will define the form, its members and any associated methods. The file will first contain the form setup (gadgets, members etc) and then the methods, following the below format:

```
setup form !!exampleForm
...
exit

define method .init()
...
endmethod
```

5.3.1 Using the .net framework

Form objects make use of the .net framework. This allows the following features to be included:

- Docking forms (automatic resizing)
- Anchoring gadgets (useful when resizing)
- Multicolumn lists
- Tabs on Forms
- Menu Additions to existing applications:
- Toolbar Additions to existing applications
- DBAR Conversions

Although it is now possible to dock forms, it does not apply to every form. There are some rules to consider when deciding if a form should dock:

- Does the form need to remain open?
- Is the form used heavily
- If a form has an OK/Cancel button, it should not need docking
- If a form has a menubar, it CANNOT be docked

To declare a form as able to dock, this has to be done on the top of the definition. By including **dialog dock**, we are stating that the form will have the ability to dock:

```
setup form !!exampleForm dialog dock left
```

To define a floating form that is able to dock, use the following line:

```
setup form !!exampleForm dialog resizable
```

To define a form of a certain size in the centre of the screen, use the following line:

```
setup form !!exampleForm document at xr 0.5 yr 0.5 size 100 100
```

If no additional details are included, the default form creation is a Dialog, non-resizable, size adjusted automatically to fit contents form. A form will always be as big as it needs to be to display the contents.

 *For more examples, refer to the FORM object in the PDMS Software Customisation Reference Manual*

5.3.2 Showing and hiding Forms

Forms are found through the **PMLLIB** search path, there is no need to load them individually. To show a form use **show !!formname**. This will load the form definition and show it in one go!

Sometimes it is useful to have the form loaded making it visible (e.g. to refer to stored information). A form can be loaded (but not shown) by typing **loadform !!formname**

5.3.3 Built-in methods for Forms

Although it is possible to define user-defined methods within user-defined forms (discussed later), all **FORM** objects have built-in methods available. Try on the following on the command line:

```
!!gphsettings.show()
q var !!gphsettings.shown()
!!gphsettings.hide()
q var !!gphsettings.shown()
```

In this example, the **.show()** method is used to show the form, the **.shown()** method returns whether the form is shown and the **.hide()** method hides the form.

 For more examples, refer to the *FORM* object in the *PDMS Software Customisation Reference Manual*

5.4 Callbacks

If an object has a **CALLBACK** member, it can be given a callback string. This means that if the user interacts with the object, an action can be performed. The callback can do one of three things (1) show a form (2) execute a command directly or (3) run a function or method

A **FORM** object has some special callbacks which are used when the form completes certain actions (e.g. is shown, is closed). The following example demonstrates the various callbacks on a **FORM** object. Show the form by typing **show !!exampleCallback**. Test the form by showing and closing it.

```
setup form !!exampleCallback
  !this.formTitle      = |Callback Example|
  !this.initCall       = |!this.init()|
  !this.firstShownCall = |!this.firstShown()|
  !this.okCall         = |!this.okCall()|
  !this.cancelCall     = |!this.cancelCall()|
  !this.quitCall       = |!this.quitCall()|
  !this.killingCall    = |!this.killCall()|

  button .ok          | OK |          OK
  button .cancel      |Cancel| at x20 CANCEL
exit

define method .exampleCallback()
  $p Constructor Method
endmethod

define method .init()
  $p Initialise Method
endmethod

define method .firstShown()
  $p First Shown Method
endmethod

define method .okCall()
  $p OK Method
endmethod

define method .cancelCall()
  $p Cancel Method
endmethod

define method .quitCall()
  $p Quit Method
endmethod

define method .killCall()
  $p Kill Method
endmethod
```



The callbacks on a form object can run any command syntax, global function, local method. In this example, the callbacks call local methods of similar names. They could have called any method, even the same method. The only important part is that there is a local method of that name

 *Callbacks can reference any method/function/command*

These event callbacks are built-in members of a **FORM** object. They are typically defined at the top of the form definition. There are 7 main event callbacks available in PDMS.

- The **CONSTRUCTOR** method was called when the form loaded (notice, it has the same name as the form).
- The **INITCALL** method is called everytime the form is shown (perfect for setting default values)
- The **FIRSTSHOWNCALL** method is called the when the form is activated (first shown)
- The **OKCALL** method is called by any button gadget with the OK in its definition
- The **CANCELCALL** method is called by any button gadget with CANCEL in its definition
- The **QUITCALL** method is called by clicking the close button on the form.
- The **KILLINGCALL** method is called when the form is unloaded (killed or reloaded)

 *If no callbacks are defined for OKCALL, CANCELCALL and QUITCALL, the default is to hide the form only*

5.5 Form Gadgets

There are many kinds of form gadgets, each an object that will have its own **MEMBERS** and **METHODS**. When you are defining gadgets on a form, there are two common aims:

- Define the area to be taken up on the form
- Define the action to be taken if the gadget is interacted with

It is the position and size of the gadget that determines the area taken up and its action is defined by its **CALLBACK** member.

5.5.1 Built-in members and methods for Gadgets

As gadgets are objects, there are a variety of useful members and built-in methods that can be used. Based on the previous example, type the following onto the command line:

To grey-out the OK button

```
!!exampleCallback.ok.active = FALSE
```

To hide the CANCEL button

```
!!exampleCallback.cancel.visible = FALSE
```

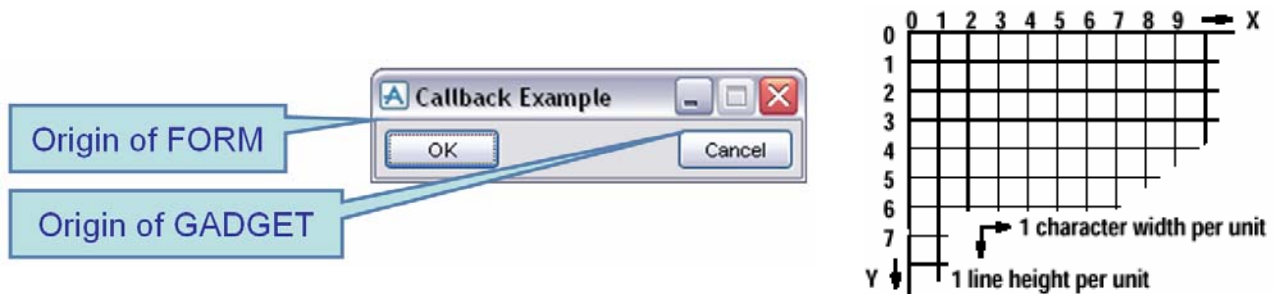
Apply a tooltip to the OK button

```
!!exampleCallback.ok.setTooltip(|This is an OK button|)
```

 *For further information, PDMS Software Customisation Reference Manual*

5.5.2 Gadget positioning

Gadgets are positioned on a form from top left using the **AT** syntax. The **AT** syntax defines the origin of the gadget in relation to the owning object (i.e. FORM or FRAME). .



Gadgets can be positioned explicitly or in relation to other objects. When referring to other objects, there 6 known positions on a gadget: **XMIN**, **XCEN**, **XMAX**, **YMIN**, **YCEN** and **YMAX**. These refer to fixed position in the x and y directions on the referenced gadget. A gadget can be thought of as an enclosing box that will enclose the geometry of the gadget (including its name tag if specified).

To position a gadget at a known position use:

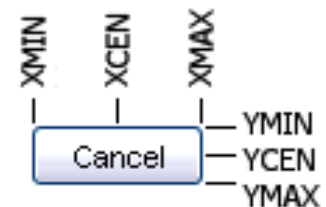
at x 0 y 0

To position the above CANCEL button using the XMAX and YMIN of OK use:

at xmax.ok + 10 ymin.ok

To position a DISMISS button in the bottom corner of a form use:

at xmax form - size ymax form



For more positioning syntax, refer to the PDMS Software Customisation Reference Manual

The available syntax and its order can be derived by referring to the **SYNTAX GRAPHS** in the reference manual. These diagrams are available for most Gadgets and can be used when initially defining them. The picture below is an example of the Syntax Graph for gadget positioning (**<fgpr1>** refers to another graph):

```
>-- <fgpos> -- AT --+ val val -----
                    +- X val -----
                    +- XMIN -.
                    +- XCEN -|
                    +- XMAX --+ <fgpr1> -|
                    \-----+ Y val -----
                              +- YMIN -.
                              +- YCEN -|
                              +- YMAX --+ <fgpr1> -|
                              \----->
```

Refer PDMS Software Customisation Reference Manual for more guidance on these graphs

5.5.2.1 Position Gadgets using the path command

The path command can be used to define the logical position of subsequent gadgets. This method has been superseded by the previous method and has been included for information.

After a gadget has been defined, the next gadget is positioned based on a PATH, HDIST or VDIST and HALIGN or VALIGN. As an example, see the picture below:

```

Button .But1                      $* default placement
PATH down
HALIGN centre
VDIST 2

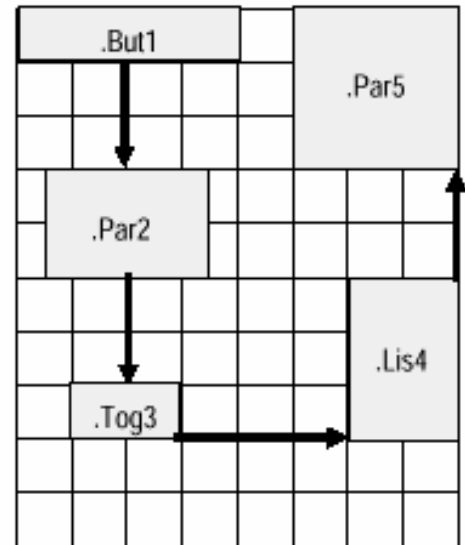
paragraph .Par2 width 3 height 2 $* auto-placed
toggle .Tog3                      $* auto-placed

PATH right
HDIST 3
VALIGN bottom

List .Lis4 width 2 height 3       $* auto-placed

PATH up
HALIGN right

Paragraph .Par5 width 3 height 3 $* auto-placed
  
```



5.5.3 Docking and Anchoring Gadgets

After a gadget has been loaded its size and position are locked. To modify the position or size manually, the alterations have to be made in the file and the form reloaded. As forms can be resized, it is necessary for gadgets to move/resize so the layout of the form remains.

There are two available syntax definitions that can help: **DOCK** or **ANCHOR**. This syntax should be included when defining a gadget and can be included if **<fgdock>** or **<fganch>** are included in the syntax graph of the gadget. Once a gadget is declared as Anchored or Docking it will remain so. If a change is required, the form definition should be updated and the form reloaded.

i The **DOCK** and **ANCHOR** are mutually exclusive so only one is defined per gadget

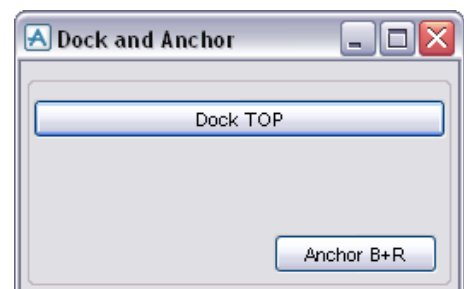
- **ANCHOR** controls the position of an edge of the gadget relative to the corresponding edge of its container. For example, if a DISMISS button is anchored to the Right + Bottom, it will remain the same distance from the bottom, right of the form if it is resized.
- **DOCK** forces the gadget to fill the available space in a certain direction. For example, if a list is docked to the left, it will maintain its width, but its height will change it fill its container. **DOCK FILL** is very useful for ensuring a gadget is the full size of its container.

Refer PDMS Software Customisation Reference Manual for the Syntax Graphs.

Show the example form by typing **show !!exampleDocking** Observe how the form behaves when it is resized. Try altering the directions which the gadgets are docked.

```

setup form !!exampleDocking dialog resizable
!this.formTitle = |Dock and Anchor|
!buttpos = |xmax.f1 - size ymax.f1 - size|
frame .f1 anchor ALL width 30 height 5
  button .but1 |Dock TOP| dock TOP
  button .but2 |Anchor B+R| at $!buttpos anchor R+B
exit
exit
  
```



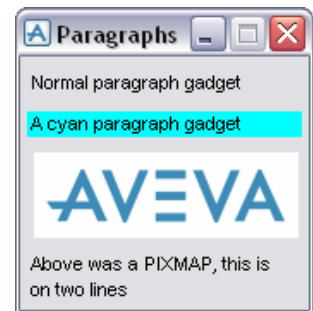
5.6 Paragraph Gadgets

Paragraph gadgets are simple named gadgets which allow a piece of **TEXT** or a **PICTURE** (PIXMAP) to be displayed on a form. It is a passive gadget that cannot be selected by the user so has no callback. Paragraph gadgets are typically used display information or images.

Show the example by typing **show !!exampleParagraphs**. Notice how the value of the last gadget was set during the **CONSTRUCTOR** method, it was only given a size to reserve the space during definition.

```
setup form !!exampleParagraphs
!this.formTitle = |Paragraphs|
para .para1 text |Normal paragraph gadget| width 20
para .para2 at x0 ymax.para1 backg 6 text |A cyan paragraph gadget| width 20
para .para3 at x0 ymax.para2 pixmap width 154 height 50
para .para4 at x0 ymax.para3 + 0.5 text || wid 20 hei 2
exit

define method .exampleParagraphs()
-- Update the displayed text using CONSTRUCTOR method
!this.para4.val = |Above was a PIXMAP, this is on two lines|
-- Use built-in method to apply picture
!this.para3.AddPixmap(|c:\temp\pmllib\icons\aveva.png|)
endmethod
```



Refer to the Reference Manual and Guide for more information about paragraph gadgets

5.7 Button Gadgets

Button gadgets are typically used to invoke an action or to display a child form. Its **CALLBACK** can call a **LOCAL METHOD**, **GLOBAL FUNCTION** or **OBJECT METHOD**. If a callback and a child form are both specified, the callback command will be run before the child form is displayed.

Show the example by typing **show !!exampleButtons**. A linklabel style button is now available and has the same appearance as a hyperlink. Notice how the value of the toggle button changes.

```
setup form !!exampleButtons
!this.formTitle = |Buttons|
button .butt1 |Normal|
button .butt2 linklabel |LinkLabel| wid 6
button .butt3 |Green| backg 5
button .butt4 |Deactive|
button .butt5 |form| form !!exampleParagraphs
button .butt6 toggle backg 4 call |!this.check()| pixmap wid 31 hei 21
exit

define method .exampleButtons()
-- Deactivate butt3 in CONSTRUCTOR method
!this.butt4.active = FALSE
-- Use built in method to apply pictures
!this.butt6.AddPixmap(|c:\temp\pmllib\icons\off.png|, |c:\temp\pmllib\icons\on.png|)
endmethod

define method .check()
-- Return the toggle button value to the command line
!check = !this.butt6.val
$P $!check
endmethod
```



Refer to the Reference Manual and Guide for more information about button gadgets

5.8 Text Entry Gadgets

A text input gadget provides the user a way of entering a single value into PDMS. A TEXT gadget needs two aspects to be defined:

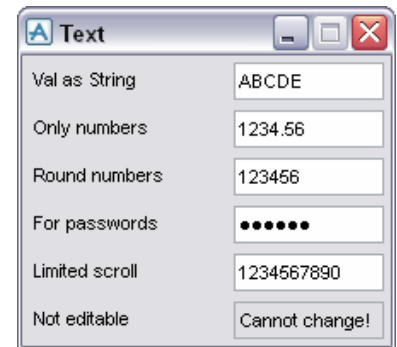
- **WIDTH** – determines the displayed number of characters. An optional scroll width can also be specified
- **TYPE** – determines the type of the variable created when inputting a value. This is important when PML uses the variable. You may also supply a **FORMAT** object (explained below) to format the value entered (e.g. 2 d.p number)

Show the example by typing **show !!exampleTexts**. Test the various text gadgets by entering values. The benefit of making a text gadget uneditable (rather than deactivated) is so the user can select its contents. A de-active text box will be full greyed out and unselectable.

```

setup form !!exampleTexts
  !this.formTitle = |Text|
  path down
  text .txt1 |Val as String | width 10 is STRING
  text .txt2 |Only numbers | width 10 is REAL format !!REALFMT
  text .txt3 |Round numbers | width 10 is REAL format !!INTEGERFMT
  text .txt4 |For passwords | width 10 NOE is STRING
  text .txt5 |Limited scroll | width 10 scroll 1 is STRING
  text .txt6 |Not editable | width 10 is STRING
exit

define method .exampleTexts()
  !this.txt6.val = |Cannot change!|
  -- Make txt6 uneditable
  !this.txt6.setEditable(FALSE)
endmethod
  
```



Refer to the Reference Manual and Guide for more information about text gadgets

5.9 Format Object

A **FORMAT** object manages the information needed to convert a number (always in mm) to a STRING. It can also be used apply a format to a text gadget. A format objects are usually defined as global variables so that they are available across PDMS. For example, type the following onto the command line:

```

!!oneDP = object FORMAT()
!!oneDP.dp = 1
q var !!oneDp
  
```

There are four standard **FORMAT** objects which are already defined in standard PDMS:

!!distanceFmt	For distance units
!!boreFmt	For Bore Units
!!realFmt	To give a consistent level of decimal places on real numbers
!!integerFmt	To force real numbers to be integers(0 dp Rounded)

To find out more information about these **FORMAT** object, query them as global variables on the command line **q var !!boreFmt**

For example the number of decimal places displayed using **!!RealFmt** could be set **!!realFmt.dp = 6** the default value is 2.

These standard format objects are used within the forms of PDMS. Changing the definition of these objects will change the way standard product behaves.

5.10 List Gadgets

A **LIST** gadget presents an **ARRAY** of values to the user. This can be a **SINGLE** or **MULTI-DIMENSIONAL ARRAY**

All the values in the gadget are set by assigning an **ARRAY**. **ARRAY** variables can be applied to a **LIST** at any time. The choice between **MULTIPLE**, **COLUMN** and **SINGLE** has to be made when the gadget is defined. The values within a **COLUMN** list (headings and values) are set by using built-in gadget methods.

Show the example by typing **show !!exampleLists**. Test the different lists by trying to select rows.

```

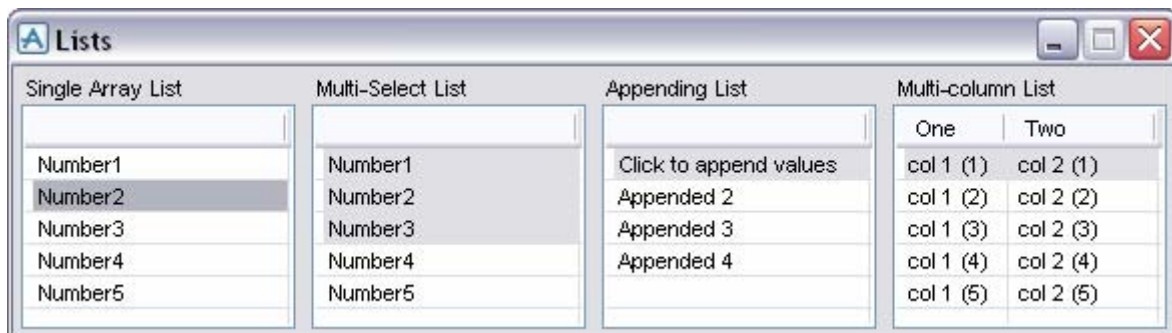
setup form !!exampleLists
  !this.formTitle = |Lists|
  !vshap = |width 15 height 6|
  list .lst1 |Single Array List| call !|this.value()| SINGLE ZEROSEL $!vshap
  list .lst2 |Multi-Select List| MULTI $!vshap
  list .lst3 |Appending List| call !|this.append()| $!vshap
  list .lst4 |Multi-column List| $!vshap
exit

define method .exampleLists()
  do !I from 1 to 5
    !values[!I] = |Number| & !I
    !rtext[!I] = !I.string()
    do !J from 1 to 2
      !multi[!I][!J] = |col | & !J & | (| & !I & |)|
    enddo
  enddo
  !this.lst1.dtext = !values
  !this.lst1.rtext = !rtext
  !this.lst2.dtext = !values
  !single[1] = |Click to append values|
  !this.lst3.dtext = !single
  !this.lst4.setRows(!multi)
  !heading = |One Two|
  !this.lst4.setHeadings(!heading.split())
endmethod

define method .value()
  !dtext = !this.lst1.selection('Dtext')
  !rtext = !this.lst1.selection('Rtext')
  $p Selected Dtext = $!<dtext> Rtext = $!<rtext> (hidden!)
endmethod

define method .append()
  !nextLine = !this.lst3.dtext.size() + 1
  !val = |Appended | & !nextLine
  !this.lst3.add(!val)
endmethod

```



5.11 Frame Gadgets

A **FRAME** is a cosmetic gadget which is used to surround a group of similar gadgets. This helps with organisation, positioning and user experience. A frame can also be declared as a **FOLDUP**, **PANEL**, **TABSET** or even a **TOOLBAR** (when defined for the main PDMS window).

Show the example by typing **show !!exampleFrames**. The example shows the various types of frames and the members/methods available. Notice how the frames immediately below the **TABSET** are its tabs.

```

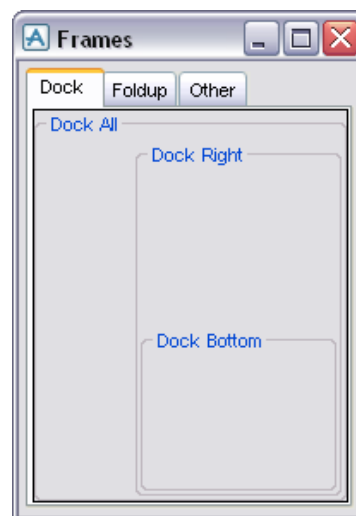
setup form !!exampleFrames dialog resizable
!this.formTitle = |Frames|
frame .tabset TABSET anchor ALL wid 15 hei 5
  frame .f1 |Dock| at 0 0 dock FILL
    frame .fA |Dock All| dock FILL
      frame .fB |Dock Right| dock RIGHT wid 10
        frame .fC |Dock Bottom| dock B hei 4
          exit
        exit
      exit
    exit
  frame .f2 |Foldup| at 0 0 dock FILL
    button .but1 linklabel |Show Panel Frame...| at 0 0 call |!this.showD()|
    frame .fD panel |Panel Frame| at x0 ymax.but1 dock FILL
      frame .fE foldup |Foldup 1| at x1 ymin.fD + 1 wid 20 hei 3
        button .but2 linklabel |Foldup Panel| call |!this.fold()|
        exit
      frame .fF foldup |Foldup 2| at xmin.fE ymax.fE wid 20 hei 3
        para .para1 text |Inside a frame|
        exit
      exit
    exit
  frame .f3 |Other| at 0 0 dock FILL
    !pos = |at xcen.fG - 0.5 * size ymax|
    frame .fG |Deactive| wid 21 hei 4
      button .but3 |Try and click!| at x 2 y 1
      exit
    button .but4 toggle |Show Frame| $!pos backg 8 call |!this.showF()|
    frame .fH |Hidden Frame| at xmin.fG ymax.but4 width.fG hei.fG
    exit
  exit
exit
exit

define method .exampleFrames()
!this.fD.visible = FALSE
!this.fE.expanded = FALSE
!this.fG.active = FALSE
!this.fH.visible = FALSE
endmethod

define method .fold()
!this.fE.expanded = FALSE
endmethod

define method .showD()
if !this.but1.val.eq(TRUE) then
!this.fD.visible = TRUE
!this.but1.tag = |Hide Panel Frame...|
else
!this.fD.visible = FALSE
!this.but1.tag = |Show Panel Frame...|
endif
endmethod

```



(continued onto next page)

```
define method .showF()
  if !this.but4.val.eq(TRUE) then
    !this.fH.visible = TRUE
    !this.but4.tag = |Hide Frame|
  else
    !this.fH.visible = FALSE
    !this.but4.tag = |Show Frame|
  endif
endmethod
```

i When creating a **FRAME** gadget, for every **FRAME** there must be an associated **EXIT**.

If insufficient exits are provided, this will cause an error and the form **WILL NOT LOAD**. As the error occurred inside the **FORM DEFINITION**, the command line will still be in form definition mode and will not function as usual. To exit this mode, type **EXIT** on the command line until an **ERROR** is received. This will mean that form definition mode has been exited and normal commands will work again.

It is good practise to provide 2 spaces when working inside a code block. This provides an easy way to spot missing exits.

5.12 Textpane Gadgets

A **TEXTPANE** gadget provides an area on a form into which a user may edit multiple lines of text and cut/paste from elsewhere on the PML screen. The inputted value is stored as an **ARRAY of STRINGS** and can be set and read from the gadget.

Show the example by typing **show !!exampleTextpane**. Choose different lines inside the textpane and press the button. The method reads the cursor position inside the gadget and it will swap that line. The textpane cannot be given a callback so any interaction has to be through something else.

```
setup form !!exampleTextpane
  !this.formTitle = |Textpane|
  !this.initCall = |!this.init()|
  text .txt1 call |!this.append()| width 30 is STRING
  button .but1 linklabel |Swap Line| at xmax.txt1 + 0.5 y 0 call |!this.swap()| wid 7
  textpane .txtp |Compiled text| at x 0 ymax wid 40 hei 5
exit
```

```
define method .init()
  !this.txt1.val = |Enter a STRING to swap|
  do !n from 1 to 10
    !val[!n] = |Text on Line Number | & !n
  enddo
  !this.txtp.val = !val
endmethod

define method .swap()
  !lineNO = !this.txtp.curPos()
  !line = !this.txtp.line(!lineNO[1])
  !this.txtp.setLine(!lineNO[1], !this.txt1.val)
  !this.txt1.val = !line
Endmethod
```



5.13 Option Gadgets

An **OPTION** gadget offers a single choice from a list of items. It may contain either **PIXMAPS** or **STRINGS**, but not a mixture. The gadget displays the **CURRENT** choice in the list. When the user presses the option gadget, the entire set of items is shown as a drop-down list and the user can then select a new item by dragging cursor to the option required. There is now also a COMBO gadget that allows the user to type in a value. If used in conjunction with an open callback (explained in Chapter XXX), it can select from the list for the user. This would be very useful when there are a long number of options.

The width of a text option gadget must be specified. A tag name is optional and is displayed to the left of the gadget. The available options is stored as an **ARRAY of STRINGS** as the gadgets **DTEXT** or **RTEXT** and can be updated by altering this array.

Show the example by typing **show !!exampleOptions**. Test the COMBO gadget by typing in part of the required colour and press enter.

```
setup form !!exampleOptions
  !this.formTitle = |Options|
  path right
  option .opt1 |Normal| width 7
  combo .opt2 |Combo| width 7
  option .opt3 |Pixmap| pixmap width 16 height 16
exit
```

```
define method .exampleOptions()
  !dtext = |Red Yellow Green Cyan Blue Purple Pink|
  !this.opt1.dtext = !dtext.split()
  !this.opt2.dtext = !dtext.split()
  do !n index !this.opt1.dtext
    !files[!n] = |/c:\temp\pmlib\icons\| & !n & |.gif|
  enddo
  !this.opt3.dtext = !files
  !this.opt3.rtext = !dtext.split()
  !this.opt2.callback = |!this.setColour(|
endmethod

define method .setColour(!gad is gadget, !event is STRING)
  if !event.eq('VALIDATE') then
    !userInput = !this.opt2.displayText()
    do !n index !this.opt2.dtext
      !chr = !userInput.length()
      !test = !this.opt2.dtext[!n].upcase().substring(1, !chr)
      if !userInput.upcase().eq(!test) then
        !this.opt2.val = !n
        break
      endif
    enddo
  endif
endmethod
```



Try querying the Rtext of the pixmap gadget by using a built-in gadget method:

```
q var !!OptionExample.opt3.selection()
```

5.14 Toggle Gadgets

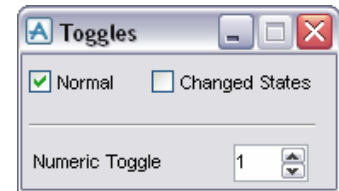
TOGGLE gadgets are used for independent on/off settings. This means they are used in situations where the user has two choices i.e. is it bold or not? Is it on or off? A toggle will return a **BOOLEAN** state (stored under its val member), but can also store different descriptions for selected and unselected.

NUMERIC INPUT gadgets can be used to allow users to enter real values within a range. The user can also use the supplied toggles to alter the value by the defined step.

Show the example by typing **show !!exampleToggles**. Click the toggles and observe the values on the command line. The method is given an argument of the gadget which called it. This means that the correct value is returned. Notice how a **LINE** gadget has been used to divide the form.

```
setup form !!exampleToggles
!this.formTitle = |Toggles|
path right
toggle .tog1 tagwid 5 |Normal| call |!this.state(!this.tog1)|
toggle .tog2 tagwid 10 |Changed States| call |!this.state(!this.tog2)| states |N| |Y|
line .line at xmin.tog1 ymax.tog1 horiz wid 21 hei 1
numeric .num |Numeric Toggle| at xmin.tog1 ymax.line range 0 10 step 1 NDP 0 wid 3
exit
```

```
define method .state(!gad is GADGET)
if (!gad.val) then
q var !gad.onvalue
else
q var !gad.offvalue
endif
endmethod
```



5.15 Radio Gadgets

A **RADIO GROUP** is used to give a user a single choice between a small fixed number of choices. Two objects can be used to define a radio group: **RGROUP** or **RTOGGLE**. An **RGROUP** element defines the entire object and the tags contained; an **RTOGGLE** is contained within a normal **FRAME** object. **RGROUP** objects can be displayed vertically or horizontally. **RTOGGLE** objects can be arranged within a **FRAME** as required and can be placed alongside other gadgets

An **RGROUP** object has been deprecated and maybe withdrawn in the future. Use an **RTOGGLE** in preference for new and upgrading code.

Show the example by typing **show !!exampleRGroups**. Notice how similar the gadgets appear and behave. Comment out the **CONSTRUCTOR** method to see that **RTOGGLE** elements can be given different callbacks, instead of the callback on the containing frame.

```
.
setup form !!exampleRGroups
!this.formTitle = |Radio Groups|
rgroup .vert |RGroup| FRAME vertical callback |!this.rg(!this.vert)|
add tag |Left| select |L|
add tag |Centre| select |C|
add tag |Right| select |R|
exit
frame .rtog |RToggle| at xmax + 1 y 0
path down
rToggle .left |Left| states || |L|
rToggle .cen |Centre| call |q var !this.cen.onvalue| at ymax - 0.1 states || |C|
rToggle .righ |Right| at ymax - 0.1 states || |R|
exit
exit
```

```
define method .exampleRGroups()
!this.rtog.callback = |!this.rt(!this.rtog)|
endmethod
```

```
define method .rg(!gad is GADGET)
q var !gad.selection()
endmethod
```

```
define method .rt(!gad is GADGET)
!rtog = !gad.rtoggle(!gad.val)
q var !rtog.onvalue
endmethod
```



5.16 User-defined form members

As a **FORM** is an **OBJECT**, it may be given **MEMBERS**. This is a useful way to store data, effectively creating global variables. The benefit of this method is that data is global without having large numbers of individual variables.

Form members replaces the previous method of **USERDATA** (PML 1 style data storage), and are given an object-type. These variables have the same lifetime as the form and are deleted when the form itself is unloaded.

Show the example by typing **show !!exampleFormMembers**. It shows how a form can be assigned an element on showing the form and how this element can be updated.). Every time the update button is pressed, the element is stored in the member **.storage**.

```
setup form !!exampleFormMembers resize
!this.formTitle = |Form Members|
!this.initcall = |!this.init()|
para .ceName wid 30 hei 1
list .attrib |Attributes:| at x 0 ymax anchor all wid 35 hei 20
button .update linklabel |Update| at xmax.ceName ymin.ceName wid 4
member .ceRef is DBREF
member .storage is ARRAY
exit

define method .exampleFormMembers()
!this.update.callback = |!this.init()|
!headings = |Attribute Value|
!this.attrib.setHeadings(!headings.split())
endmethod

define method .init()
!this.ceRef = !!CE
!this.ceName.val = !!CE.FLNN
!this.storage.append(!this.ceRef)
!this.fill()
endmethod

define method .fill()
!atts = !this.ceRef.attributes()
do !n index !atts
!dtext[1][!n] = !atts[!n].string()
!dtext[2][!n] = !this.ceRef.attribute(!atts[!n]).string()
enddo
!this.attrib.setColumns(!dtext)
endmethod
```

Investigate the information stored on the form members by typing:

```
q var !!exampleFormMembers.ceRef
q var !!exampleFormMembers.storage
```

5.17 Tooltips

Tooltips are small help boxes which pop up when you move the mouse over an active gadget. Tooltips are typically used to provide more information to the user (for example, on a pixmap button). An example of the syntax is as follows:

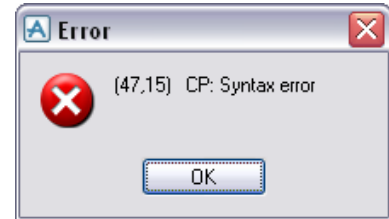
```
BUTTON .B1 |SAVE| PIXMAP TOOLTIP |Save Work|
```



 Refer to the specific gadgets in the PDMS Software Customisation Reference Manual

Exercise 5 - Working with a form

- 1
 - You have been provided with a form called !!ex5. To show the form, type **show !!ex5**
 - The form has not been maintained and there are a number of bugs in the form. At least 3 are preventing it from being shown and at least 2 are causing the wrong information to be displayed.
 - Debug the form so that it can be shown and displays the correct information. The fixes are listed in Appendix B



- 2
 - The form demonstrates poor gadget arrangement and impacts on the user experience. Work through the gadgets and improve their arrangement and sizes.
 - An example of the aligned form is shown in the below screenshot.

- 3
 - Write a method that will run when the form is shown. This method shall be used to fill default values into the input frame.
 - When the input values are set, the method should then run the methods that fill in the output frame.
- 4
 - Add a new button to the form that will fill the temperature conversion chart with Fahrenheit to Celsius values
 - The method has already been written. Consider which method needs to be called and what argument needs to be passed

No.	Celsius	=	Fahrenheit
1	0	=	32.00
2	25	=	77.00
3	50	=	122.00
4	75	=	167.00

- 5
- There is currently no method available to split the temperature string in lower frames.
 - Write a method to do the following:
 - Split the input string based on the delimiter
 - Read the individual temperatures and convert them (this will depend on °C or °F)
 - Compile the converted values as a space separated string
 - Write the compiled string back to the form and display how many temperatures
 - This method should be run when the form is shown or the button is pressed.

Temperture Split		Temperature Split Result	
Input	10° C/30° F/20° C/5° F	No. of Temp	4
Delimiter	/ Split temperature >>	Result	50° F -1° C 68° F -15° C



An example of the completed procedure can be found in Appendix B

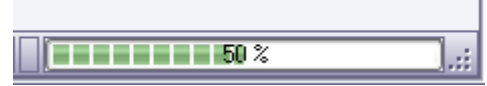
5.18 Progress bars and interrupting methods

The standard PDMS global object **!!FMSYS** provides two pieces of functionality which can be applied when designing a form:

- An progress bar (appearing at the bottom right of the main program)
- The ability to interrupt a method

The progress bar is activated by passing a real number to the **.setProgress()** method. . For example:

!!FMSYS.setProgress(50)



Passing it a zero argument will cause the progress bar to disappear

To identify a gadget which can be used to interrupt a method, it must be passed as an argument to the **.setInterrupt()**. The method can then check the **!!FMSYS** object to see if the gadget has been pressed through its **.interrupt()** method

Show the example by typing **show !!exampleProgressBar**. Press the start button and press it again to stop the method early

```

setup form !!exampleProgressBar dialog NoAlign
    button .b1 at xmin ymax call [|this.counter()] pixmap wid 100 hei 100
    numeric .num at xmax - size ymax + 0.1 range 0 10000 step 1 ndp 0 val 0 wid 4
exit

define method .exampleProgressBar()
    !this.b1.addPixmap (!!pml.getpathname('gobutton.png'))
endmethod

define method .counter()
    !!FMSYS.setInterrupt (!!exampleProgressBar.b1)
    !this.b1.addPixmap (!!pml.getPathName('stopbutton.png'))
    !this.b1.refresh()
    do !n from 1 to 10000
        !this.num.val = !n
        !this.num.refresh()
        !percent = !n / 100
        -- Check if the method has been interrupted. Break if it has
        if (!!FMSYS.interrupt()) then
            !!alert.message(!n & | loops were completed - | & !percent.string(|D1|) & |%)
            break
        endif
        !!FMSYS.setProgress(!percent)
    enddo
    !this.b1.addPixmap (!!pml.getPathName('gobutton.png'))
    !!FMSYS.setProgress(0)
endmethod
    
```



5.19 Open Callbacks

An **OPEN CALLBACK** is a way of providing change information about the gadget the user is interacting with. It means that for every interaction event, the callback will be called. Two pieces of information are passed during an open callback:

- the gadget being interacted
- a keyword (as a STRING).

These keywords indicate the event which caused the callback. Different keywords will be generated by different gadgets under different circumstances e.g. a multi-selection gadget may have a 'SELECT' 'UNSELECT', as well as the more standard 'START' 'STOP'

PDMS will recognise an open callback if a method only has one bracket e.g. **call !!this.opencall(|**

For an open callback to work, there must be an appropriate method defined. An open callback method must be able to accept two arguments (1) the gadget and (2) the keyword

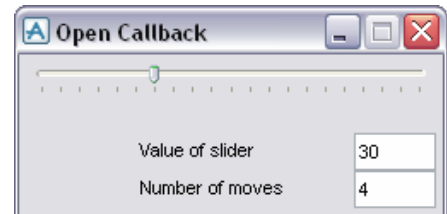
e.g. **define method .opencall(!a is GADGET, !b is STRING)**

As the method is called at every significant event, the open callback method should be written to recognise keywords. The example has been written around the slider gadget. Show the example by typing **show !!exampleOpenCallback**. Investigate the form by sliding the slider. Review the methods to identify how it works. Also, revisit the !!exampleOptions to review that opencallback.

```
setup form !!exampleOpenCallback
  !this.formTitle = |Open Callback|
  slider .slide anchor L+R horizontal range 0 100 step 5 val 50 width 30
  text .text |Value of slider| at xmax - size ymax + 1 anchor B+R width 5 is REAL
  text .moves |Number of moves| at xmax.text - size ymax anchor B+L width 5 is REAL
  member .store is REAL
exit
```

```
define method .exampleOpenCallback()
  !this.slide.callback = !!this.slideMove(|
  !this.moves.val = 0
  !this.store = 0
  !this.text.val = !this.slide.val
endmethod
```

```
define method .slideMove(!gad is GADGET, !val is STRING)
  !this.text.val = !gad.val
  !this.text.refresh()
  if !val eq |MOVE| then
    !this.store = !this.store + 1
  elseif !val eq |STOP| then
    !this.moves.val = !this.store
    !this.store = 0
  endif
endmethod
```



The **.refresh()** method has been applied to the **.text** gadget to ensure its value updates as the slider is moved. The number of moves is stored as a member of the form. This saves a new global variable from being defined and means the increasing value can be shared between the open callbacks.

i *An open callback can be given to any gadget that accepts a callback. Different gadgets may generate different event keywords*

5.20 Menus

Menu objects are used to provide options to a user and can be applied to a form as either a:

- Menu bar across the top of the form
- Popup menu on a gadget

In both cases, a menu object must be defined to represent the options part of the menu bar or popup menu.

5.20.1 Defining a menu object on a form

Within the form definition, the form method `.newMenu()` creates a named menu object. Once the object is defined, the `.add()` method on the menu object can be used to add named menu fields. A menu field can do one of three things:

- Execute a callback
- Display a form
- Display a sub-menu

You can also add a visual separator between fields.

```
!menu = !this.newmenu(|exampleMenu|, |MAIN|)
!menu1.add(|CALLBACK|, |Query|, |q atts|)
!menu1.add(|FORM|, |Progress Bar...|, |exampleProgressBar|)
!menu1.add(|SEPARATOR|)
!menu1.add(|MENU|, |Pull-right1|, |Pull1|)
```

This creates a menu object called `exampleMenu` with 3 fields (`Query`, `Hello...` and `Pull-right1`) and a separator between the last two fields

- The `Query` field when picked will execute the **CALLBACK** command 'q att' (prints the CE attributes to the command window)
- The `Hello...` field when picked will load and display the **FORM** from the previous section `!!exampleProgressBar`. By convention, the text on a menu field leading to a form ends with three dots, which you must include with the text displayed for the field.
- The **SEPARATOR**, usually a line, will appear after the previous field.
- The `Pull-right1 MENU` field when picked will display the sub-menu `!this.Pull1` to its right. A menu field leading to a sub-menu ends with a `>` symbol (this is added automatically)

5.20.2 Defining a bar menu on a form

Forms may have a bar menu gadget which appears as a row of options across the top of the form. A bar menu is defined within form definition and specifies the options the user has to choose from. There are three types that can be added:

- Choose – displays a user-defined menu object (reference by its name)
- Window – displays a list of the open forms
- Help – displays the standard help options

After the bar command, use the bar object method `.add()` to add extra options to the bar menu. As there can only be one bar menu, `!this.bar` refers to the bar menu. For example:

```
bar
!this.bar.add(|Choose|, |exampleMenu|)
!this.bar.add(|Window|, |Window|)
!this.bar.add(|Help|, |Help|)
```

 *Bar menus can only be added to forms which are not dockable*

5.20.3 Defining a popup menu on a form

Pop-up Menu is a MENU object that are displayed from a gadget by right-clicking on it. This is a useful method of providing the user with relevant functionality relating to the associated gadget.

```
!this.exampleList.setPopup(!this.exampleMenu)
```

The MENU object is applied to a gadget through the .setPopup() method (available on a number of gadgets). The MENU object can be applied at any time, so popup menus can change to ensure the content is always relevant.

5.20.4 Adding menus objects to a form

Show the example by typing show !!exampleMenus.

```
setup form !!exampleMenus
!this.formTitle = |Menus|
!this.initcall = |!this.init()|
bar
!this.bar.add(|Choose|, |exampleBarMenu|)
!this.bar.add(|Window|, |WINDOW|)
!this.bar.add(|Help|, |HELP|)

list .list |List of EQUIs| callback |!this.pickList()| wid 20 hei 12

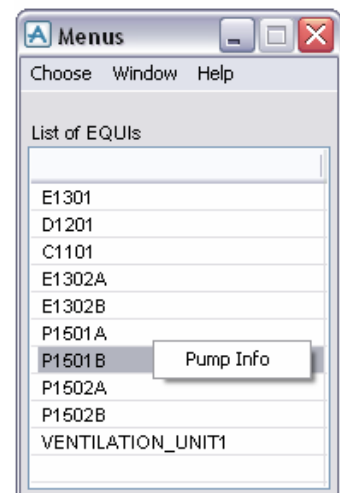
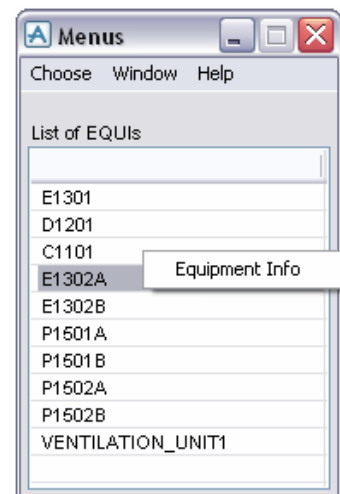
!menu = !this.newMenu(|exampleBarMenu|)
!menu.add(|CALLBACK|, |Query|, |q atts|)
!menu.add(|FORM|, |Progress Bar...|, |exampleProgressBar|)
!menu.add(|SEPARATOR|)
!menu.add(|MENU|, |Pull-right1|, |Pull1|)

!menu = !this.newMenu(|exampleEquiMenu|)
!menu.add(|CALLBACK|, |Equipment Info|, |$p EQUI command|)

!menu = !this.newMenu(|examplePumpMenu|)
!menu.add(|CALLBACK|, |Pump Info|, |$p PIPE command|)
exit

define method .init()
-- Collect the equipment from the Stabilizer Plant
var !equipment coll all EQUI for /EQUIP
var !equipmentNames eval FLNN for all from !equipment
!this.list.dtext = !equipmentNames
!this.list.rtext = !equipment
endmethod

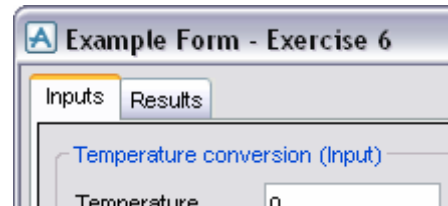
define method .pickList()
-- Set the popup menu based on the the first letter of name
if !this.list.selection(|DTEXT|).substring(1,1).eq(|P|) then
!this.list.setPopup(!this.examplePumpMenu)
else
!this.list.setPopup(!this.exampleEquiMenu)
endif
endmethod
```



 Refer to the *PDMS Customisation Reference Manual* for the details of which gadgets permit a popup menu.

Exercise 6 – Extending the form

- 1
 - Save the form from the previous exercise as ex6.pmlfrm and update the name of the form and name of the constructor method. Type **PML REHASH ALL** so PDMS finds the new file
 - Upgrade the frames from exercise 5 to a tabset
 - Remove the callbacks from the existing gadgets and any buttons from the form. The methods should now be called through an **OPEN CALLBACK** on the Results tab. When the tab is shown, the methods run.



- 2
 - Add an **ARRAY** member to the form that will store the values of the input gadgets. This will provide the ability to reset the values if they are changed.
 - Write a method that will allow data to be applied to and taken from this member.
 - Add two buttons to the base of the form. One to update the stored values (Accept changes) and the other with apply the stored values to the gadgets (Discard changes)



- The buttons will be **linklabels** and should call the method to store/reset data. To the left of the buttons there should be **16x16 pixmap paragraph**. Using the **.addPixmap()** method, apply standard PDMS images to them.

```
.addPixmap(!pml.getPathName('accept.png'))
.addPixmap(!pml.getPathName('discard.png'))
```

- 3
 - As the buttons have been removed from the form, only one type of conversion can be applied to the List gadget (depending on which method the open callback calls)
 - Add a **POPUP MENU** to the list gadget which will allow the other type of conversion to be done



- There should be two new menu objects created on the form; one will run the method as Celsius and the other as Fahrenheit. After a choice has been made, the popup menu needs to be swapped to the other one. Consider what methods will need to be called and how the popup menus can be managed.



An example of the completed procedure can be found in Appendix B

6 PML Objects

6.1 Built in PML object types

There are a large number of standard objects which are supplied with PDMS. These include All Variable types, BORE, DIRECTION, DBREF, FORMAT, MDB, ORIENTATION, POSITION, FILE, PROJECT, SESSION, TEAM, USER, ALERT, FORM, all form Gadgets and various graphical aid objects.

Each object has a set of built in methods for setting or formatting the object contents. The following are the objects covered by the Software Customisation Reference Manual:

ARRAY	DATEFORMAT	FORMAT	REAL
BANNER	DATETIME	LOCATION	REPORT
BLOCK	DB	MACRO	SESSION
BOOLEAN	DBREF	MDB	STRING
BORE	DBSESS	OBJECT	TABLE
COLLECTION	DIRECTION	ORIENTATION	TEAM
COLUMN	EXPRESSION	POSITION	UNDOABLE
COLuMNFORMAT	FILE	PROJECT	USER

The following objects from the reference manual cover forms and menus:

ALERT	FORM	OPTION	TEXTPane
BAR	FRAME	PARAGRAPH	TOGGLE
BUTTON	LINE	RTOGGLE	VIEW ALPHA
COMBOBOX	LIST	SELECTOR	VIEW AREA
CONTAINER	MENU	SLIDER	VIEW PLOT
FMSYS	NUMERIC	TEXT	VIEW VOLUME

The following objects from the reference manual cover 3D geometry:

ARC	LOCATION	POINTVECTOR	PROFILE
LINE	PLANE	POSTEVENTS	RADIAL GRID
LINEARGRID	PLANTGRID	POSTUNDO	XYPOSITION

6.2 Methods available to all PML objects

In addition to the object specific methods, there are methods that are common to all objects. The following table lists the methods available to all objects:

Name	Result	Purpose
Attribute('Name')	ANY	To set or get a member of an object, providing the member name as a STRING.
Attributes()	ARRAY OF STRINGS	To get a list of the names of the members of an object as an array of STRING.
Delete()	NO RESULT	Destroy the object - make it undefined
EQ(any)	BOOLEAN	Type-dependent comparison
LT(any)	BOOLEAN	Type-dependent comparison (converting first to STRING if all else fails)
Max(any)	ANY	Return maximum of object and second object

Min(any)	ANY	Return minimum of object and second object
NEQ(any)	BOOLEAN	TRUE if objects do not have the same value(s)
ObjectType()	STRING	Return the type of the object as a string
Set()	BOOLEAN	TRUE if the object has been given a value(s)
String()	STRING	Convert the object to a STRING
Unset()	BOOLEAN	TRUE if the object does not have a value

 Refer to the *Software Customisation Reference Manual* for further details.

6.3 The FILE Object

The **FILE** object is an example of how older functionality has been replaced with a PML 2 style object. This object replaces the 'openfile' 'readfile' 'writefile' 'closefile' syntax and provides increased functionality (file path, if the file is open etc). It is now possible to read or write to a file in a single operation.

To create a file object:

```
!input = object file('C:\FileName')  
!output = object file('C:\FileName.out')
```

To Open a file:

```
!output.open('WRITE')                      Available options = READ, WRITE, OVERWRITE, APPEND
```

6.3.1 Using FILE objects

To read a line from file:

```
!line = !input.readRecord()                      File must be open
```

To write a line to file:

```
!output.writeRecord(!line)                      File must be open
```

To read all the input file:

```
!fileArray = !input.readFile()                      Files are opened and Closed automatically
```

To write all of the data to file

```
!output.writeFile('WRITE', !fileArray)                      Files are opened and Closed automatically
```

 The `.ReadFile()` method has a default maximum file size which it can read. This can be increased by passing the method a `REAL` argument (representing the number of lines in the file)

6.3.2 Opening a FILE object in Notepad

You could use the **SYSCOM** command to open an external program like Notepad, by adding an & to the end of the line will opens a new process. If you do not use then & PDMS will be suspended until the program is closed.

```
!file = object file(|C:\temp\pml\lib\forms\ex5.pmlfrm|)  
syscom |c:/WINDOWS/notepad.exe $!file&|
```

6.3.3 Using the standard file browser

PDMS is supplied with a standard file browser that can be used to load forms. The best way to load the form is to use the `!!fileBrowser` function as the function arguments will initialise the browser. To show the browser form, type the following:

```
!!fileBrowser('c:\', '*.pmlfrm', 'Example', false, 'q var !!fileBrowser.file')
```

Where the first argument is the initial file path; the second is the file type filter; the third is the title for the browser form; the fourth is if the file needs to exist (true or false – used to save files); the fifth argument is the callback on the 'Open' button.

Navigate to a file and press open, a file object should be printed to the command window:

```
<FILE> C:\temp\pmllib\forms\ex5.pmlfrm
```

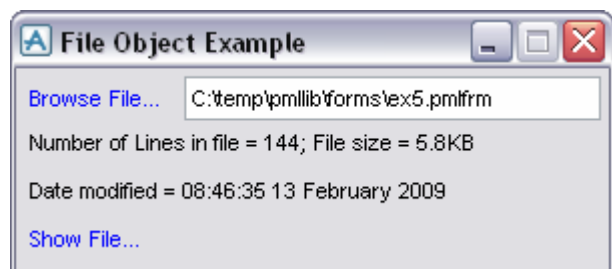
This shows that once a file has been chosen, it is held as the `.file` member of the `fileBrowser` form and is available for use. Show the example by typing `show !!exampleFile`. The example demonstrates how the object can be used to read a file, gain information about it and even show it.

```
setup form !!exampleFile
!this.formTitle = |File Object Example|
button .browse linklabel |Browse File...| wid 8
text .fileName || call !!this.read(object file(!this.txt1.val), 1)| width 25 is string
para .par1 at x 0 ymax text || width 35
para .par2 at x 0 ymax text || width 35
button .load linklabel |Show File...| at x 0 ymax call !!this.load()| wid 8
member .file is FILE
exit

define method .exampleFile()
!this.browse.callback = !!fileBrowser('C:\temp\pmllib\forms', '*.pmlfrm', $
'Load File', TRUE, '!!exampleFile.read(!!fileBrowser.file, 2)')|
!this.load.visible = FALSE
endmethod

define method .read(!file is file, !flag is REAL)
!this.file = !file
if !flag.eq(2) then
!this.fileName.val = !file.string()
endif
!file.open('READ')
!n = 0
do
!line = !file.ReadRecord()
if !line.unset() then
BREAK
else
!n = !n + 1
endif
enddo
!file.close()
!this.load.visible = TRUE
!date = !file.DTM()
!size = (!file.size() / 1000)
!this.par1.val = |Number of Lines in file = | & !n & |; File size = | $
& !size.string(|d1|) & |KB|
!this.par2.val = |Date modified = | & !date
endmethod

define method .load()
!file = !this.file
syscom |c:/WINDOWS/notepad.exe $!file&|
endmethod
```



6.4 User-Defined Objects

User Defined Objects can contain any variable or object type as a member. Information is grouped within an object so it can be easily reference and used.

User-defined objects can also contain user-defined methods that will typically use the information stored within the object. These methods may represent a calculation, or information retrieval etc.

A PML object is defined within a .pmlobj file and starts with the definition of the object and its members. If required, the file then lists the objects members (typically starting with the CONSTRUCTOR method). After an object has been declared as a variable, the methods can be applied to the variable. For example, an **ELEMENT** object may be defined:

```
define object ELEMENT
  member .type is STRING
  member .material is STRING
endobject

define method .fullDescription() is STRING
  return |Type = | & !this.type & |; Material = | & !this.material
endmethod
```

Once the object is assigned to a variable, it becomes an **ELEMENT**. Type out the following:

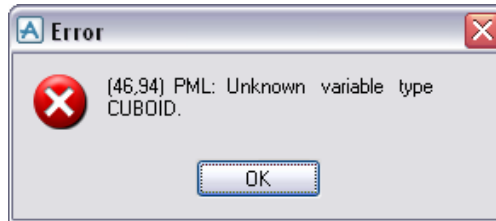
```
!item = object ELEMENT()
!Item.type = |Flange|
!Item.material = |Carbon Steel|
!name = !item.fullDescription()
q var !name
```

It is possible to define more than one constructor method. They will have the same name, but different arguments. This allows more than one constructor method to be defined, providing they have different arguments

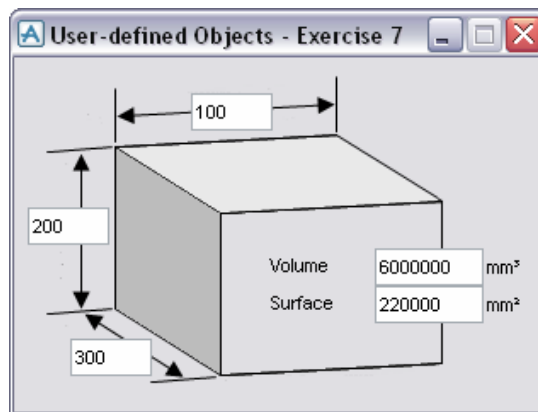
The advantage of defining an object is that its definition can be applied across other objects, forms and functions. This saves the repetition of common functionality.

Exercise 7 – User-defined objects

- 1
 - You have been provided with a form called !!ex7. Show the form by typing **show !!ex7**
 - On trying to show the form, you will receive the following error:



- The form is trying to use a **CUBOID** object and its members and methods. Review the form definition to identify how it is trying to be used.
- Without changing the form definition, create a **CUBOID** object with the correct members and methods so that the form works correctly



- 2
 - Revisit the form from exercise 6. Consider how a user-defined object could have been used to store the information in the gadgets. How could an object collect and store the values on the form? The benefit of developing an object to do this means it could be applied to other similar forms.
 - Create a new object called **DATASTORE**. This object should have two members; one to store the gadgets objects as an **ARRAY** and the other to store the gadget values as an **ARRAY**.
 - The constructor method of the object should be able to accept a **FORM** object and it should collect all the **TEXT** gadgets from that form and store the information in its members
 - A method will be needed to update the stored information and another will be needed to transfer the information back to the gadgets
 - Once completed, this object should be robust enough so it could be applied to any **FORM** object to store the values of **TEXT** gadgets

7 Collections

A very powerful feature of the PDMS database is the ability to collect and evaluate data according to rules. There are two available methods for collection that are valid in PML. The first is a command syntax PML 1 style and the other is with a PML **COLLECTION** object. Both are still valid, but when developing new PML, a **COLLECTION** object should be used in preference,

7.1 COLLECT command syntax (PML 1 style)

The **COLLECT** syntax is based around three specific pieces of information:

- What element type is required?
- If specific elements are required, what is different about them?
- Which part of the hierarchy to look it?

If you wish to collect all the EQUI elements for the current ZONE, type the following:

```
var !equipment collect all EQUI for ZONE
q var !equipment
```

If you wish to collect all the piping components owned by a specific BRAN, type the following:

```
var !pipeComponents collect ALL with owner eq /200-B-4-B1 for SITE
q var !pipeComponents
```

if you wish to collect all the BOX primitives below the current element, type the following:

```
var !boxes collect all BOX for ce
q var !boxes
```

 You do not need to specify level of the hierarchy to search within.

 For more examples, refer to Chapter 11 of the Database Management Reference Manual and 2.3.10 Design Reference manual general commands

7.2 EVALUATE command syntax (PML 1 style)

After elements have been collected through the **COLLECT** syntax, they are stored as an ARRAY. This array (like any array) can be processed using the **EVALUATE** syntax to provide further information

To get the names of all the elements held in **!equipment**, type the following:

```
var !equipmentNames evaluate NAME for ALL from !equipment
q var !equipmentNames
```

To get the fullnames of the elbows held within **!pipeComponents**, type the following:

```
var !elbows evaluate FLNN for all ELBO from !pipeComponents
q var !elbows
```

To get the names (without the leading slash) of the pumps in **!equipment**, type the following:

```
var !pumps evaluate NAMN for ALL with MATCHWILD(NAMN, |P*|) from !equipment
q var !pumps
```

To get the volume of all the boxes in **!boxes**, type the following:

```
var !volume eval (xlen * ylen * zlen) for ALL from !box
q var !volume
```

 Notice how the expressions are command syntax and must return a BOOLEAN. Refer to the Software Customisation Reference Manual for more examples. COLLECTION object (PML 2 style)

7.3 COLLECTION object (PML 2 style)

A **COLLECTION** object is another example of a PML object that has directly replaced some command syntax. It is recommended that all new PML uses **COLLECTION** objects as required. One advantage of using a **COLLECTION** object is that an **ARRAY OF DBREF** objects is returned.

A **COLLECTION** object is assigned to a variable in the same way as other objects:

```
!collection = object COLLECTION()  
q var !collection  
q var !collection.methods()
```

Once assigned, the methods on the object are used to set up the parameters of the collection. To set the element type for the collection to only **EQUI** elements, type the following:

```
!collection.type(|EQUI|)
```

If more than one element type is required, the following could be typed:

```
!elementTypes = |EQUI BRAN SCTN|  
!collection.type(!elementTypes.split())
```

The scope of the collection must be a **DBREF** object and should be passed as an argument to the **.scope()** method. For example, type the following:

```
!collection.scope(!!ce)
```

To filter the results, the **.filter()** method must be passed an **EXPRESSION** object. This means that the process is in two steps:

```
!expression = object EXPRESSION(|name of owner eq '/200-B-4-B1'|)  
!collection.filter(!expression)
```

Once the collection object has been set up, the **.results()** is used to return the collected elements as an **ARRAY of DBREF** objects.

```
!results = !collection.results()  
q var !results
```

7.4 Evaluating the results from a COLLECTION object

After an **ARRAY of DBREF** objects has been generated from a **COLLECTION** object, the entire array can be evaluated using the **.evaluate()** method on an array object. The argument for the **.evaluate()** method must be a **BLOCK** object defined with the expression that needs evaluating.

To get an **ARRAY of STRINGS** holding the full names of the collected elements, type the following:

```
!block = object BLOCK(|!results[!evalIndex].flnn|)  
!resultNames = !results.evaluate(!block)  
q var !resultNames
```

Notice how the **BLOCK** object uses the local variable **!evalIndex**. This variable effectively allows the **.evaluate()** method to loop through the ARRAY. To get the positions of the collected elements, type the following:

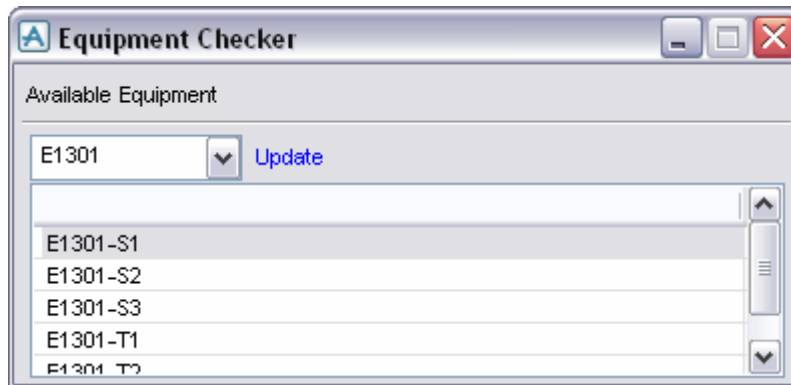
```
!resultPos = !results.evaluate(object BLOCK(|!results[!evalIndex].pos|))  
q var !resultPos
```

Instead of defining the block as a separate variable, this second example shows that the object can be defined within the argument to another object.

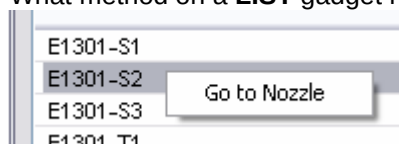
 Notice how the evaluated ARRAY contains the correct object types generated from the evaluation

Exercise 8 – Equipment Collections

- 1
 - Create a new form that will act as an **Equipment Checker**. The purpose of the form is to provide users information about **EQUI** elements from the hierarchy
 - The form shall use an **OPTION** gadget to display to the user the available EQUI elements and a **LIST** gadget to display the **NOZZ** elements owned by the chosen EQUI. Layout the form as the below screenshot.

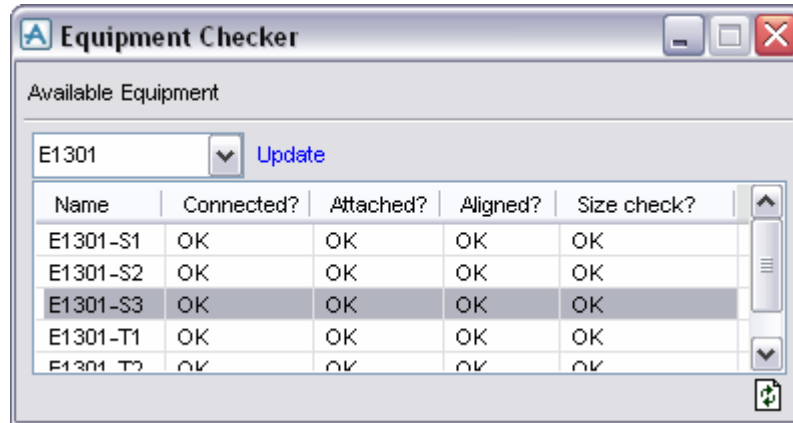


- 2
 - Write a method that collects all the **EQUI** elements for the **ZONE** of the **CE**. Try and write the collection part of the method in a PML 1 style. What happens if no pieces of equipment are found in the **ZONE**? Call this method when the form is shown.
 - Add an '**Update**' button that will re-run this method if a different **ZONE** is chosen
 - Write a method that runs after the piece of equipment has been chosen. This method should collect all the **NOZZ** elements for that **EQUI** element. Try writing the collection part of the method in a PML 2 style. After the method has run, set the full names of the collected **NOZZ** elements to the **LIST**. Make use of the **RTEXT** and **DTEXT** members of the gadgets (to store names and **DBREFs**)
- 3
 - Add a popup menu to the **LIST** gadget to allow the users to navigate to the chosen element (set the current element). What method on a **LIST** gadget retrieves the chosen **RTEXT**?

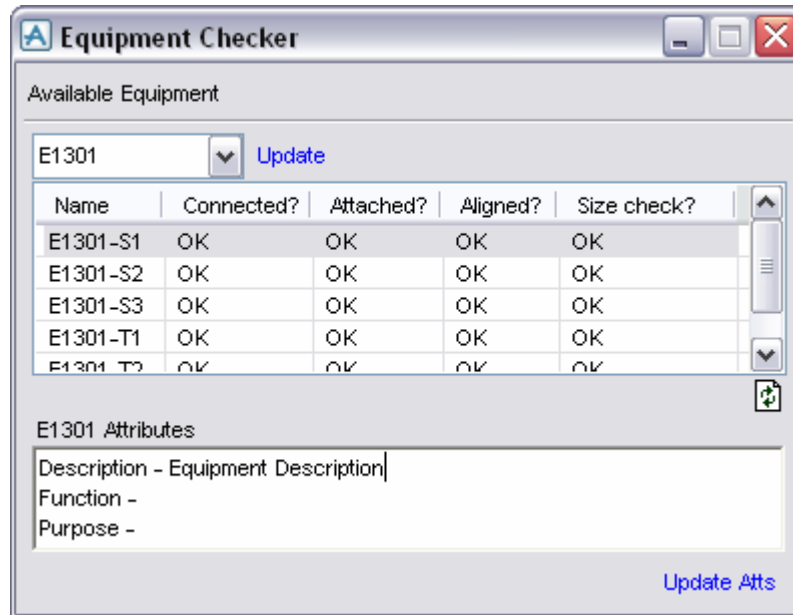


- 4
 - Write a method that looks at each of the collected **NOZZ** element and checks the following:

Is it connected?	This can be decided by checking the NOZZ element's CREF attribute. If the attribute is unset or the reference is invalid, then the user will have to check that nozzle.
Is it attached?	Check the positions of the NOZZ and PIPE elements and if they are different, then the NOZZ should be checked
Is it aligned?	Check the directions of the NOZZ and PIPE elements. If they are different, then they should be checked
Is it sized correctly?	Compare the diameter of the PIPE to the size of NOZZ. If they are different, then they should be checked.
 - Display the results of the check in the LIST gadget using columns.
 - Provide an Refresh button to recheck the nozzles incase the users changes anything in response to the check. Why not use the refresh graphic from standard PDMS?



- 5
 - Add a **TEXT**PANE to the form to allow users to enter attribute values for the chosen piece of equipment. The **TEXT**PANE should initially be filled with three attributes (**Description**, **Function** and **Purpose**) and the values for the chosen equipment. As the user chooses other pieces of equipment, the displayed attribute values should update
 - Write a method to read the attributes and values from the **TEXT**PANE. This will involve splitting the information entered by the user. If it is a valid value and attribute, the attribute should be set.
 - This method should cope with new attributes being entered by the user. Consider what should happen if an invalid attribute or value is entered.



- As the **TEXT**PANE has no callback, provide a button to call this method.

8 View Gadgets

VIEW gadgets are named gadgets which are used to display the information from available databases. The way the information is displayed depends on the VIEW gadget type.

General View Types:

- ALPHA** views for displaying text output and / or allowing command input.
- PLOT** views for displaying non-interactive 2D plotfiles.

Application-specific View Types

- AREA** views for displaying interactive 2D graphical views.
- VOLUME** views for displaying interactive 3D graphical views.

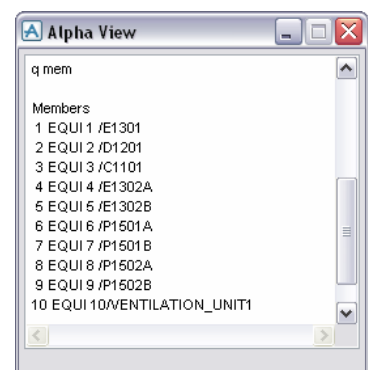


Refer to the correct VIEW element type in the PDMS Customisation Reference Manual for more information

8.1 Alpha Views

An **ALPHA** view gadget is the same that is used for the standard command window. To show the Alpha View example, type **show !!exampleAlphaView:**

```
setup form !!exampleAlphaView
!this.formTitle = |Alpha View|
view .input AT X 0 Y 0 ALPHA
height 15 width 30
channel REQUESTS
channel COMMANDS
exit
exit
```

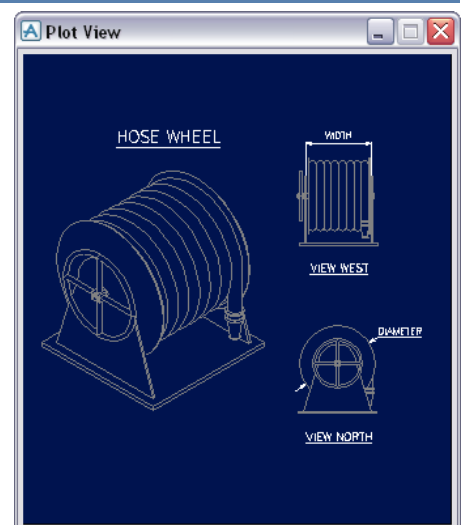


8.2 Plot View Example

A **PLOT** view can be used to provide the user with extra information. The extra information would be contained in a **.plt** file and is applied to the view during the constructor method. To show the Plot View example, type **show !!examplePlotView:**

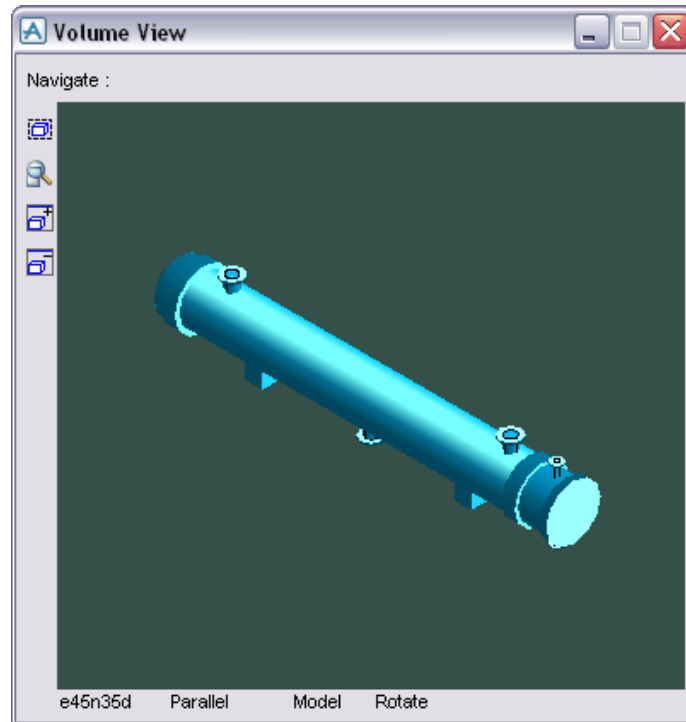
```
setup form !!examplePlotView
!this.formTitle = |Plot View|
view .plot plot width 41 hei 15
CURS NOCURSOR
exit
exit

define method .examplePlotView()
!this.plot.borders = FALSE
!this.plot.add(|/%PMLLIB%\icon\hosewheel.plt|)
endmethod
```



8.3 Volume View example

A **VOLUME** view can be used to see the 3D data held within the database. The main PDMS window is an example of a volume view. To show the Volume View example, type **show !!exampleVolumeView**:



 *The code that supports this example is in Appendix B*

It is now possible in PDMS to assign different drawlists to different views. This has been demonstrated by giving the volume view in this example a different drawlist to the main view. This is possible through the use of two standard PDMS objects: **!!gphviews** and **!!gphdrawlists**

To investigate the **!!gphdrawlists**, type the following into the command window:

```
q var !!gphDrawlists
<GPHDRAWLISTS> GPHDRAWLISTS
  DRAWLISTS <ARRAY> 1 Elements
q var !!gphDrawlists.drawlists
<ARRAY>
  [1] <DRAWLIST> DRAWLIST
q var !!gphDrawlists.methods()
```

The following are some examples of the methods which are available on **!!gphDrawlists**. To find out information about all the drawlists in the global object, use:

```
!!gphDrawlists.listall()
```

To create a drawlist object in the global object, use:

```
!!gphDrawlists.createDrawlist()
```

To associate a drawlist from the global object with a view gadget, use:

```
!!gphDrawlists.attachView(DRAWLIST, GADGET)
```

Drawlists can only be attached to a view that has been registered with the **!!gphViews** object. To investigate the **!!gphViews**, type the following into the command window:

```
q var !!gphViews
```

```
<GPHVIEWS> GPHVIEWS
  ACTIVEVIEW <GADGET> Unset
  SELECTEDVIEWS <ARRAY> 0 Elements
  VIEWS <ARRAY> 1 Elements
q var !!gphViews.methods()
```

To add a view to the global object, use:

```
!!gphViews.add(GADGET)
```

To set the limits of a view based on a DBREF object, use:

```
!!gphViews.limits(GADGET, DBREF)
```

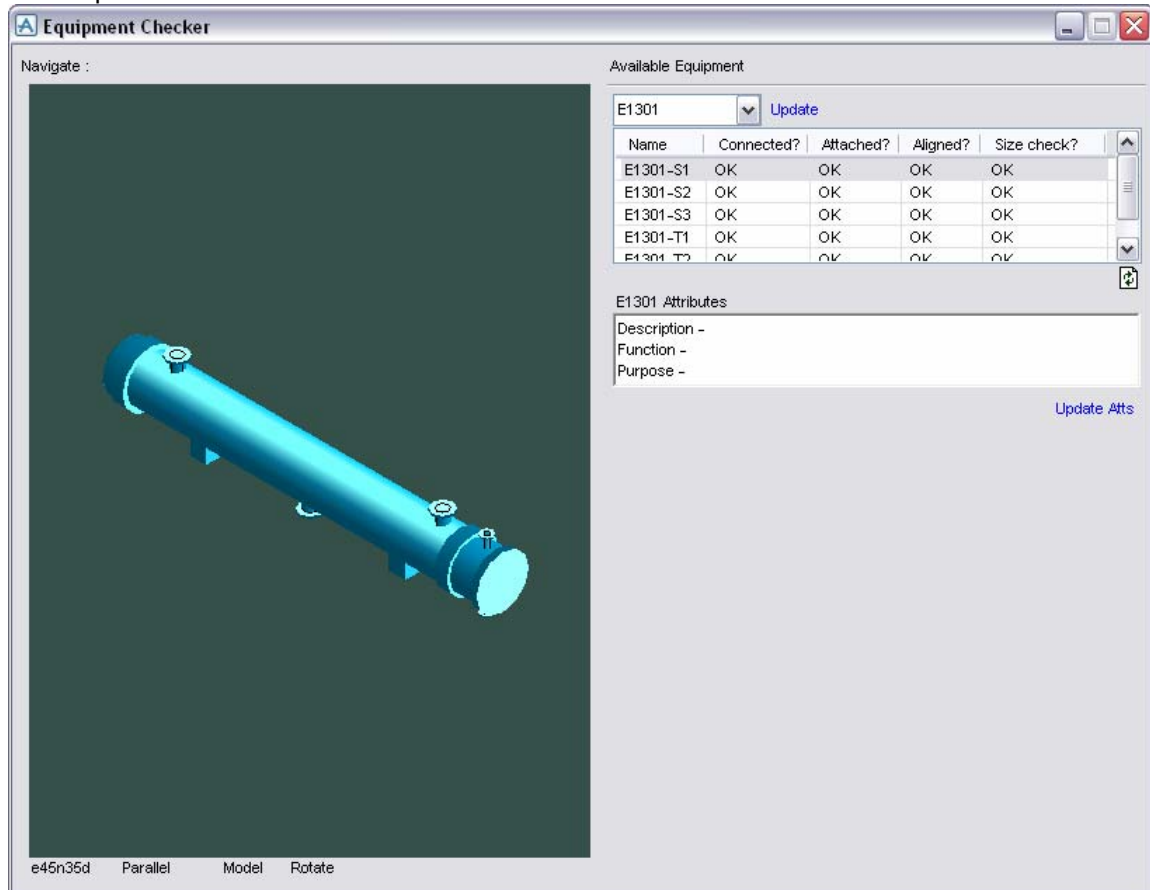
The example volume view form mimics the functionality of the main 3D window by providing the following four buttons:

-  set view limits to CE
-  Walk to Drawlist (set limits of the view based on drawlist)
-  Add the current element to the drawlist
-  Remove the current element from the drawlist

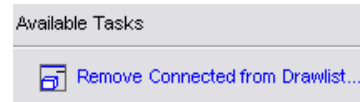
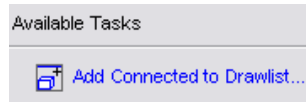
These buttons are not always necessary when defining a volume view element, but have been included in the example to demonstrate some further methods on the object. Review form definition and its method to understand how the process works.

Exercise 9 – Adding a Volume View

- 1
 - Add a **VOLUME VIEW** element to the form created in Exercise 8. The Volume View should have its own drawlist and should correctly display the chosen piece of equipment only
 - Look at **!!exampleVolumeView** and see which parts of the PML can be re-used. How will the method know which element has been chosen and how will the drawlist/limits be updated?



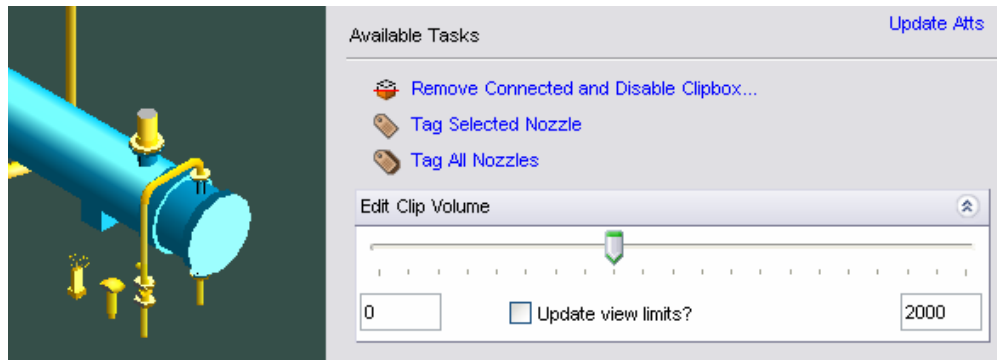
- 2
 - Add an '**Available Tasks**' section to the form. As functionality is added to the form, it shall be added here.
 - Write a method that adds any connected elements to the drawlist (a **NOZZ** is connected through its **CREF** attribute). Elements should be added and removed through a toggle style linklabel button (with a pixmap – see screenshots)



- 3
- Add a method to the form to tag **NOZZ** elements on the chosen piece of equipment. This will allow to the user to identify which nozzle is which.
 - Provide two extra tasks: **Tag Selected Nozzle** (chosen in the list) and **Tag All Nozzles**

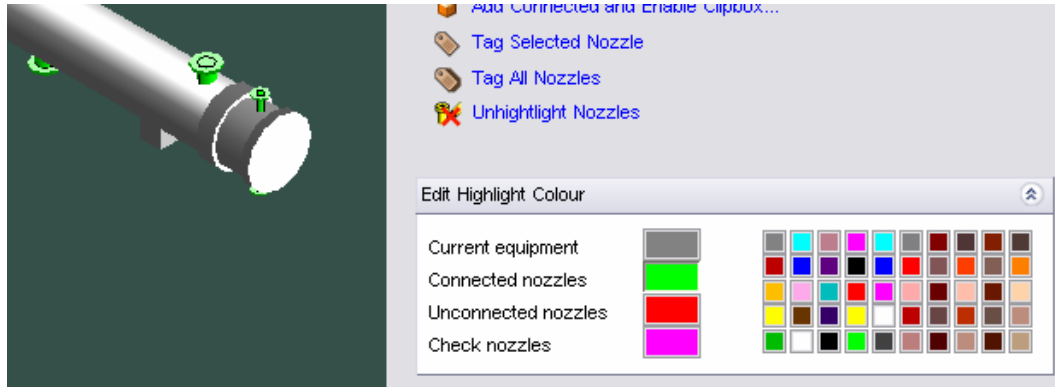


- When the user tags a nozzle, use the **MARK** or **AID TEXT** syntax to put the name of the nozzle into the volume view. The method should keep track of which nozzles are tagged so that they are only tagged once (use a form member?). This will also manage the state of the toggle so that it is always correct.
- 4
- Update the '**Add Connected**' method to include a clipbox. This will limit how much the user sees. A clipbox is added to a Volume View by using a **GPHCLIPBOX** object. The **VIEW** member of a **GPHCLIPBOX** object holds the name of the Volume View you wish to clip.
 - The **GPHCLIPBOX** object should be stored as a member of the form. This is so it can be resized as different elements are chosen. To find out more about the **GPHCLIPBOX** object, open **gphclipbox.pmlobj**
 - When the user turns the clipbox on, they should then see a hidden fold-up panel which holds a slider. This slider will allow the user to dynamically change the size of the clipbox. Make use of the **.refresh()** method to update the volume view during the slide



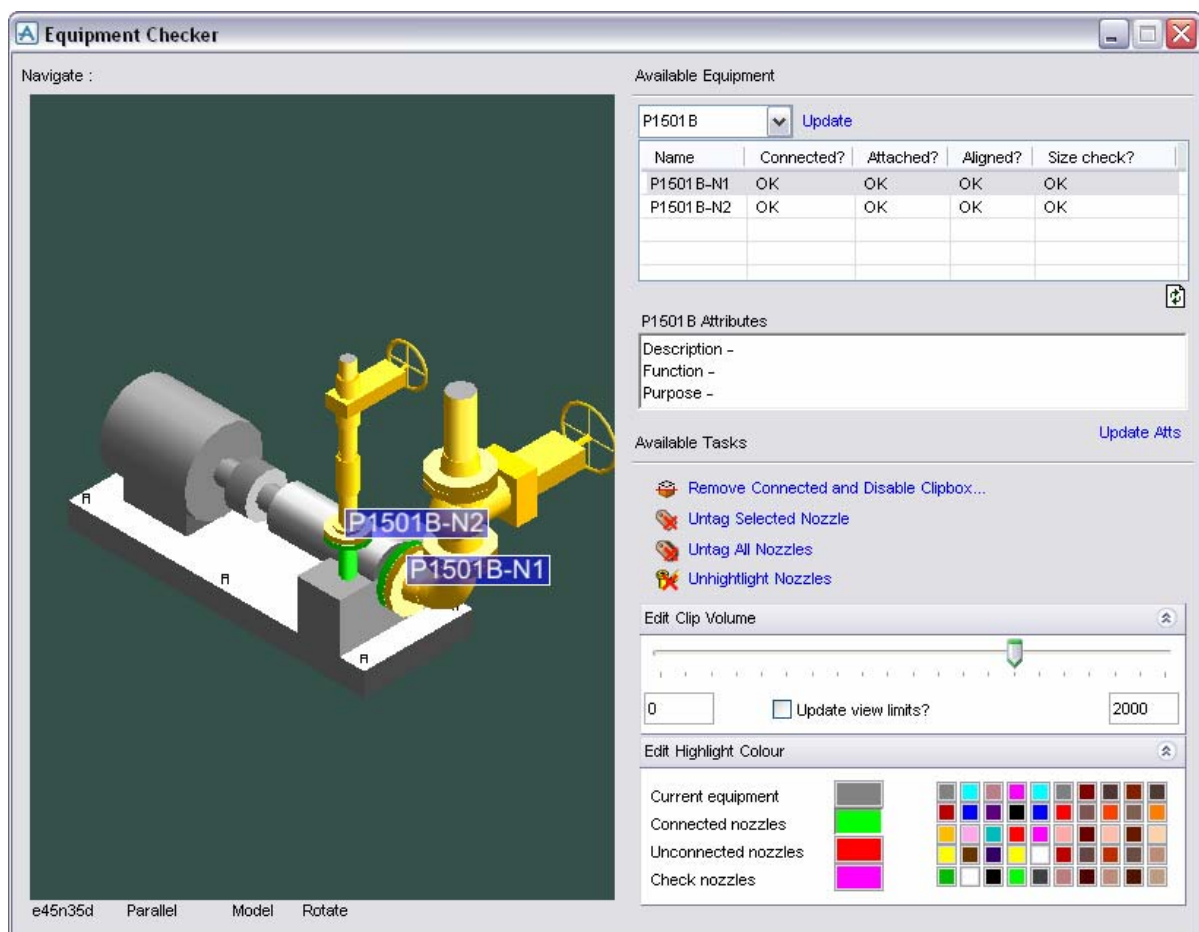
- Add a method to update the range of the slider based on the input into two text boxes. This will be in case the equipment is too large/small for the default value.
 - Add a method that will update the limits of the view when the clipbox is resized. This is to stop the clipbox becoming larger than the limits of the view.
- 5
- Add a method to the form that will add colour to the elements of the drawlist to indicate the results of the nozzle check. Colour can be added through the **.highlight()** method on a **DRAWLIST** object.
 - Three different colours should be added. A colour for unconnected nozzles, a colour for nozzles that need checking and a colour for connected nozzles which pass the checks. You may also wish to highlight the chosen piece of equipment.

- When the user turns on the highlighting, show a hidden frame which allows users to change the colours. Four buttons shall display the current colours. On pressing on of those buttons, a hidden panel frame shall be shown which will allow the users to update the colour. Think about how a **DO** loop could be used to create this grid of buttons.



- The methods will need to manage which elements are coloured, which colours should be used and the visibility of the gadgets on the form

- 6
- The form is now complete. Test it as a complete form and solve any errors that occur during this process.



9 Event Driven Graphics (EDG)

The EDG has been developed to allow a common interface for appware developers to use when setting up the graphic canvas for graphical selection. This method is relatively simple and easily extendible. This will allow the developer to concentrate on the development of their own application without the need to know the underlying mechanism (core implementation) of the EDG interaction handlers and system.

The system handles all the underlying maintenance of the current events stacked e.g. associated forms, initialisation sequences, close sequences, etc.

The current implementation of the system has mainly been developed for interaction with the 3D graphic views in the Design module. However, the interface can be used with any of the modules that use the same executable and the standard 3D graphic views. It is intended that EDG will supersede the old ID@ syntax. When setting up an EDG event, the following should be considered:

- What items will the user be picking?
- How many picks are required and in what order?
- What happens when the user makes a pick?
- What happens when the user has finished picking?

These aspects are controlled by methods on the objects associated with EDG. The main object is an **EDGPACKET** object. Once setup, this object is added to the **!!EDGCtrl** object that will activate the event.

For more information about EDG, refer to the EDG interface manual

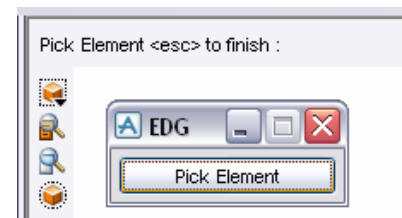
9.1 A simple EDG event

To demonstrate a simple EDG setup, show the example by typing **show !!exampleSimpleEDG:**

```
setup form !!exampleSimpleEDG
  !this.formTitle = |EDG|
  button .but1 | Pick Element | call |!this.pick()|
exit

define method .pick()
  -- Define event packet object
  !packet = object EDGPACKET()
  -- Define object as a predefined standard element pick
  !packet.elementPick(|Pick Element <esc> to finish|)
  -- Query the item member of the returned information from the pick
  !packet.action = |!!exampleSimpleEDG.process(!this.return[1])|
  -- Set what happens when the user presses esc
  !packet.close = |$p Finished|
  -- Add the event packet to the global EDG control object
  !!EDGCtrl.add(!packet)
endmethod

define method .process(!item is ANY)
  q var !item
endmethod
```



The example sets up a predefined element pick event by running the **.elementPick()** method on a **EDGPACKET** object. The subsequent methods specify what happens when a pick is made and what happens when the event is finished.

Notice how the action method of the EDGPACKET object references the form method explicitly. It displays the returned information (described in EDG interface manual)

9.2 Using EDG

The following example shows how EDG can be applied to a form to provide picking functionality for users. The show the example, type **show !!exampleEDG**:

```

setup form !!exampleEDG
  !this.formTitle = |Pick Equipment|
  !this.initcall = |!this.init()|
  !this.quitcall = |!this.clear()|
  button .pick |Start Pick| call |!this.init()|
  textpane .txt1 |Picked Equipments| at x 0 ymax width 25 height 10
  member .storage is array
exit

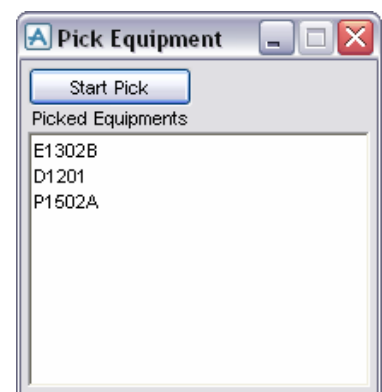
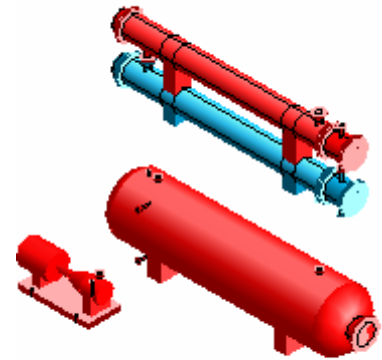
define method .init()
  !this.clear()
  !packet = object EDGPACKET()
  !packet.elementPick(|Pick Equipments <esc> to add to list|)
  !packet.description = |Identify Equipment|
  !packet.action = |!!exampleEDG.identify(!this.return[1].item)|
  !packet.close = |!!exampleEDG.setInfo()|
  !!EDGCtrl.add(!packet)
endmethod

define method .identify(!pick is DBREF)
  do
    if !pick.type.eq(|EQUI|) or !pick.type.eq(|WORL|) then
      break
    else
      !pick = !pick.owner
    endif
  enddo
  if !pick.type.eq(|EQUI|) then
    if !this.storage.findfirst(!pick).unset() then
      !!gphDrawlists.drawlists[1].highlight(!pick, 314)
      !this.storage.append(!pick)
    else
      !this.storage.remove(!this.storage.findfirst(!pick))
      !!gphDrawlists.drawlists[1].unhighlight(!pick)
    endif
  endif
endmethod

define method .setInfo()
  !block = object BLOCK(|!this.storage[!evalIndex].flnn|)
  !names = !this.storage.evaluate(!block)
  !this.txt1.val = !names
endmethod

define method .clear()
  do !i values !this.storage
    !!gphDrawlists.drawlists[1].unhighlight(!i)
  enddo
  !!edgCtrl.remove(|Identify Equipment|)
  !this.txt1.clear()
  !this.storage.clear()
endmethod

```

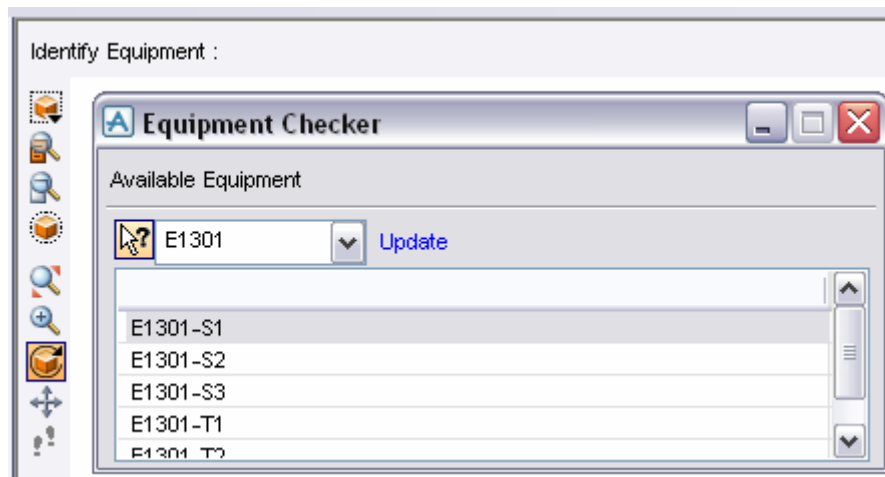


This example shows how a method can take the returned information and use it. In this case, the picked element type is checked whether it is a piece of equipment. If it is not, the method loops up the hierarchy until one is found (or the world is reached). The EQUI is then highlighted if its new or unhighlighted if already chosen. There is a different method for the close action. For this reason, the picked elements are collected a form member. The close method applies the form member to the textpane gadget.

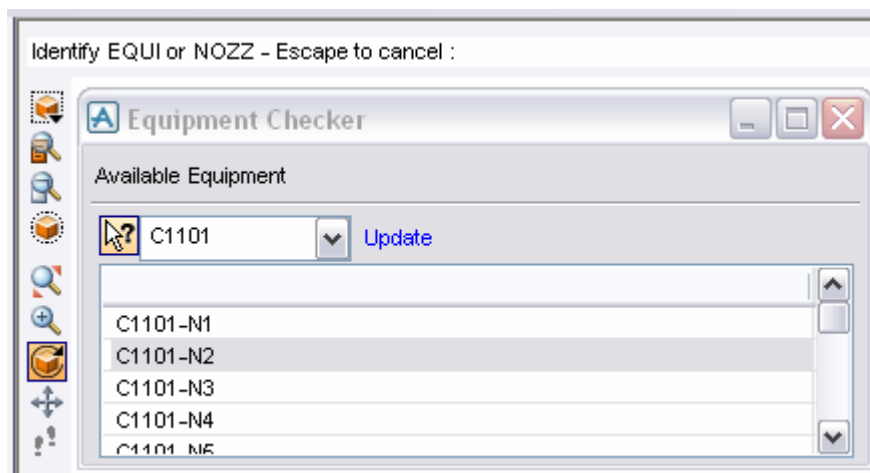
i Notice how the form uses its **.clear()** method

Exercise 10 – Adding EDG to forms

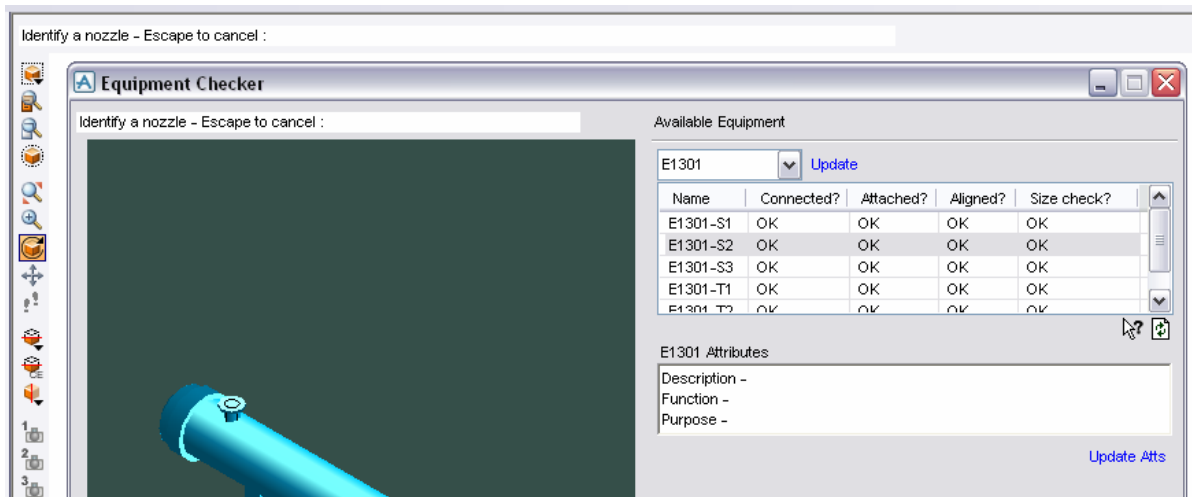
- 1
 - Add a pixmap button to the form created in exercise 8 that will allow users to pick which equipment they want from the main 3D window. Once picked, that piece of equipment (if available) should be selected in the **OPTION** gadget
 - The button should initialise an EDG event. This EDG event should run a method on picking an element that does the following:
 - o Turn the EDG event off after something has been chosen
 - o Identify the type of element chosen
 - o If an EQUI has been chosen, OK
 - o Otherwise, loop up the hierarchy to find an EQUI.
 - o If an EQUI cannot be found, report to the user and re-initialise the EDG
 - o Find the chosen EQUI in the **OPTION** gadget and set it to the correct equipment
 - o Run the nozzle collection method



- 2
 - Extend the method to allow **NOZZ** elements to be chosen (as well as EQUIs). When a nozzle is chosen, highlight it in the **LIST**. Consider what happens if a nozzle is chosen which is not in the list, but is owned by a piece of equipment in the **OPTION** gadget.



- 3
- Add an EDG event to form created in exercise 9 that will allow the users to make their pick within the form's volume view. As the view will only contain one piece of equipment, the EDG should only be able to pick NOZZ elements.
 - For this method to work, several **INMODEs** need to be applied to the volume view. Using **show !!pmlforms**, have a look at the volume view in the main **!!gph3Ddesign** form. Take a look at the function which defines this form (**gph3ddesign.pmlfnc**)



- The volume view on the form will need to be registered with the **!!EDGCtrl** object before it can be picked from. Use the **.addView()** and **.removeView()** methods on the **!!EDGCtrl** object to manage this.

10 Miscellaneous

The following sections described some other useful PML actions.

10.1 Recursive Pml2 Functions

PML2 functions may be called recursively. This can be used to produce iterative solvers:

```
define function !!fibonacci(!last is REAL, !previous is REAL)
  !next = !last + !previous
  !!resultArray.append(!next)
  if !!resultArray.Size().lt(100) then
    -- Here the function calls itself
    !!fibonacci(!next, !last)
  endif
endfunction
```

This iterative solver can be used to compile an array of Fibonacci numbers (an increasing series of numbers, where the next number is the sum of the previous two). The example requires a global variable !!resultArray before it will work.

10.2 Undo and Redo

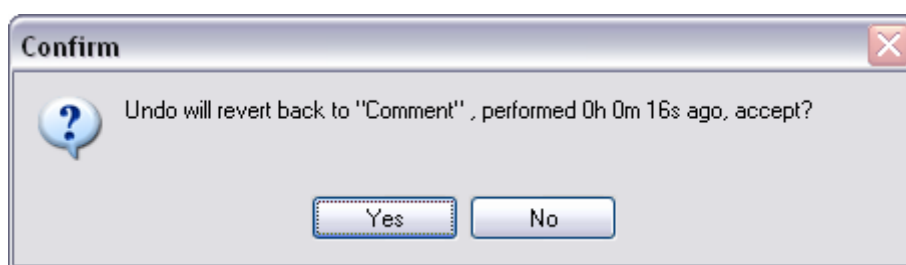
If you build an application that creates or deletes items from the PDMS Database it is good practice to handle the ability to undo the modifications. Undo is a useful feature that users expect to be able to use.

10.2.1 Marking the database

The undo button goes back to the last database mark or savework so your application should mark the database before modification.

MarkDB 'Comment'

The text string is output if the **Undodb** command is used



10.2.2 Undo and Redo Database Commands

- | | | |
|--------|---|--|
| UndoDB | - | Undo the database back to the last database mark |
| RedoDB | - | Redo the last Undo |

These actions can be performed by including the above lines within your code. There is also an UNDOABLE object with built-in methods which can extend this functionality further.

10.3 Error tracing

It is possible to list all the commands that PDMS runs when a PML operation is carried out. This list can either be printed to the command window, or written to an external file.

Type **\$r109 /c:\log.log** onto the command line

where **c:\log.log** is the output file name (it shall be created if it does not exist)

Now every PML action will be written to the external file. The file will list every line of code and action carried out. Lines read will be indicated by a line number in square brackets. Lines not read will be indicated by a line number between round brackets. Entry and exit points between methods, functions and objects are indicated as well as any errors.

Type **\$r110** onto the command line to print the same lines to the command window.

 *Printing to the screen should only be done when a small number of lines are expected. The command window may run out of lines to display the information*

Once you are finished with error tracing, type **\$r** to the command line. If the file is not closed, information will continue to be added to it

 *This will need to be typed before the output file can be opened*

 *To find out more information about the \$r command, type **\$HR** into the command window.*

10.4 PML Publisher

It is now possible to encrypt any files you create before sharing them. Once encrypted, the files can still be used in any compatible AVEVA program, but they are not easily read through a normal text editor. Encrypted files may be used without additional licenses, but the encryption utility described below is separately distributed and licensed.

 *Once encrypted, PML cannot be decrypted. A reference copy of all PML should be kept safe*

The default encryption is implemented using the Microsoft Base Cryptographic Provider, which is included in, among other operating systems, Windows 2000 and Windows XP. There is a limit to the strength of this encryption and is not secure against all possible attacks. AVEVA may release improved algorithms with future releases of the product. If this happens, encrypted files would require re-encrypting.

Move the pmlencrypt.exe to the root folder where the files for encryption exist. Write a local .bat file containing the following line, where **-pmllib** is an instruction to encrypt PML files, **ForEncryption** is a folder containing the source file and **Encrypted** is a folder where the encrypted files shall be put

pmlencrypt -pmllib ForEncryption Encrypted

If the example form **!!nameCE** was encrypted (from page 17) the following file would be created:

```
--<004>-- Published PML 1.0 >--
return error 99 'Unable to decrypt file in this software version'
$** abdf919b3008494b6399edda08b66004
$** MR+zhtg-egE2Ig9IiHSVmdPo08ChKexa7wbfcy0DTbfjTFWU02pK3v4sXq5i
$** TKW3dEFRJCd60uSzaLXdc5fvLe0KqX071uFLZ1vEsI00Hvq8viAwis4rGXg
$** XLgFFVG7mpsmnFtrQDN3o51aiAgicFS6u08C7r8IaxUTUQA0dXeBmlp4TLXc
$** 9KR5LtAIugLrC9a7NxbF+0Hn-c5t0hUAEBG
```

 *Reading an encrypted file is slower than reading a decrypted one. Making use of the Buffer argument can help. Refer to the PML Publisher User Guide for more information*

11 Menu Additions

Previous to 11.6, if you required additions to the menus, you copied the **FSYSTEM/DBAR** files and manually edited them to add new menus.

Every time a new version of PDMS is released, you have to update any files you have copied.

As part of the .net work, this process has been removed so that no update will be needed in subsequent versions of PDMS. This process has been replaced with the new **ADDIN** File syntax.

To add new menus and toolbars to PDMS 11.6 or later, the **ADDIN** file syntax (**FSYSTEM/DBAR**) is no longer valid.

11.1 Miscellaneous Notes

To find out the menus currently contained within a model, use **q var !!appcatmain** for Paragon and **q var !!appdesmain** for Design.

More examples can be found in directory pmlib/design/objects. All the add-in application objects begin with the letters app****.pmlobj e.g. **appaccess.pmlobj**.

11.2 Modules that use the new Addins functionality

Addins are available in modules that have a form named appxxxmain.pmlfrm where xxx is the module such as des, cat, dra etc. These modules are Design (des), Draft (dra), Paragon (cat) and Spooler.

 *The spooler form is named spooler.pmlfrm but still uses addins.*

Isodraft currently uses the old FSYSTEM and cannot use addins. Admin and Monitor use separate forms named adminapplic.pmlfrm and monitormain.pmlfrm and also cannot use addins.

11.3 Adding a Menu/Toolbar

For a file created in **%PDMSUI%\DES\ADDINS** directory, the name doesn't matter, except for identification. The file will contain similar information to the old applications file in **%PDMSUI%**. This file is used for menu additions, new toolbars and standard application switching

11.4 Application Design

Below is an example of an ADDIN file which is used to load the Equipment application within design. Notice how the file only contains a number of file references. The addin file is automatically run by PDMS, so whatever this file references will also be run.

name:	EQUI	
directory:	EQUI	
showOnMenu:	TRUE	
object:	appEqui	
title:	Equipment	
callback:	CALLE XEQUIAPP	Macro to call when the menu is selected
synonym:	CALLE	Synonym to call equipment macros
startup:	\$M /%PDMSUI%\DES\EQUI\INIT	Macro to run on startup of the EQUI application (for the first time)

11.5 Form and Toolbar Control

Forms are controlled using the APPFORMCNTRL object. Forms can be registered to:

- Appear only in certain applications
- Be reshown when PDMS restarts (serialisation)
- !!appFormCntrl.registerForm(!form is FORM)

Toolbars are controlled using the **APPTBARCNTRL** object: to avoid having too many toolbars on screen, each is enabled only when the user is in the right application. Users can add more forms/toolbars to these control objects, and those items will be shown to the user.

11.6 Converting Existing Applications

For users upgrading from the **FSYSTEM/DBAR** method, it is possible to transfer your existing files to the **ADDIN** syntax:

- Existing application has an entry in the PDMSUI%DES\DLTS\APPLICATIONS file
- New application has an add-in definition file in %PDMSUI%DES\ADDINS

OLD	NEW
# Equipment Application	Name: EQUI
ApplicationDirectory:	ShowOnMenu: TRUE
Title:	Directory: EQUI
Form:	Title: Equipment
Callback:	Object: APPEQUI
Synonym:	Callback: CALLE XEQUIAPP
	Synonym: CALLE
	Startup: \$M /%PDMSUI%DES\EQUI\INIT

Previously, the main form was swapped when changing application. Existing applications defined menus and toolbars in the DBAR macro, which was called from the FSYSTEM macro. Now the main form stays the same and menus are shown and hidden as needed. Menus and toolbars are defined in an add-in object.

11.6.1 DBAR and Add-in Object

The contents of the ADD-IN object and the DBAR macro are very similar. The difference is the syntax used.

DBAR macro

```
$* Main Bar Definitions (Application specific)
label /BAR
  add |Position| _POSITION
  add |Orientate| _ORI
  add |Connect| _CONN
return

$* Create Pull Down Data
label /CREATE
  add sep
  add |Equipment...| FORM_CDEQUI
  add |Sub-Equipment...| FORM_CDSUBEQ
  add |Primitives...| FORM_CDPRIM
  add |Standard...| FORM_CDSTAND
return

$* Data for Form (Application specific)
label /DATA
  exit
return

$* Menu Definitions (for application specific bar)
label /MENU
  menu _POSITION
  add |Explicitly (AT)...| ||pos3DExplicit||
  add |Relatively (BY)...| ||pos3DRelatively||
  add |Move| MENU_MOVE
  add |Drag| MENU_DRAG
  add |Plane Move| MENU_PLMOVE
  add SEPARATOR
  add |Equipment Point| MENU_PPEAT
  exit
  menu _PPEAT
  add |At Explicit...| FORM_CDEPEX
  add |At General...| FORM_CDEPATG
  add |Through...| FORM_CDEPMOVT
  add |Clearance...| FORM_CDEPMOVC
  add |Towards...| FORM_CDEPMOVT
  exit
  menu _MOVE
```

ADD-IN object

```
define method .barMenu()
  !bmenu = object APPBARMENU()
  !bmenu.add(|Position|, 'POSITION')
  !bmenu.add(|Orientate|, 'ORI')
  !bmenu.add(|Connect|, 'CONN')
  !appMenuCntrl.addBarMenu(!bmenu, 'EQUI')
endmethod
-- End of method definition for .barMenu()

define method .createMenu()
  !menu = object APPMENU('sysCrt')
  !menu.add(|SEPARATOR|)
  !menu.add(|FORM|, |Equipment...|, 'CDEQUI', 'equiEquipment')
  !menu.add(|FORM|, |Sub-Equipment...|, 'CDSUBEQ', 'equiSubEquipment')
  !menu.add(|FORM|, |Primitives...|, 'CDPRIM', 'equiPrimitives')
  !menu.add(|FORM|, |Standard...|, 'CDSTAND', 'equiStandard')
  !appMenuCntrl.addMenu(!menu, 'EQUI')
endmethod
-- End of method definition for .createMenu()

define method .toolbars()
  frame .equiToolbar toolbar 'Equipment Toolbar'
  --define buttons
  !appTbarCntrl.addToolbar('equiToolbar', 'EQUI')
endmethod
-- End of method definition for .toolbars()

define method .menus()
  !menu = object APPMENU('POSITION')
  !menu.add(|CALLBACK|, |Explicitly (AT)...|, ||pos3DExplicit||, 'Explicitly')
  !menu.add(|CALLBACK|, |Relatively (BY)...|, ||pos3DRelatively||, 'Relatively')
  !menu.add(|MENU|, |Move|, 'MOVE', 'Move')
  !menu.add(|MENU|, |Drag|, 'DRAG', 'Drag')
  !menu.add(|MENU|, |Plane Move|, 'PLMOVE', 'PlaneMove')
  !menu.add(|SEPARATOR|)
  !menu.add(|MENU|, |Equipment Point|, 'PPEAT', 'EquipmentPoint')
  !appMenuCntrl.addMenu(!menu, 'EQUI')

  !menu = object APPMENU('PPEAT')
  !menu.add(|FORM|, |At Explicit...|, 'CDEPEX', 'AtExplicit')
  !menu.add(|FORM|, |At General...|, 'CDEPATG', 'AtGeneral')
  !menu.add(|FORM|, |Through...|, 'CDEPMOVT', 'Through')
  !menu.add(|FORM|, |Clearance...|, 'CDEPMOVC', 'Clearance')
  !menu.add(|FORM|, |Towards...|, 'CDEPMOVT', 'Towards')
  !appMenuCntrl.addMenu(!menu, 'EQUI')

  !menu = object APPMENU('MOVE')
```

Add to bar menu

Add to Create menu

Define toolbars

Add new menus to form

11.6.2 Bar Menu and Toolbar

Add to bar menu

OLD

label /BAR

add |Position| _POSITION

add |Orientate| _ORI

add |Connect| _CONN

return

NEW

define method .barMenu()

!bmenu = object APPBARMENU()

!bmenu.add('Position', 'POSITION')

!bmenu.add('Orientate', 'ORI')

!bmenu.add('Connect', 'CONN')

!!appMenuCntrl.addBarMenu(!bmenu, 'EQUI')

endmethod

11.6.3 Define toolbar

OLD

label /DATA

-- define buttons

-- for toolbar

return

NEW

define method .toolbars()

frame .equiToolbar toolbar 'Equipment Toolbar'

-- define buttons for toolbar

exit

!!appTbarCntrl.addToolbar('equiToolbar', 'EQUI')

endmethod

11.6.4 Adding to Menus

OLD

label /CREATE

add sep

add |Equipment...| FORM _CDEQUI

add |Sub-Equipment...| FORM _CDSUBEQ

add |Primitives...| FORM _CDPRIM

add |Standard...| form !!equCreateStd

return

NEW

define method .createMenu()

!menu = object APPMENU('sysCrt')

!menu.add('SEPARATOR')

!menu.add('FORM', 'Equipment...', 'CDEQUI', 'equiEquipment')

!menu.add('FORM', 'Sub-Equipment...', 'CDSUBEQ', 'equiSubEquipment')

!menu.add('FORM', 'Primitives...', 'CDPRIM', 'equiPrimitives')

!menu.add('FORM', 'Standard...', 'equCreateStd', 'equiStandard')

!!appMenuCntrl.addMenu(!menu, 'EQUI')

endmethod

11.6.5 Adding new menus to form

OLD

label /MENU

menu _POSITION

add |Explicitly (AT)...| [!!pos3DExplicit()]

add |Relatively (BY)...| [CALLIBD

X3DPOSR]

add |Move| MENU _MOVE

add |Drag| MENU _DRAG

add |Plane Move| MENU _PLMOVE

add SEPARATOR

add |Equipment Point| MENU _PPEAT

exit

...

return

NEW

define method .menus()

!menu = object APPMENU('POSITION')

!menu.add('CALLBACK', |Explicitly (AT)...|, [!!pos3DExplicit()], 'Explicitly')

!menu.add('CALLBACK', |Relatively (BY)...|, [CALLIBD X3DPOSR], 'Relatively')

!menu.add('MENU', |Move|, 'MOVE', 'Move')

!menu.add('MENU', |Drag|, 'DRAG', 'Drag')

!menu.add('MENU', |Plane Move|, 'PLMOVE', 'PlaneMove')

!menu.add('SEPARATOR')

!menu.add('MENU', |Equipment Point|, 'PPEAT', 'EquipmentPoint')

!!appMenuCntrl.addMenu(!menu, 'EQUI')

...

endmethod

11.7 Example object including toolbars, bar menus and menus

This is only an example showing adding toolbars, bar menus and menus. The effect of this object will be to add a new toolbar and main menu. It will also add some extra menu options to the EQUI application, including some under the Utilities menu. It will demonstrate how user-defined forms/functions/callbacks can be added to the PDMS menu system.

 *The methods `.modifyForm()` and `.modifyMenus()` run automatically, so can be used to call the objects methods.*

Type out the following and save it as appcompa.pmlobj

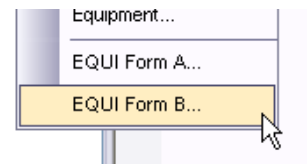
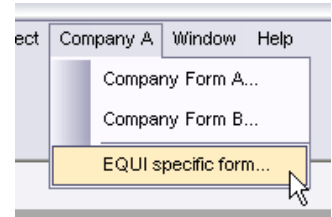
```
define object APPCOMPA
endobject

-----
-- Method:      .modifyForm()
-- Description: Standard method
--              Makes modifications to the main form
-----
define method .modifyForm()
    !this.toolbars()
endmethod

-----
-- Method:      .modifyMenus()
-- Description: Standard Method
--              Runs all other menu creation methods
-----
define method .modifyMenus()
    !this.barMenu()
    !this.menus()
    !this.specificMenus()
    !this.UtilsMenu()
endmethod

-----
-- Method:      .UtilsMenu()
-- Description: Adds to the Utilities menu in the EQUI application
-----
define method .UtilsMenu()
    !menu = object APPMENU('SYSUTIL')
    !menu.add('SEPARATOR')
    !menu.add('CALLBACK', |EQUI Form A...|, |$p Form A|)
    !menu.add('CALLBACK', |EQUI Form B...|, |$p Form B|)
    !!appMenuCntrl.addMenu(!menu, 'EQUI')
endmethod

-----
-- Method:      .toolbars()
-- Description: Creates toolbar for all applications
-----
define method .toolbars()
    frame .MyFormToolbar toolbar 'CompanyA Toolbar'
        !iconSize = !!comSysOpt.iconSize()
        !Pixmap1 = !!pml.getPathName('createpipe-' & !iconSize & '.png')
        !Pixmap2 = !!pml.getPathName('exclamation-' & !iconSize & '.png')
        button .but1 'Show Form A' tooltip 'Show Company Form A' $
        pixmap /$!<Pixmap1> width $!iconSize height $!iconSize callback |$p Form A|
        button .but2 'Show Form B' tooltip 'Show Company Form B' $
        pixmap /$!<Pixmap2> width $!iconSize height $!iconSize callback |$p Form B|
        exit
        !!appTbarCntrl.addToolbar('CompAToolbar', 'ALL')
endmethod
```



```

-----
-- Method:      .barMenu()
-- Description: Adds options to the bar menu for all applications
-----
define method .barMenu()
  !bmenu = object APPBARMENU()
  !bmenu.add(|Company A|, 'CompAMenu')
  !!appMenuCtrl.addBarMenu(!bmenu, 'ALL')
endmethod

-----
-- Method:      .menus()
-- Description: Creates menus to the bar for all applications
-----
define method .menus()
  !menu = object APPMENU('CompAMenu')
  !menu.add('CALLBACK', |Company Form A...|, |$p Form A|)
  !menu.add('CALLBACK', |Company Form B...|, |$p Form B|)
  !!appMenuCtrl.addMenu(!menu, 'ALL')
endmethod

-----
-- Method:      .specificMenus()
-- Description: Creates menus to the bar for specific applications
-----
define method .specificMenus()
  !menu = object APPMENU('CompAMenu')
  !menu.add('SEPARATOR')
  !menu.add('CALLBACK', |EQUI specific form...|, |$p EQUI specific|)
  !!appMenuCtrl.addMenu(!menu, 'EQUI')
endmethod

```

To call the above object, copy out the following into a new file called **COMPA** (no file extension) and save it into %PDMSUI%\des\laddin.sl. Update %PDMSUI% in PDMS.bat if not done already.

```

name:      COMPA
title:     Company A app
showOnMenu: FALSE
object:    APPCOMPA

```

Where: Name is the unique id for user and addin code; Title is the text description of the add-in; ShowOnMenu determines whether the add-in has an entry on the Applications menu, i.e. whether the add-in defines an application (false by default). If it were an application addin like MDS this would be TRUE; Object is the object file that is used to define the menu/toolbar additions.

These are some optional lines that can be included. **Callback** (typically CALLE XEQUI Macro to call when the menu is selected) **Startup** (typically \$m/%pdmsui%/des/equi/init Macro to run on startup of the equi app (for first time)) **ModuleStartup** (the callback run when the PDMS module first starts) **StartupModify** (Name of application to modify and the callback run when an application is first started, separated by a colon.e.g. EQUI:!!equiAppStartupCall()) **SwitchModify** (Name of application to modify and the callback to run when the application is switched to, separated by a colon. e.g. PIPE:!!pipeAppSwitchCall())

To add a startup macro to more than one application, add more entries into the add-in file typically:-
 SwitchModify:EQUI:!!equiAppStartupCall()
 SwitchModify:PIPE:!!equiAppStartupCall()

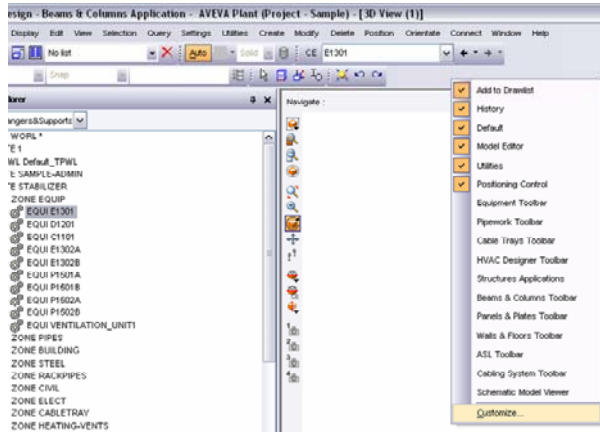
The following keys are only used for applications, i.e. add-ins that have a menu option on the Applications menu and can be switched to.

Menu	Entry for application on the applications menu (the title is used if this isn't specified)
Directory	Name of the application directory under %PDMSUI%\module
Synonym	Synonym created for the directory (only used if the directory is specified)
Group	Group number (applications with the same group number are put into a submenu together)
GroupName	Description of submenu to appear on main menu

It is possible to create toolbars and menus with a PDMS session by choosing Customize... from the available toolbars.

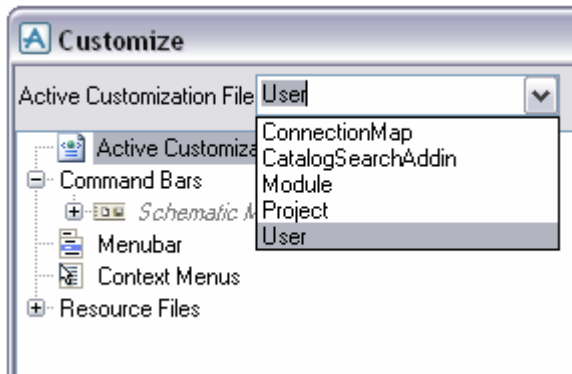
Worked Example – Creating a toolbar within Design

1



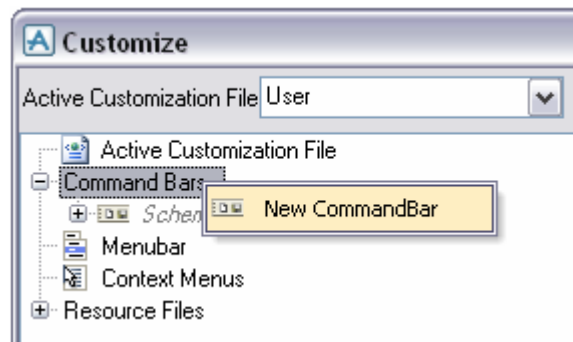
- Open the Customisation Form. Do this by right clicking on an empty part of the toolbar and choose Customize...

2



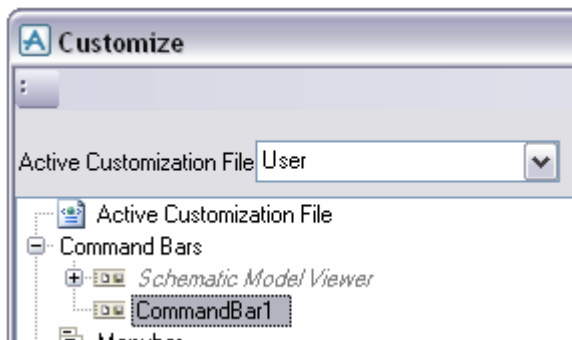
- On the Active Customization File, choose User.
- This will save any changes to a specific user file (file path given on the right hand side of the form)

3

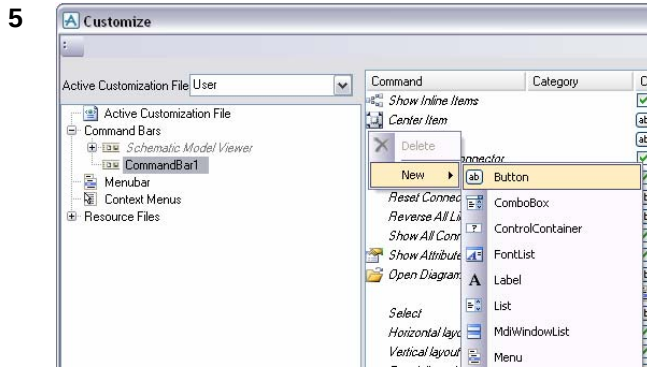


- Right click on the Command Bars heading in bottom window, and click on New CommandBar

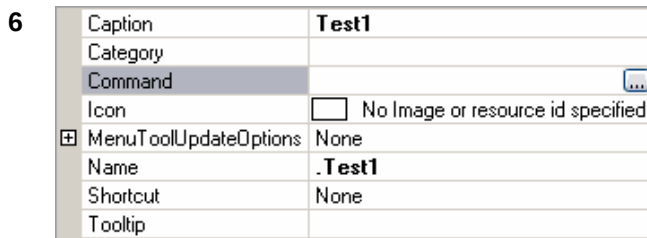
4



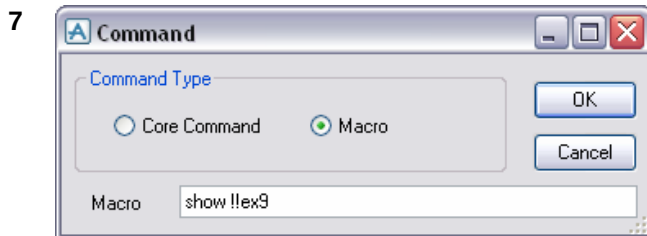
- You will now see a preview of the command bar in the top left of the form (when the new CommandBar is selected)



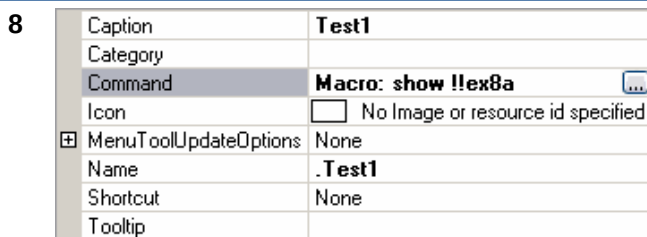
- Right click in the middle window in the form
- Create a new button object in the tools window



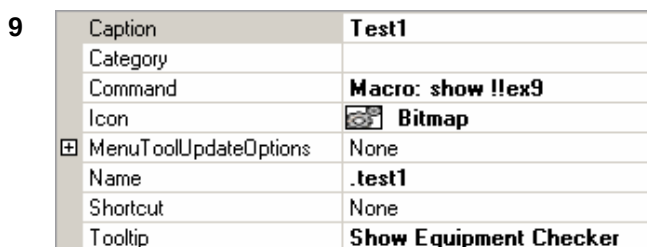
- With the new button selected, look to the right window on the form
- Select the line titled Command and click the small "..." button which appears on the right. This will let us set the command.



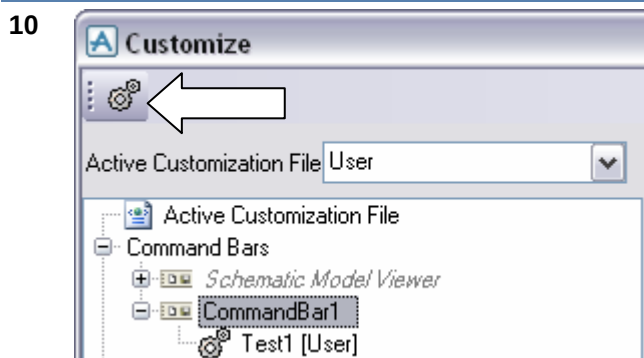
- A command can be a core command or a PML command.
- Choose Macro, and type **show !!ex9**. Close the form by pressing OK.



- The associated command is now
- Select the line titled **Icon**. Click the "..." button that appears to the right.
- Browse for the file equi.png (a supplied file)
- Click Open
- displayed in the properties window

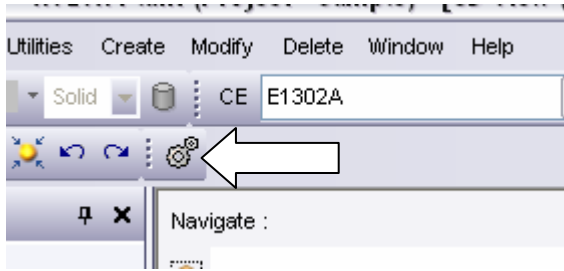


- The icon is now displayed in the right hand window.
- Set the tooltip to 'Show Nozzle Checker'



- Select the new button in the middle window.
- Drag and Drop the new button beneath the new CommandBar object to make the association.
- Select the CommandBar object and notice the button is displayed in the preview.

11



- Apply and OK the form and the new Command Bar will be displayed in the main application.
- Test the button

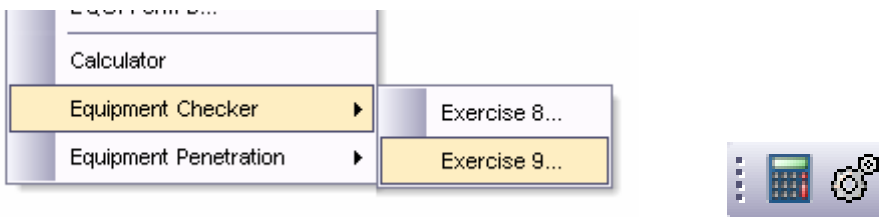
Exercise 11 – Add a Utility and Toolbar menu

1

- Create some additional menus that will sit within the Equipment application and allow you access to your forms.
- Create an addin file that will point to the definition of the new menus. Call your file **PML** (with no file extension) and save it to **%pdmsui%/des/addins**. The addin should call apppml.pmlobj
- Once the files are saved, type **PML REHASH ALL** and the **DESIGN** to re-enter Design. Test the menus in a Design session

2

- Create a user toolbar using the Customize... option in Design. Use some of the supplied icons to create buttons that will show some the form created.
- Test the buttons



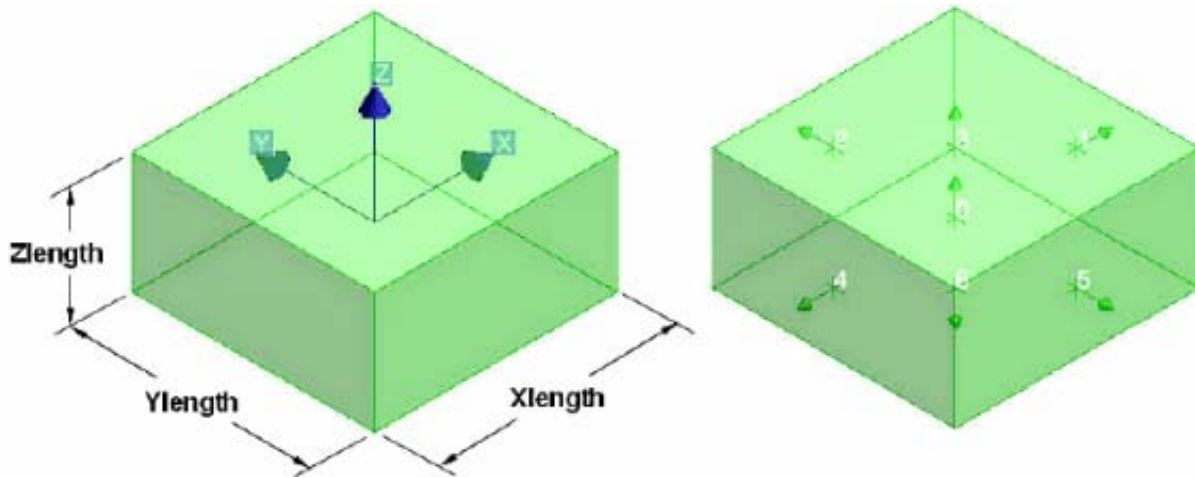
i Please note that even a simply spelling mistake in the above exercise can prevent design from loading

If Design fails to load, do the following:

- Read the error message in the command window and action appropriately
- If objects are “not found”, check the file paths and type **PML REHASH ALL**
- Once the error has been fixed, type **DESIGN** onto the command line and Design should load.
- If typing **DESIGN** generates an error, the command line is still in setup mode and needs exiting – type **EXIT**
- To close the command line, type **FINISH**

Appendix A – PDMS Primitives

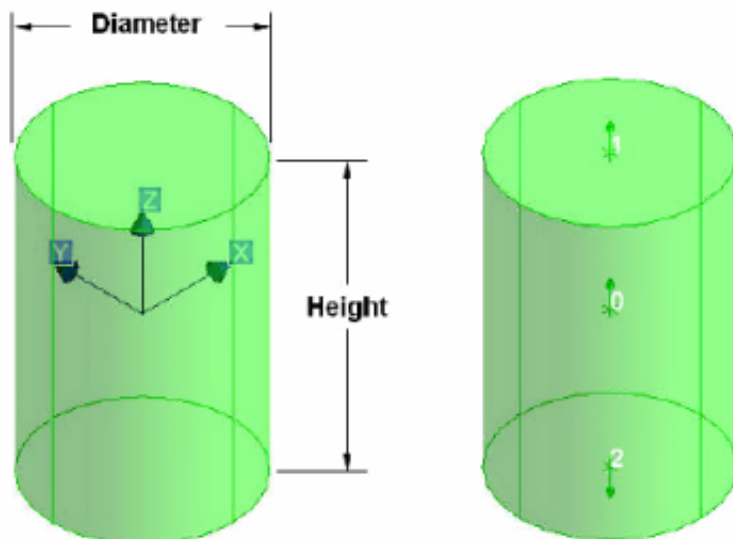
Box (BOX)



Specific geometric attributes:

Xlength	Length parallel to X axis
Ylength	Length parallel to Y axis
Zlength	Length parallel to Z axis

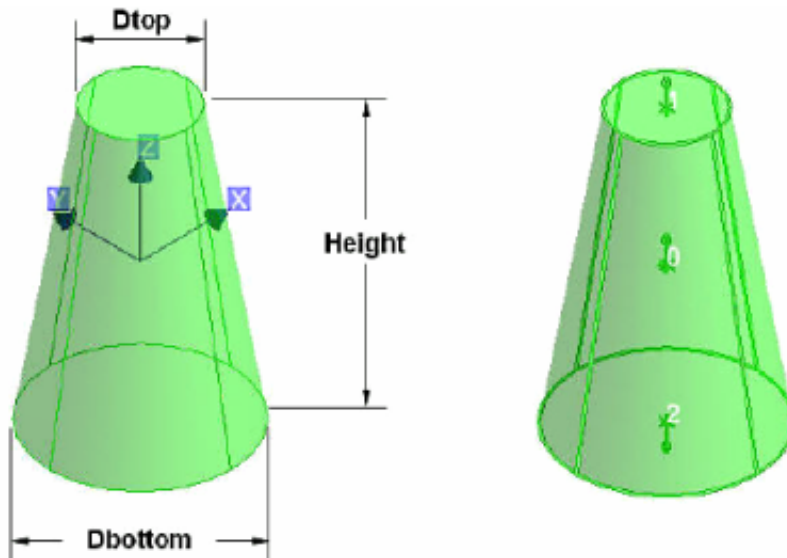
Cylinder (CYLI)



Specific geometric attributes:

Diameter	Diameter of cylinder
Height	Length parallel to Z axis

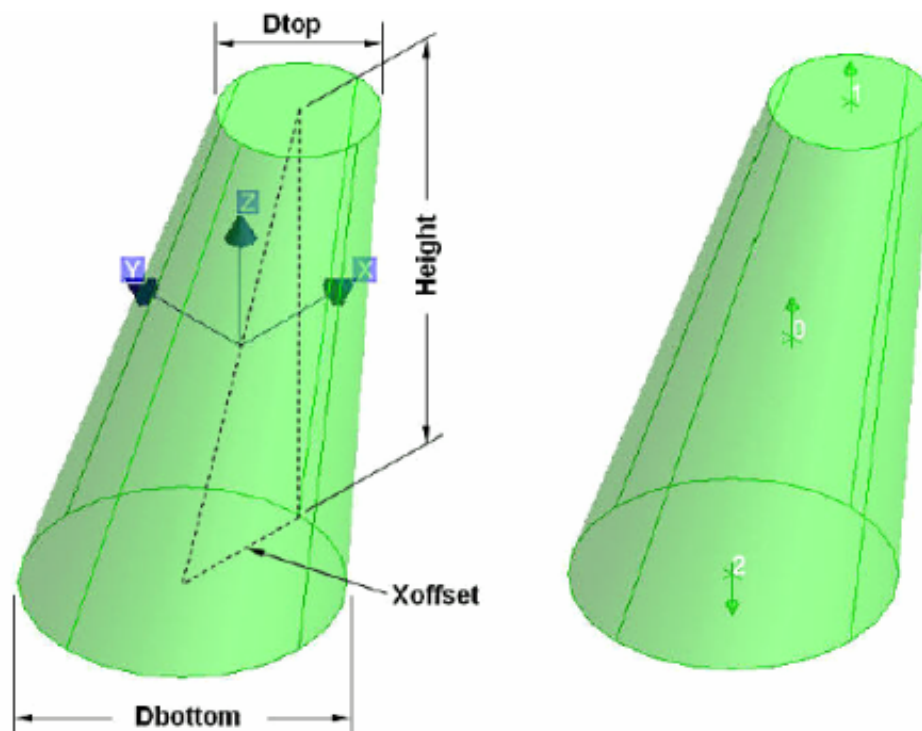
Cone (CONE)



Specific geometric attributes:

Dtop	Diameter at top of cone
Dbottom	Diameter at bottom of cone
Height	Length parallel to Z axis

Snout (SNOU)

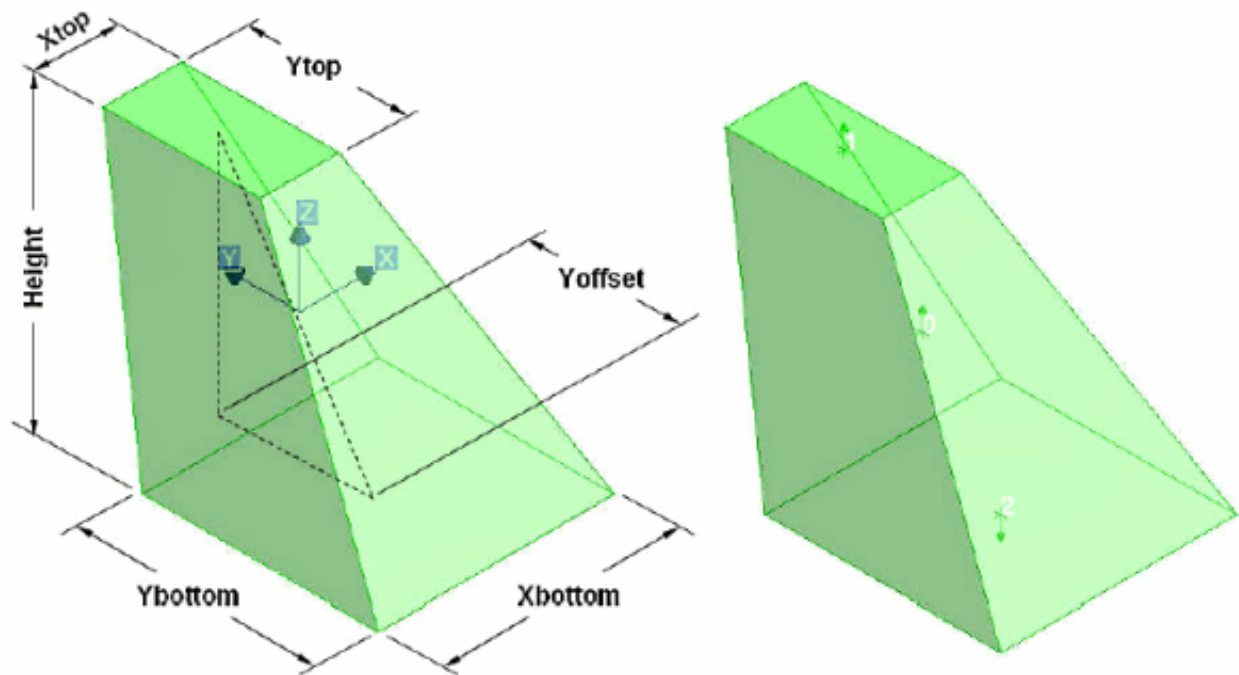


Specific geometric attributes:

Dtop	Diameter at top of snout
Dbottom	Diameter at bottom of snout
Xoffset	Offset of centre of top from centre of bottom on X axis
Yoffset	Offset of centre of top from centre of bottom on Y axis
Height	Length parallel to Z axis

i Only an Xoffset is show in this example, however, both Yoffset and Xoffset may be set.

Pyramid (PYRA)

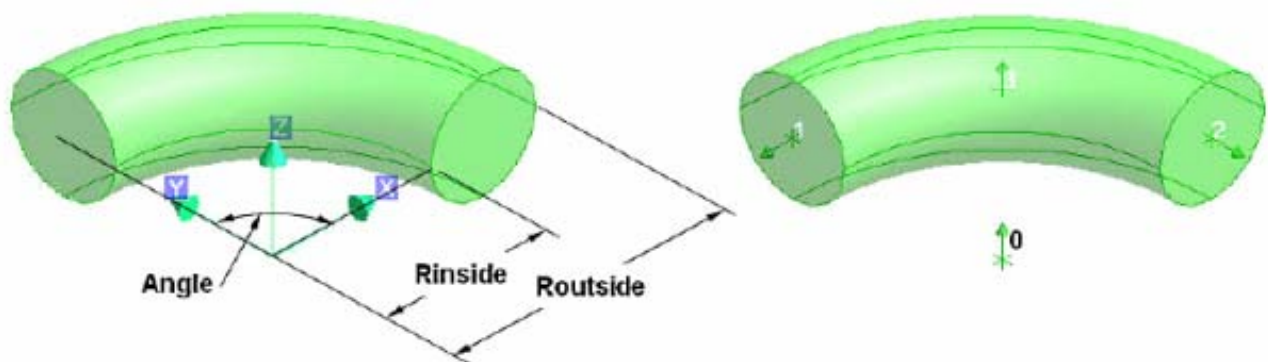


Specific geometric attributes:

Xbottom	Length of bottom of pyramid parallel to X axis
Ybottom	Length of bottom of pyramid parallel to Y axis
Xtop	Length of top of pyramid parallel to X axis
Ytop	Length of top of pyramid parallel to Y axis
Height	Length parallel to Z axis
Xoffset	Offset of centre of top from centre of bottom on X axis
Yoffset	Offset of centre of top from centre of bottom on Y axis

① Only a Yoffset is shown in this example, however, both Yoffset and Xoffset may be set.

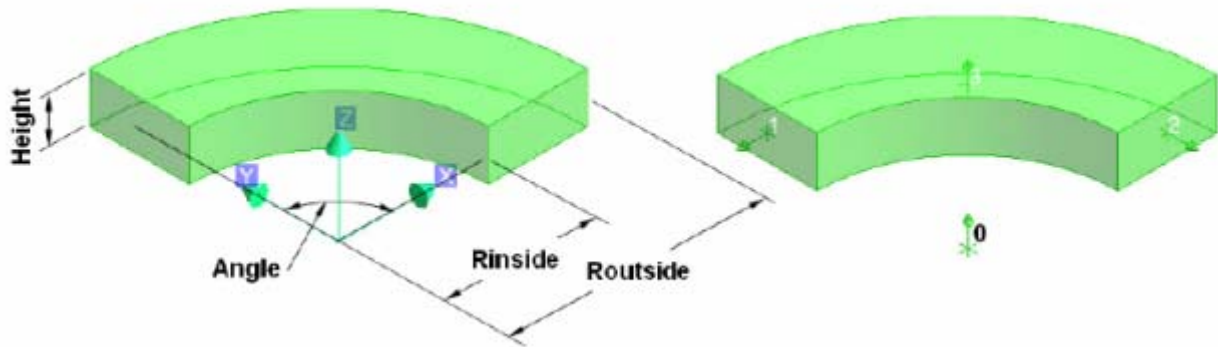
Circular Torus (CTOR)



Specific geometric attributes:

Rinside	Inside radius in XY plane
Routside	Outside radius in XY plane
Angle	Subtended angle (maximum 180°)

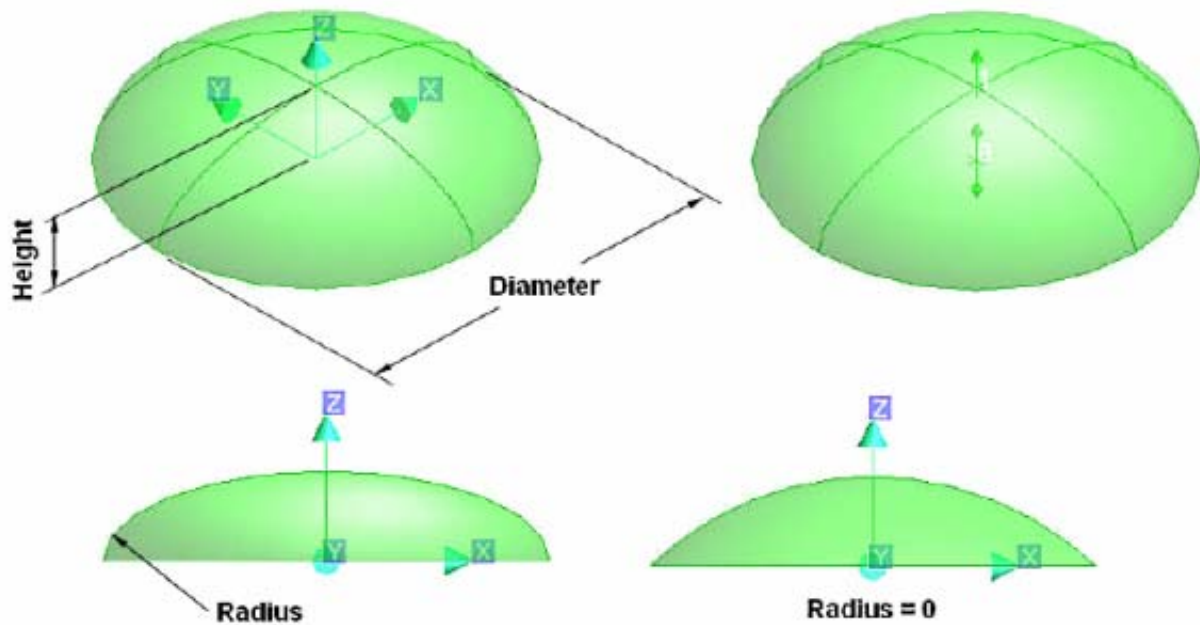
Rectangular Torus (RTOR)



Specific geometric attributes:

Rinside	Inside radius in XY plane
Routside	Outside radius in XY plane
Height	Length parallel to Z axis
Angle	Subtended angle (maximum 180°)

Dish (DISH)

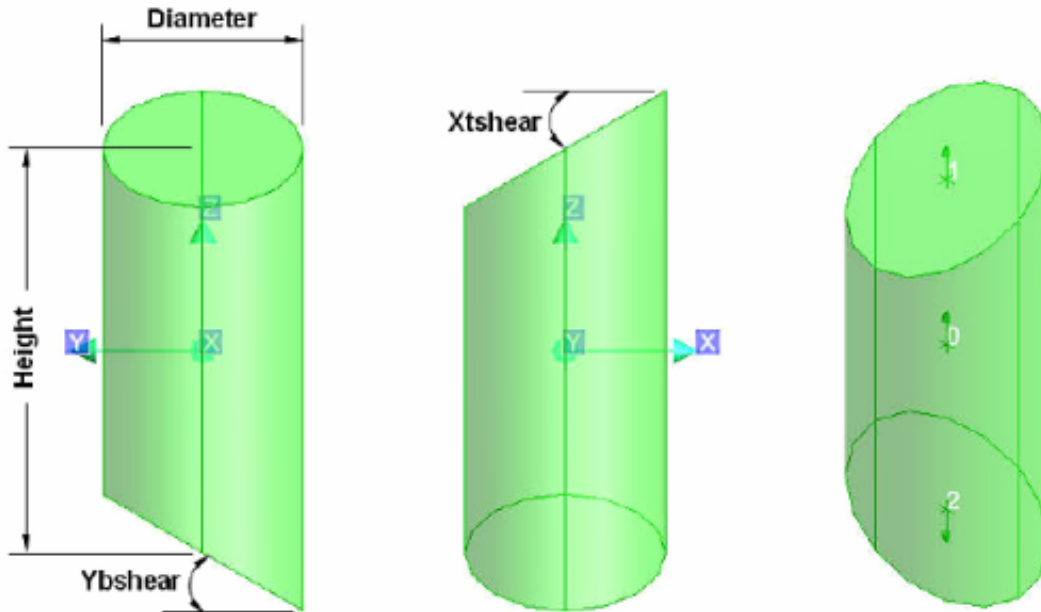


Specific geometric attributes:

Diameter	Diameter of dish in XY plane.
Height	Height of dish parallel to Z axis
Radius	Knuckle radius

i If the knuckle radius is 0 then the dish is represented as a segment of a sphere. If the knuckle radius is greater than 0 then the dish is represented as a partial ellipsoid, generally used to represent a torispherical end to a vessel.

Sloped Cylinder (SCYL)

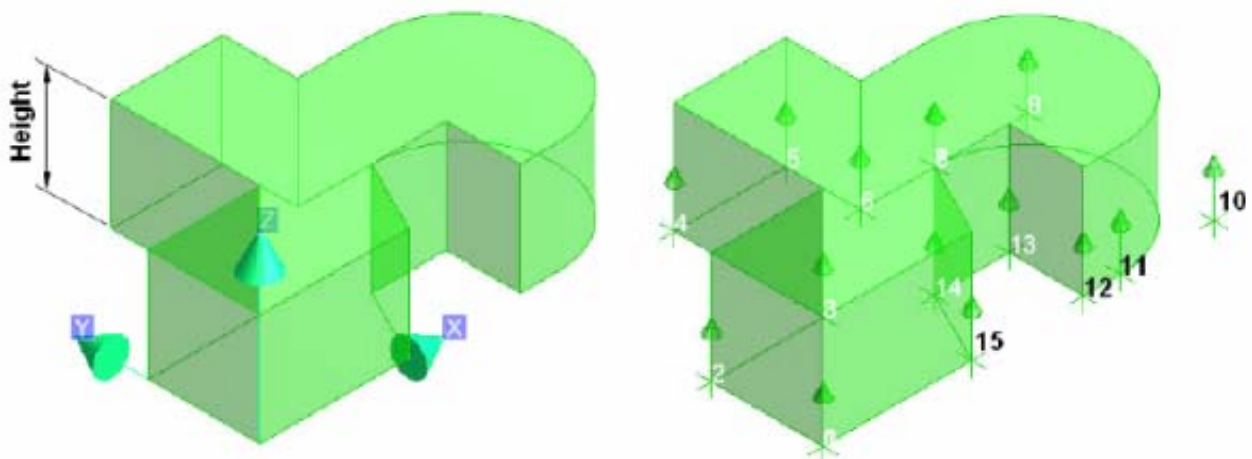


Specific geometric attributes:

Diameter	Diameter of sloped cylinder
Height	Length in Z axis from bottom centre to top centre
Xtshear	Inclination of top of cylinder in the XZ axis (in degrees)
Ytshear	Inclination of top of cylinder in the YZ axis (in degrees)
Xbshear	Inclination of bottom of cylinder in the XZ axis (in degrees)
Ybshear	Inclination of top of cylinder in the YZ axis (in degrees)

- ① Only an Xtshear and Ybshear are shown in this example, however, Xtshear, Ytshear, Xbshear and Ybshear may be set in any combination to obtain the required results. The values for these attributes may be +ve or -ve.

Extrusion (EXTR)

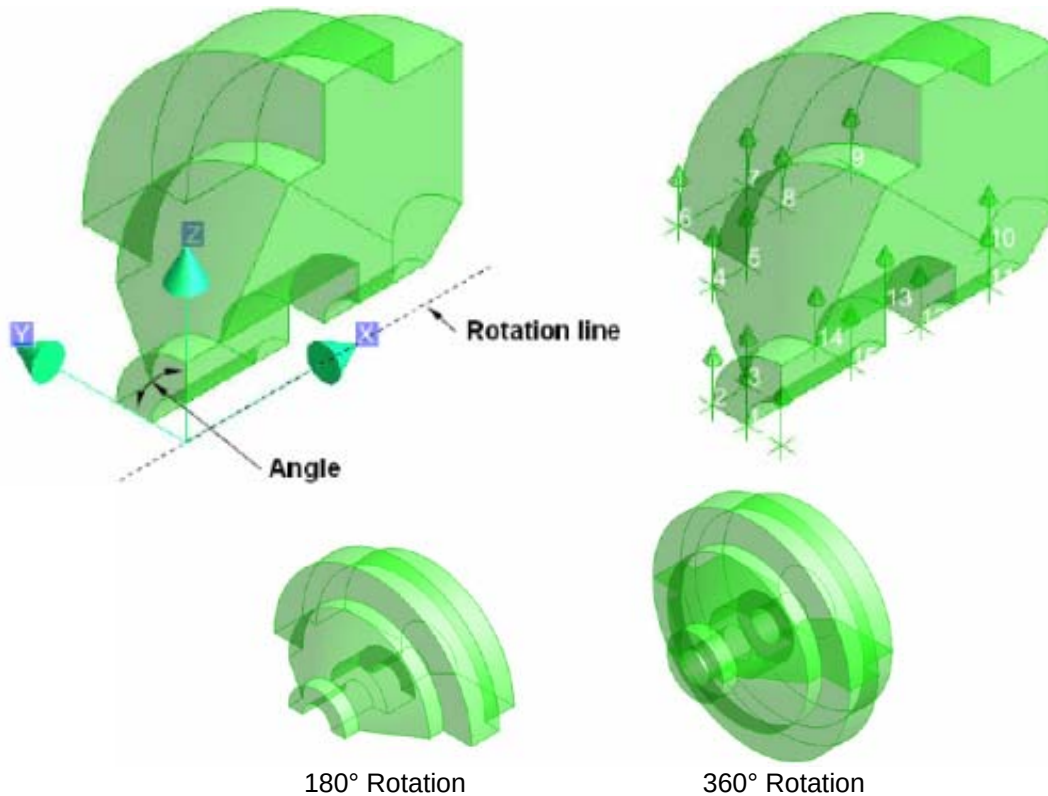


Specific geometric attributes:

Height	Height of extrusion in Z axis
--------	-------------------------------

- ① An extrusion is a 2D shape, defined by a series of vertices at each change in direction, extruded through a height. The primitive consists of three element types, i.e. EXTR, LOOP and VERTs.

Solid of Revolution (REVO)



Specific geometric attributes:

Angle Rotation angle around X axis (selected rotation line)

i A solid of revolution is a 2D shape, defined by a series of vertices at each change in direction, rotated through a specified angle around a specified rotation axis. The primitive consists of three element types, i.e. REVO, LOOP and VERTs.

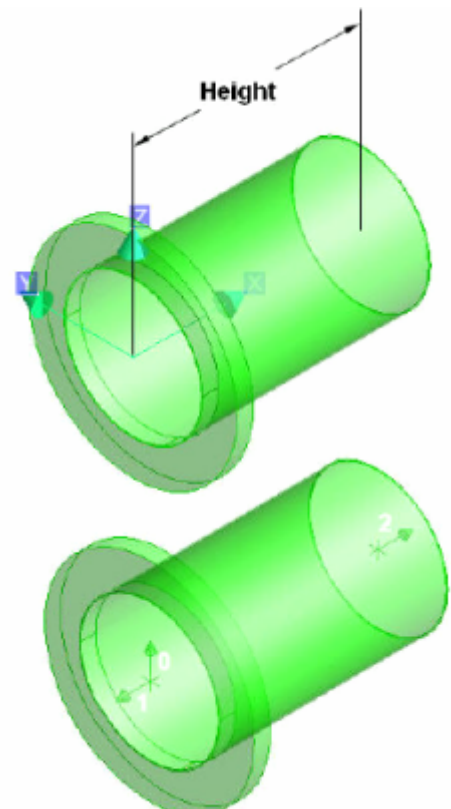
Nozzle (NOZZ)

Although a nozzle is classed as a primitive, it is unlike the other primitives in that its geometry is determined in Paragon as part of a catalogue component. Nozzles of different types and geometry may be constructed in Paragon to suit the requirements of the Piping Specification.

The specific nozzle type is referenced from Paragon using the **Spref** (Specification Reference) attribute.

Specific geometric attributes:

Height Height between nozzle face and end, i.e. from P1 to P2.



Appendix B – Example code

This appendix contains examples of code that provide solutions to each of the exercises. The whole code has not been included in this appendix. Only new/modified code is included so it will require the user to read and understand this code.

Appendix B1 - Example ex2.mac

```
NEW EQUIPMENT /HandWheel

NEW BOX XLEN 100 YLEN 100 ZLEN 100
NEW CYLINDER DIAM 80 HEIG 5
CONN P1 TO P3 OF PREV
NEW BOX XLEN 50 YLEN 50 ZLEN 15
CONN P6 TO P2 OF PREV
NEW DISH DIAM 50 HEIG 15
CONN P2 TO P3 OF PREV

REM ALL
ADD /HandWheel AUTO /HandWheel
```

Appendix B2 - Example ex3.mac

```
$d1=HandWheel
$d2=500

NEW EQUIPMENT /$1

NEW SUBE /$1-Centre
NEW BOX /$1-Centre-Box XLEN 100 YLEN 100 ZLEN 100
NEW CYLINDER DIAM 80 HEIG 5
CONN P1 TO P3 OF PREV
  NEW BOX XLEN 50 YLEN 50 ZLEN 15
CONN P6 TO P2 OF PREV
  NEW DISH DIAM 50 HEIG 15
CONN P2 TO P3 OF PREV

NEW SUBE /$1-Arm-1
  NEW CYLINDER DIAM 50 HEIG ($2/ 2 - 75)
CONN P1 TO P2 OF /$1-Centre-Box
  NEW CTORUS RINS ($2 / 2 - 50) ROUT ($2 / 2)
CONN P0 TO P0 OF /$1-Centre-Box

do !p from 2 to 4
  NEW SUBE /$1-Arm-$!p COPY PREV
  ROTATE BY 90 ABOUT S
enddo

REM ALL
ADD /$1 AUTO /$1
```

Appendix B3 - Example ex4.mac (fixed)

```
$p
$p HOSE REEL MACRO (VERSION 1.0)
$p -----
$p

$P ENTER REFNO OF THE HOSEREEL (e.g. HOSE-REEL-001)

VAR !NAME |HoseReel|
NEW EQUIP /$!NAME

$P
$P BUILDING HOSEREEL...

VAR !WHEELDIA (500)
VAR !REELDIA (1000)
```

Fix1: READ syntax replaced with standard variable declaration

```
NEW SUBE /$!NAME-BASE
NEW BOX XLEN 1200 YLEN 725 ZLEN 75
```

Fix2: TO instead of TOO

```
do !N FROM 0 TO 1
  NEW EXTRUSION HEIG 25
  POS W 0 N (-350 + ($!N * 675)) U 0
  ORI Y is E and Z is N
  NEW LOOP
  NEW VERTEX
  POS E 25 S 500 U 0
  NEW VERTEX
  POS E 25 N 500 U 0
  NEW VERTEX
  POS E ($!REELDIA / 2 + 250) N ($!WHEELDIA / 3) U 0
  NEW VERTEX
  POS E ($!REELDIA / 2 + 250 + $!WHEELDIA / 3) N ($!WHEELDIA / 3) U 0
  FRAD ($!WHEELDIA / 3)
  NEW VERTEX
  POS E ($!REELDIA / 2 + 250 + $!WHEELDIA / 3) S ($!WHEELDIA / 3) U 0
  FRAD ($!WHEELDIA / 3)
  NEW VERTEX
  POS E ($!REELDIA / 2 + 250) S ($!WHEELDIA / 3) U 0
enddo
```

Fix3: FRAD instead of PRAD

```
NEW CYLINDER /$!NAME-BASE-AXLE DIAM 100 HEIG 750
POS E 0 N 0 U ($!REELDIA / 2 + 250)
ORI Y is E and Z is N
NEW NOZZLE
POS E 0 N 450 U ($!REELDIA / 2 + 250)
ORI Y is E and Z is U
HEIG 100
CATR /DICHFL_C/DEZFBRONN
```

```
NEW SUBE /$!NAME-HOSE
POS E 0 N 0 U ($!REELDIA / 2 + 250)
ORI Y is D and Z is S
do !N FROM 0 to 1
  NEW CYLINDER DIAM $!REELDIA HEIG 10
  POS E 0 N 0 U (-305 + ($!N * 610))
enddo
do !I FROM 0 TO 5
  do !J FROM 0 TO 1
    NEW CTORUS RINS (($!REELDIA / 2) - 150) ROUT (($!REELDIA / 2) - 50) ANGL 180
    POS E 0 N 0 D (-250 + ($!I * 100))
    ROTATE BY ($!J * 180) ABOUT S WRT /*
  enddo
enddo
```

Fix4: S instead of S10E

```
NEW CYLINDER DIAM 100 HEIG ($!REELDIA / 3)
CONN P1 TO P2 OF PREV
```

Fix5: P1 to P2 instead of P1 to P1

```
NEW CYLINDER DIAM 140 HEIG 25
CONN P1 TO P2 OF PREV
NEW CYLINDER DIAM 110 HEIG 50
CONN P1 TO P2 OF PREV
NEW CYLINDER DIAM 140 HEIG 25
CONN P1 TO P2 OF PREV
NEW CONE DTOP 110 DBOT 80 HEIG 200
CONN P1 TO P2 OF PREV
NEW REVOLUTION ANGL 360
NEW LOOP
  NEW VERTEX
  NEW VERTEX
  POS W 10 N 0 U 0
  NEW VERTEX
  POS W 10 N 45 U 0
  FRAD 5
  NEW VERTEX
  POS W 0 N 45 U 0
  FRAD 5
REVO
CONN P1 TO P2 OF PREV
ROTATE BY 90 ABOUT D
```

```
NEW SUBE /$!NAME-CENTRE
NEW BOX /$!NAME-CENTRE-BOX XLEN 100 YLEN 100 ZLEN 100
CONN P6 TO P2 OF /$!NAME-BASE-AXLE
NEW CYLINDER DIAM 80 HEIG 5
CONN P1 TO P3 OF PREV
```

```

NEW BOX XLEN 50 YLEN 50 ZLEN 15
  CONN P6 TO P2 OF PREV
NEW DISH DIAM 50 HEIG 15
  CONN P2 TO P3 OF PREV

NEW SUBE /$!NAME-ARM-1
  VAR !POS POS /$!NAME-CENTRE-BOX
  POS $!POS
  NEW CYLINDER DIAM 50 HEIG ($!WHEELDIA / 2 - 75)
  CONN P1 TO P2 OF /$!NAME-CENTRE-BOX
  NEW CTORUS RINS ($!WHEELDIA / 2 - 50) ROUT ($!WHEELDIA / 2)
  CONN P0 TO P0 OF /$!NAME-CENTRE-BOX

DO !P FROM 2 TO 4
  NEW SUBE /$!NAME-ARM-$!P COPY PREV
  ROTATE BY 90 ABOUT S
enddo

REM ALL
ADD /$!NAME AUTO /$!NAME
EQUIP

$P
$P DONE!
$P

```

Fix6: BOX instead of BOXES

Appendix B4 - Example ex4.pmlfnc

```

define function !!ex4(!name is STRING, !wheelDia is REAL)
NEW EQUIP /$!NAME

VAR !REELDIA (1000)

NEW SUBE /$!NAME-BASE
NEW BOX XLEN 1200 YLEN 725 ZLEN 75

do !N FROM 0 TO 1
-----
--
-- As original Macro
--
-----
EQUIP

endfunction

```

Middle part, same as the original macro

Appendix B5 - Example ex5.pmlfrm

```

setup form !!ex5 dialog resizable
!this.formTitle = |Example Form - Exercise 5|
!this.initcall = |!this.init()|
path down
frame .inputFrame |Inputs| at x 0 anchor T+L+B
  frame .tempInputFrame |Temperature conversion (Input)| at x 1
    text .tempInput |Temperature| call |!this.temperatureConvert()| width 10 is REAL
    frame .tempChoiceFrame panel at xmax.tempInput ymin.tempInputFrame + 0.5
      rToggle .celsius |°C| tagwid 2 at xmax.tempInput + 3 ymin.tempInput
      rToggle .fahrenheit |°F| tagwid 2 at xmax.celsius ymin.tempInput
    exit
  frame .tempRangeFrame |Temperature Range| at x 1 width.tempInputFrame
    text .minimum |Minimum| call |!this.check(1)| at x 1 width 10 is REAL
    text .maximum |Maximum| call |!this.check(2)| at x 1 width 10 is REAL
    text .step |Step Size| call |!this.check(0)| at x 1 width 10 is REAL format !!INTEGERFMT
    button .fillFahrenheit linklabel |Fill with °F to °C >>| call |!this.fill('Fahrenheit')| at
xmax.step + 1 ymin.step wid 12
    button .fillCelsius linklabel |Fill with °C to °F >>| call |!this.fill('Celsius')| at
xmin.fillFahrenheit ymin.fillFahrenheit - size wid 12
  exit
  frame .stringSplitFrame |Temperture Split| anchor L+B+R at x 1 width.tempInputFrame
    text .stringInput |Input| width 24 is STRING
    text .delimiter |Delimiter| width 10 is STRING
    button .split linklabel |Split temperature >>| call |!this.split()| at xmax.delimiter + 1
ymin.delimiter wid 12
  exit
exit
frame .results |Results| at xmax + 1 y 0 anchor ALL

```

Fix1: U not valid

Fix2: REAL with one L

```

frame .tempOutputFrame |Temperature conversion (Output)| anchor L+T+R at x 1
width.tempInputFrame height.tempInputFrame
text .tempOutput |Temperature| call || width 10 is REAL
para .unit at xmax.tempOutput ymin.tempInput text || width 5
exit
frame .tempRangeOutFrame |Temperature Results| anchor L+T+R at x1 width.tempRangeFrame
height.tempRangeFrame
list .tempList dock fill columns width 1 height 1
exit
frame .stringSplitOutFrame |Temperature Split Result| anchor L+B+R at x1 ymin.stringSplitFrame
width.stringSplitFrame height.stringSplitFrame
text .number |No. of Temp| width 10 is REAL format !!INTEGERFMT
text .result |Result | width.stringInput is STRING
exit
exit
exit

```

Fix3: Width+height at the end of

```

define method .ex5()
!this.tempChoiceFrame.callback = |!this.temperatureConvert()|
endmethod

```

Fix4: Incorrect constructor name

```

define method .init()
!this.tempInput.val = 0
!this.tempChoiceFrame.val = 1
!this.minimum.val = 0
!this.maximum.val = 100
!this.step.val = 25
!this.stringInput.val = |10°C/30°C/20°C/5°C|
!this.delimiter.val = | |
!this.temperatureConvert()
!this.fill(|Celsius|)
!this.split()
endmethod

```

A new init method applies default values

```

define method .celsiusToFahrenheit(!celsius is REAL) is REAL
!fahrenheit = !celsius * 1.8 + 32
return !fahrenheit
endmethod

```

```

define method .fahrenheitToCelsius(!fahrenheit is REAL) is REAL
!celsius = (!fahrenheit - 32 ) / 1.8
return !celsius
endmethod

```

```

define method .temperatureConvert()
if !this.tempChoiceFrame.val.eq(1) then
!this.tempOutput.val = !this.CelsiusToFahrenheit(!this.tempInput.val)
!this.unit.val = |°F|
elseif !this.tempChoiceFrame.val.eq(2) then
!this.tempOutput.val = !this.FahrenheitToCelsius(!this.tempInput.val)
!this.unit.val = |°C|
endif
endmethod

```

Fix5: F and C the wrong way around

```

define method .fill(!whichTemperature is STRING)
!n = 0
do !temperature from !this.minimum.val to !this.maximum.val by !this.step.val
!n = !n + 1
!tempArray[!n][1] = !n.string()
!tempArray[!n][2] = !temperature.string()
!tempArray[!n][3] = |=|
if !whichTemperature.eq(|Celsius|) then
!fahrenheit = !this.celsiusToFahrenheit(!temperature)
!tempArray[!n][4] = STRING(!fahrenheit,!!realFMT)
else
!celsius = !this.fahrenheitToCelsius(!temperature)
!tempArray[!n][4] = STRING(!celsius,!!realFMT)
endif
enddo
if !whichTemperature.eq(|Celsius|) then
!headings = |No. Celsius = Fahrenheit|
else
!headings = |No. Fahrenheit = Celsius|
endif
!this.tempList.setHeadings(!headings.split())
!this.tempList.setRows(!tempArray)
endmethod

```

```
define method .check(!flag is REAL)
  if !flag.eq(0) then
    if !this.step.val.eq(0) then
      !this.step.val = 1
    endif
  elseif !flag.eq(1).or(!flag.eq(2)) then
    if !this.maxi.val.unset() then
      !this.maxi.val = !this.mini.val + !this.step.val
    elseif !this.mini.val.unset() then
      !this.mini.val = !this.maxi.val - !this.step.val
    endif
    if (!this.mini.val.lt(-273)) then
      !this.mini.val = -273
      if (!this.maxi.val.leq(!this.mini.val)) then
        !this.maxi.val = -272
      endif
    elseif (!this.maxi.val.lt(-273)) then
      !this.mini.val = -273
      !this.maxi.val = -272
    endif
    if (!this.maxi.val.leq(!this.mini.val)) then
      if !flag.eq(1) then
        !this.maxi.val = !this.mini.val + !this.step.val
      elseif !flag.eq(2) then
        !this.mini.val = !this.maxi.val - !this.step.val
      endif
    endif
  endif
endmethod
```

New method to split the string

```
define method .split()
  !split = !this.stringInput.val.trim().split(!this.delimiter.val)
  !this.number.val = !split.size()
  !result = ||
  do !n index !split
    if !split[!n].substring(!split[!n].length().upcase().eq(|C|) then
      !result = !result & !this.celsiusToFahrenheit(!split[!n].substring(1, !split[!n].length() -
2).real()).string(!!INTEGERFMT) & |°F |
    else
      !result = !result & !this.fahrenheitToCelsius(!split[!n].substring(1, !split[!n].length() -
2).real()).string(!!INTEGERFMT) & |°C |
    endif
  enddo
  !this.result.val = !result
endmethod
```

Appendix B6 - Example ex6.pmlfrm

```
setup form !!ex6 dialog resizable
  !this.formTitle = |Example Form - Exercise 6|
  !this.initcall = |!this.init()|
  path down
  frame .tabset TABSET anchor all
    frame .inputFrame |Inputs| at x 0 y 1 dock fill
      frame .tempInputFrame |Temperature conversion (Input)| at x 1 anchor T+L+R
        text .tempInput |Temperature| width 10 is REAL
        frame .tempChoiceFrame panel at xmax.tempInput ymin.tempInputFrame + 0.5
          rToggle .celsius |°C| tagwid 2 at xmax.tempInput + 3 ymin.tempInput
          rToggle .fahrenheit |°F| tagwid 2 at xmax.celsius ymin.tempInput
        exit
      frame .tempRangeFrame |Temperature Range| at x 1 anchor T+L+R width.tempInputFrame
        text .minimum |Minimum| call |!this.check(1)| at x 1 width 10 is REAL
        text .maximum |Maximum| call |!this.check(2)| at x 1 width 10 is REAL
        text .step |Step Size| call |!this.check(0)| at x 1 width 10 is REAL format
!!INTEGERFMT
      exit
      frame .stringSplitFrame |Temperture Split| anchor all at x 1 width.tempInputFrame
        text .stringInput |Input| width 24 is STRING
        text .delimiter |Delimiter| width 10 is STRING
      exit
    frame .results |Results| at xmin.inputFrame ymin.inputFrame dock fill
      frame .tempOutputFrame |Temperature conversion (Output)| anchor L+T+R width.tempInputFrame
height.tempInputFrame
        text .tempOutput |Temperature| width 10 is REAL
        para .unit at xmax.tempOutput ymin.tempInput text || width 5
```

```

        exit
        frame .tempRangeOutFrame |Temperature Results| anchor L+T+R at x1 width.tempRangeFrame
height.tempRangeFrame
        list .tempList dock fill columns width 1 height 1
        exit
        frame .stringSplitOutFrame |Temperature Split Result| anchor all width.stringSplitFrame
height.stringSplitFrame
        text .number |No. of Temp| width 10 is REAL format !!INTEGERFMT
        text .result |Result | width.stringInput is STRING
        exit
        exit
        exit
        para .paAcceptChanges at xmin.tempInputFrame ymax+0.5 anchor L+B pixmap wid 16 hei 16
button .lkAcceptChanges |Accept changes| at xmax.paAcceptChanges+1 ymin anchor L+B linklabel wid
10
button .lkDiscardChanges |Discard changes | at xmax.tempInputFrame - size ymin anchor R+B
linklabel wid 10
para .paDiscardChanges at xmin.lkDiscardChanges - 1.5 * size ymin anchor R+B pixmap wid 16 hei 16
member .data is ARRAY
!menu = !this.newMenu(|celsiusMenu|)
!menu.add(|Callback|, |Switch to °F = °C|, |!this.fill('Fahrenheit')|)
!menu = !this.newMenu(|fahrenheitMenu|)
!menu.add(|Callback|, |Switch to °C = °F|, |!this.fill('Celsius')|)
exit

define method .ex6()
    !this.results.callback = |!this.tabCall(|
    !this.paAcceptChanges.addPixmap(!pml.getPathName('accept.png'))
    !this.paDiscardChanges.addPixmap(!pml.getPathName('discard.png'))
    !this.lkAcceptChanges.callback = |!this.setData(1)|
    !this.lkDiscardChanges.callback = |!this.setData(2)|
endmethod

define method .init()
    !this.tempInput.val = 0
    !this.tempChoiceFrame.val = 1
    !this.minimum.val = 0
    !this.maximum.val = 100
    !this.step.val = 25
    !this.stringInput.val = |10°C/30°C/20°C/5°C|
    !this.delimiter.val = | / |
    !this.setData(1)
endmethod

define method .tabCall(!gad is GADGET, !type is STRING)
    if !type eq |SHOWN| then
        !this.temperatureConvert()
        !this.fill(|Celsius|)
        !this.split()
    endif
endmethod

define method .setData(!flag is REAL)
    -- Either save the values, or bring them back (array of strings)
    if !flag.eq(1) then
        !this.data[1] = !this.tempInput.val
        !this.data[2] = !this.tempChoiceFrame.val
        !this.data[3] = !this.minimum.val
        !this.data[4] = !this.maximum.val
        !this.data[5] = !this.step.val
        !this.data[6] = !this.stringInput.val
        !this.data[7] = !this.delimiter.val
    else
        if !this.data.size().eq(7) then
            !this.tempInput.val = !this.data[1]
            !this.tempChoiceFrame.val = !this.data[2]
            !this.minimum.val = !this.data[3]
            !this.maximum.val = !this.data[4]
            !this.step.val = !this.data[5]
            !this.stringInput.val = !this.data[6]
            !this.delimiter.val = !this.data[7]
            !this.temperatureConvert()
            !this.fill(|Celsius|)
            !this.split()
        endif
    endif
endmethod

```

New menu objects

New Accept/Discard buttons

Opencallback on the Results tab

Method to transfer to and from the
.data member

Appendix B7 - Example cuboid.pmlobj

```
define object CUBOID
  member .xlen is REAL
  member .ylen is REAL
  member .zlen is REAL
endobject

define method .cuboid()
  !this.xlen = 0
  !this.ylen = 0
  !this.zlen = 0
endmethod

define method .cuboid(!x is REAL, !y is REAL, !z is REAL)
  !this.xlen = !x
  !this.ylen = !y
  !this.zlen = !z
endmethod

define method .volume() is REAL
  return !this.xlen * !this.ylen * !this.zlen
endmethod

define method .surfaceArea() is REAL
  return !this.xlen * !this.zlen * 2 + !this.ylen * !this.zlen * 2 + !this.xlen * !this.ylen * 2
endmethod
```

Appendix B8 - Example datastore.pmlobj

```
define object DATASTORE
  member .formTextGadgets is ARRAY
  member .gadgetValues is ARRAY
endobject

define method .dataStore(!form is form)
  !this.formTextGadgets.clear()
  !this.gadgetValues.clear()
  !formMembers = !form.attributes()
  do !n index !formMembers
    if !form.attribute(!formMembers[!n]).objectType().eq(|GADGET|) then
      if !form.attribute(!formMembers[!n]).type().eq(|TEXT|) then
        !this.formTextGadgets.append(!form.attribute(!formMembers[!n]))
        !this.gadgetValues.append(!form.attribute(!formMembers[!n]).val)
      endif
    endif
  enddo
endmethod

define method .updateInfo()
  do !n index !this.formTextGadgets
    !this.gadgetValues[!n] = !this.formTextGadgets[!n].val
  enddo
endmethod

define method .restoreInfo()
  do !n index !this.formTextGadgets
    !this.formTextGadgets[!n].val = !this.gadgetValues[!n]
  enddo
endmethod
```

Appendix B9 - Example ex8.pmlfrm

```
setup form !!ex8
  !this.formTitle      = |Equipment Checker|
  !this.initcall       = |!this.init()|
  !this.newMenu(|pop1|)
  !this.pop1.add('CALLBACK', 'Go to Nozzle', '!!CE = !this.nozz.selection().dbref()', 'NOZZ')
  para .title text |Available Equipment|
  line .titleLine at xmin.title ymax.title horiz wid 48
  combo .equip at xmin.title + 0.5 ymax.titleLine + 0.2 call |!this.validate(| width 10
  list .nozz at xmin.equip ymax.equip width 45 length 5
  button .site linklabel |Update| at xmax.equip + 0.5 ymin.equip call |!this.init()|
  button .refreshNozz at xmax.nozz - size ymax.nozz tooltip |Refresh Nozzles| call
  |!this.checkNozz()| pixmap wid 16 hei 16
  textpane .atta |Equipment Attributes| at xmin.equip ymax.refreshNozz width 47 height 3.5
```

```

    button .updateAtta linklabel |Update Atts| at xmax.nozz - size ymax.atta + 0.2 call
|!this.updateAtt(2)| wid 7
exit

define method .ex8()
    !nozz = |Name/Connected?/Attached?/Aligned?/Size check?|
    !this.nozz.setHeadings(!nozz.split(|/|))
    !info[1] = 'Description - Unset'
    !info[2] = 'Function - Unset'
    !info[3] = 'Purpose - Unset'
    !this.atta.val = !info
    !this.nozz.setpopup(!this.pop1)
    !this.refreshNozz.addPixmap(!!pml.getPathName(|refresh16.png|))
endmethod

define method .init()
    if !!CE.type.eq(|SITE|) then
        !level = |SITE|
    else
        !level = |ZONE|
    endif
    VAR !coll COLL ALL EQUIP FOR $!level
    handle ANY
        !!Alert.Error(|Make sure you are at an EQUI, ZONE or SITE element|)
    elsehandle NONE
        if !coll.size().eq(0) and !level.eq(|ZONE|) then
            VAR !coll COLL ALL EQUIP FOR SITE
            !!alert.Message(|No equipment in current ZONE, so whole SITE will be searched|)
        endif
        VAR !name EVAL FLNN FOR ALL FROM !coll
        !this.equip.dtext = !name
        !this.equip.rtext = !coll
        !this.collNozz()
    endhandle
endmethod

define method .validate(!gad is GADGET, !event is STRING)
    if !event.eq('VALIDATE') then
        !userInput = !this.equip.displayText()
        do !n index !this.equip.dtext
            !Chrs = !userInput.length()
            !test = !this.equip.dtext[!n].upcase().substring(1, !Chrs)
            if !userInput.upcase().eq(!test) then
                !this.equip.val = !n
                break
            endif
        enddo
    endif
    !this.collNozz()
endmethod

define method .collNozz()
    !equipRef = !this.equip.selection().dbref()
    if !equipRef.unset() then
        !this.nozz.clear()
    else
        !nozzColl = object COLLECTION()
        !nozzColl.type('NOZZ')
        !nozzColl.scope(!equipRef)
        !results = !nozzColl.results()
        !this.nozz.dtext = !results.evaluate(object block ('!results[!evalIndex].flnn'))
        !this.nozz.rtext = !results.evaluate(object block ('!results[!evalIndex].string()'))
        !this.checkNozz()
        !this.updateAtt(1)
    endif
endmethod

define method .checkNozz()
    if !this.nozz.rtext.unset().not() then
        do !n index !this.nozz.rtext
            !nozz = !this.nozz.rtext[!n].dbref()
            do !m from 2 to 4
                !result[!m] = |N/A|
            enddo
            if !nozz.cref.unset().or(!nozz.cref.badref()) then
                !result[1] = |Check|
            else
                !result[1] = |OK|
            endif
        enddo
    endif
endmethod

```

```

!end = |t|
if !nozz.cref.href.eq(!nozz) then
    !end = |h|
endif
if !nozz.pos.wrt( /* ).eq(!nozz.cref.attribute(!end & |pos|).wrt( /* )) then
    !result[2] = |OK|
endif
if !nozz.pdir[1].wrt( /* ).eq(!nozz.cref.attribute(!end & |dir|).wrt( /* )) then
    !result[3] = |OK|
endif
if !nozz.cpar[1].eq(!nozz.cref.attribute(!end & |bore|).real()) then
    !result[4] = |OK|
endif
endif
!nozzInfo[!n][1] = !nozz.flnn
!nozzInfo[!n].appendArray(!result)
enddo
!rtext = !this.nozz.rtext
!this.nozz.setrows(!nozzInfo)
!this.nozz.rtext = !rtext
endif
endmethod

define method .updateAtt(!flag is REAL)
!equip = !this.equip.selection().dbref()
!availAtts = !equip.attributes()
!this.atta.tag = !equip.flnn & ' Attributes'
!rows = !this.atta.val
!out = ARRAY()
do !n index !rows
    !text = !rows[!n]
    !split = !text.split('-')
    !attrib = !split[1].trim()
    !avail = !availAtts.evaluate(object block ('!availAtts[!evalIndex].upcase().substring(1,
!attrib.length()))))
    if !avail.findfirst(!attrib.upcase()).unset().not() then
        if !flag.eq(1) then
            !out.append(!attrib & | | - | &
!equip.attribute(!availAtts[!avail.findfirst(!attrib.upcase())]))
        elseif !flag.eq(2) then
            !val = !split[2].trim()
            !equip.attribute(!availAtts[!avail.findfirst(!attrib.upcase())]).assign(!val)
            !out.append(!attrib & | - | & !val)
        endif
    endif
enddo
!this.atta.val = !out
endmethod

```

Appendix B10 - Example exampleVolumeView.pmlfrm

```

setup form !!exampleVolumeView
!this.formTitle = |Volume View|
!this.initcall = |!this.init()|
!this.firstShownCall = |!this.firstShown()|
!this.killingCall = |!this.close()|
!this.newMenu(|pop1|)
!this.pop1.add('CALLBACK', 'Limits CE', '!this.limitsCE()')
!pix = |pixmap wid hei 16|
path DOWN
para .prompt text |Navigate : | width 46 lines 1
button .but1 tooltip |Limits CE| call |!this.limitsCE()| $!pix
button .but2 tooltip |Walk to Drawlist| call |!this.walkDrawlist()| $!pix
button .but3 tooltip |Add to Drawlist| call |!this.setDrawlist(1, FALSE)| $!pix
button .but4 tooltip |Remove from Drawlist| call |!this.setDrawlist(2, FALSE)| $!pix
view .volumeView at xmax.but1 ymax.prompt volume
    width 46 height 15
    limits auto
    isometric 3
exit
member .drawlist is REAL
exit

define method .exampleVolumeView()
!this.but1.addPixmap(!!pml.getPathName(|autoce16.png|))
!this.but2.addPixmap(!!pml.getPathName(|ng_zoomtodrawlist.png|))
!this.but3.addPixmap(!!pml.getPathName(|addece16.png|))

```

```

!this.but4.addPixmap(!!pml.getPathName(|removece16.png|))
!this.volumeView.borders = FALSE
!this.volumeView.background = |darkslate|
!this.volumeView.shaded = TRUE
!this.volumeView.projection = |PARALLEL|
!this.volumeView.radius = 100
!this.volumeView.range = 500.0
!this.volumeView.eyemode = FALSE
!this.volumeView.step = 25
!this.volumeView.setpopup(!this.pop1)
-- Create local drawlist within the global object
!this.drawlist = !!gphDrawlists.createDrawList()
endmethod

define method .firstShown()
  -- Add 3D view to view system
  !!gphViews.add(!this.volumeView)
  -- Add local drawlist add to 3D view
  !!gphDrawlists.attachView(!this.drawlist, !this.volumeView)
  --!!gphDrawlists.drawlists[!this.drawlist].holes(TRUE)
endmethod

define method .init()
  !this.setDrawlist(1, TRUE)
  -- Active the volume view
  !this.volumeView.active = TRUE
endmethod

define method .close()
  !!gphDrawlists.detachView(!this.volumeView)
  !!gphDrawlists.deleteDrawlist(!this.drawlist)
endmethod

define method .walkDrawlist()
  !drawlist = !!gphDrawlists.drawlist(!this.drawlist)
  -- derive a volume object from the members of the drawlist
  !volume = object volume(!drawlist.members())
  !limits[1] = !volume.from.east
  !limits[2] = !volume.to.east
  !limits[3] = !volume.from.north
  !limits[4] = !volume.to.north
  !limits[5] = !volume.from.up
  !limits[6] = !volume.to.up
  !this.volumeView.limits = !limits
endmethod

define method .limitsCE()
  -- Set the Views limits based on the chosen element
  !!gphViews.limits(!this.volumeView, !!CE)
endmethod

define method .setDrawlist(!flag is REAL, !reset is BOOLEAN)
  !drawlist = !!gphDrawlists.drawlist(!this.drawlist)
  -- Clear the drawlist is a reset is requested
  if !reset then
    !drawlist.removeall()
  endif
  -- Add/Remove CE
  if !flag.eq(1) then
    !drawlist.add(!!CE)
  elseif !flag.eq(2) then
    !drawlist.remove(!!CE)
  endif
  !this.walkDrawlist()
endmethod

```

Appendix B11 - Example ex9.pmlfrm

```

setup form !!ex9
!this.formTitle      = |Equipment Checker|
!this.initcall       = |!this.init()|
!this.firstShownCall = |!this.firstShown()|
!this.killingCall    = |!this.close()|
!this.quitcall       = |!this.clear()|
!this.newMenu(|pop1|)
!this.pop1.add('CALLBACK', 'Go to Nozzle', '!!CE = !this.nozz.selection().dbref()', 'NOZZ')
para .prompt text |Navigate : | width 46 lines 1
frame .view panel at x 0.5 ymax.prompt wid 1 hei 1
    view .volumeView at x 0.5 ymax.prompt VOLUME
        width 50 aspect 0.707
        border off
        shading on
        isometric 3
    exit
exit
para .title at xmax.view + 0.5 ymin.prompt text |Available Equipment|
line .titleLine at xmin.title ymax.title horiz wid 48
combo .equip at xmin.title + 0.5 ymax.titleLine + 0.2 call |!this.validate(| width 10
list .nozz at xmin.equip ymax.equip call |!this.check(| width 45 length 5
button .site linklabel |Update| at xmax.equip + 0.5 ymin.equip call |!this.init()|
button .refreshNozz at xmax.nozz - size ymax.nozz tooltip |Refresh Nozzles| call |!this.checkNozz()|
pixmap wid 16 hei 16

textpane .atta |Equipment Attributes| at xmin.equip ymax.refreshNozz width 47 height 3.5
button .updateAtta linklabel |Update Atts| at xmax.nozz - size ymax.atta + 0.2 call
|!this.updateAtt(2)| wid 7

para .taskTitle at xmin.title ymax.atta + 0.5 text |Available Tasks|
line .taskLine at xmin.title ymax.taskTitle horiz wid.titleLine

para .clipPix at xmin.equip + 1 ymax.taskLine + 0.2 pixmap wid 16 hei 16
button .clipBut linklabel at xmax.clipPix + 1 ymin.clipPix |Add Connected and Enable Clipbox...|
call |!this.enableClip()|
para .tagNozzPix at xmin.equip + 1 ymax.clipPix + 0.2 pixmap wid 16 hei 16
button .tagNozzBut linklabel at xmax.clipPix + 1 ymin.tagNozzPix |Tag Selected Nozzle| call
|!this.tag(!this.tagNozzBut)|
para .tagNozzlesPix at xmin.equip + 1 ymax.tagNozzPix+ 0.2 pixmap wid 16 hei 16
button .tagNozzlesBut linklabel at xmax.clipPix + 1 ymin.tagNozzlesPix |Tag All Nozzles| call
|!this.tag(!this.tagNozzlesBut)|
para .hlNozzPix at xmin.equip + 1 ymax.tagNozzlesPix+ 0.2 pixmap wid 16 hei 16
button .hlNozzBut linklabel at xmax.clipPix + 1 ymin.hlNozzPix |Highlight Nozzles| call
|!this.highlight()|
frame .limitslide foldup |Edit Clip Volume| at xmin.atta ymax.hlNozzPix + 0.5 width 46
    slider .slide call |!this.slide()| dock t horizontal range 0 +2000 step 100 val 0 width 1 height 1.5
    text .minslide at xmin.limitslide+0.2 ymin.limitslide+2.5 call |!this.updateRange()| width 5 is
REAL
    text .maxslide at xmax.limitslide-size ymin.limitslide+2.5 call |!this.updateRange()| width 5 is
REAL
    toggle .incl |Update view limits?| at xmax.minslide + 5 ymin.limitslide+2.5
exit
frame .colour foldup |Edit Highlight Colour| at xmin.atta ymax.limitslide width.limitslide
    para .para1 at xmin.colour + 0.5 ymin.colour + 1.2 text |Current equipment |
    button .butt1 | | at xmax.para1 - 3 ymin.para1 call |!this.colourpick('A' )| pixmap wid 30 hei 15
    para .para2 at xmin.colour + 0.5 ymax.butt1 text |Connected nozzles |
    button .butt2 | | at xmax.para2 - 3 ymax.butt1 call |!this.colourpick('B' )| pixmap wid 30 hei 15
    para .para3 at xmin.colour + 0.5 ymax.butt2 text |Unconnected nozzles|
    button .butt3 | | at xmax.para3 - 3 ymax.butt2 call |!this.colourpick('C' )| pixmap wid 30 hei 15
    para .para4 at xmin.colour + 0.5 ymax.butt3 text |Check nozzles |
    button .butt4 | | at xmax.para3 - 3 ymax.butt3 call |!this.colourpick('D' )| pixmap wid 30 hei 15

    frame .colourpick panel at xmax.butt1 + 3.5 ymin.colour + 0.5 width 10
        !val = 0
        do !I from 1 to 10
            do !J from 0 to 4
                !val = !val + 1
                !no = !I & |000| & !J
                !x = (!I * 2) - 1
                !y = (!J * 0.6) + 0.75
                button .butt$!no | | pixmap at x$!x ymin.colourpick + $!y call |!this.colourpick($!val)|
            enddo
        enddo
    enddo
exit

```

```

exit
member .clipboard is GPHCLIPBOX
member .drawlist is REAL
member .limits is ARRAY
member .col is REAL
member .tagged is ARRAY
member .highlight is ARRAY
exit

define method .ex9()
    !nozz = |Name/Connected?/Attached?/Aligned?/Size check?|
    !this.nozz.setHeadings(!nozz.split(|/|))
    !info[1] = 'Description - Unset'
    !info[2] = 'Function - Unset'
    !info[3] = 'Purpose - Unset'
    !this.atta.val = !info
    !this.nozz.setpopup(!this.pop1)
    !this.volumeView.borders = FALSE
    !this.volumeView.background = |darkslate|
    !this.volumeView.shaded = TRUE
    !this.volumeView.projection = |PARALLEL|
    !this.volumeView.radius = 100
    !this.volumeView.range = 500.0
    !this.volumeView.eyemode = FALSE
    !this.volumeView.step = 25
    !this.minslide.val = 0
    !this.maxslide.val = 2000
    !this.clipPix.addPixmap(!pml.getPathName(|clip_off.png|))
    !this.refreshNozz.addPixmap(!pml.getPathName(|refresh16.png|))
    !this.tagNozzPix.AddPixmap(!PML.GetPathName('single.png'))
    !this.tagNozzlesPix.AddPixmap(!PML.GetPathName('multi.png'))
    !this.hlNozzPix.AddPixmap(!PML.GetPathName('highlight.png'))
    !this.colourpick.visible = FALSE
    !this.limitsslide.visible = FALSE
    !this.limitsslide.expanded = FALSE
    !this.colour.visible = FALSE
    !this.colour.expanded = FALSE
    !this.drawlist = !gphDrawlists.createDrawList()
    !this.clipbox = object GPHCLIPBOX()
    !this.clipbox.view = !this.volumeView
    !this.clipbox.capOn()
    !this.butt1.background = 1
    !this.butt2.background = 20
    !this.butt3.background = 18
    !this.butt4.background = 23
endmethod

define method .firstShown()
    -- Add 3D view to view system
    !gphViews.add(!this.volumeView)
    -- Add local drawlist add to 3D view
    !gphDrawlists.attachView(!this.drawlist, !this.volumeView)
endmethod

define method .close()
    !gphDrawlists.detachView(!this.volumeView)
    !gphDrawlists.deleteDrawlist(!this.drawlist)
endmethod

define method .init()
    !this.volumeView.active = TRUE
    if !!CE.type.eq(|SITE|) then
        !level = |SITE|
    else
        !level = |ZONE|
    endif
    VAR !coll COLL ALL EQUIP FOR $!level
    handle ANY
        !!Alert.Error(|Make sure you are at an EQUI, ZONE or SITE element|)
    elsehandle NONE
        if !coll.size().eq(0) and !level.eq(|ZONE|) then
            VAR !coll COLL ALL EQUIP FOR SITE
            !!alert.Message(|No equipment found in current ZONE, so whole SITE will be searched|)
        endif
        VAR !name EVAL FLNN FOR ALL FROM !coll
        !this.equip.dtext = !name
        !this.equip.rtext = !coll
    
```

```

        !this.collNozz()
    endhandle
endmethod

define method .validate(!gad is GADGET, !event is STRING)
    if !event.eq('VALIDATE') then
        !userInput = !this.equip.displayText()
        do !n index !this.equip.dtext
            !Chrs = !userInput.length()
            !test = !this.equip.dtext[!n].upcase().substring(1, !Chrs)
            if !userInput.upcase().eq(!test) then
                !this.equip.val = !n
                break
            endif
        enddo
    endif
    !this.collNozz()
endmethod

define method .collNozz()
    !equipRef = !this.equip.selection().dbref()
    if !equipRef.unset() then
        !this.nozz.clear()
    else
        !nozzColl = object COLLECTION()
        !nozzColl.type('NOZZ')
        !nozzColl.scope(!equipRef)
        !results = !nozzColl.results()
        !this.nozz.dtext = !results.evaluate(object block ('!results[!evalIndex].flnn'))
        !this.nozz.rtext = !results.evaluate(object block ('!results[!evalIndex].string()'))
        !this.checkNozz()
        !this.clear()
        !this.updateAtt(1)
        !this.setDrawlist()
        !this.enableClip()
        !this.highlight()
        !this.tagged.clear()
        do !n index !this.nozz.dtext
            !this.tagged[!n] = FALSE
        enddo
    endif
endmethod

define method .checkNozz()
    if !this.nozz.rtext.unset().not() then
        do !n index !this.nozz.rtext
            !nozz = !this.nozz.rtext[!n].dbref()
            do !m from 2 to 4
                !result[!m] = |N/A|
            enddo
            if !nozz.cref.unset().or(!nozz.cref.badref()) then
                !result[1] = |Check|
            else
                !result[1] = |OK|
                !end = |t|
                if !nozz.cref.href.eq(!nozz) then
                    !end = |h|
                endif
                if !nozz.pos.wrt( /* ).eq(!nozz.cref.attribute(!end & |pos|).wrt( /* )) then
                    !result[2] = |OK|
                endif
                if !nozz.pdir[1].wrt( /* ).eq(!nozz.cref.attribute(!end & |dir|).wrt( /* )) then
                    !result[3] = |OK|
                endif
                if !nozz.cpar[1].eq(!nozz.cref.attribute(!end & |bore|).real()) then
                    !result[4] = |OK|
                endif
            endif
            !nozzInfo[!n][1] = !nozz.flnn
            !nozzInfo[!n].appendArray(!result)
        enddo
        !this.highlight = !nozzInfo
        !rtext = !this.nozz.rtext
        !this.nozz.setrows(!nozzInfo)
        !this.nozz.rtext = !rtext
    endif
endmethod

```

```

define method .updateAtt(!flag is REAL)
    !equip = !this.equip.selection().dbref()
    !availAtts = !equip.attributes()
    !this.atta.tag = !equip.flnn & ' Attributes'
    !rows = !this.atta.val
    !out = ARRAY()
    do !n index !rows
        !text = !rows[!n]
        !split = !text.split('-')
        !attrib = !split[1].trim()
        !avail = !availAtts.evaluate(object block ('!availAtts[!evalIndex].upcase().substring(1,
!attrib.length()'))
        if !avail.findfirst(!attrib.upcase()).unset().not() then
            if !flag.eq(1) then
                !out.append(!attrib & | - | &
!equip.attribute(!availAtts[!avail.findfirst(!attrib.upcase())]))
            elseif !flag.eq(2) then
                !val = !split[2].trim()
                !equip.attribute(!availAtts[!avail.findfirst(!attrib.upcase())]).assign(!val)
                !out.append(!attrib & | - | & !val)
            endif
        endif
    enddo
    !this.atta.val = !out
endmethod

define method .setDrawlist()
    !equip = !this.equip.selection().dbref()
    !drawlist = !gphDrawlists.drawlist(!this.drawlist)
    !drawlist.removeall()
    !drawlist.add(!equip)
    !!gphViews.limits(!this.volumeView, !equip)
endmethod

define method .slide()
    !value = !this.slide.val
    if !this.incl.val.eq(TRUE) then
        !limits = !this.limits
        !modlimit[1] = !limits[1] - !value
        !modlimit[2] = !limits[2] + !value
        !modlimit[3] = !limits[3] - !value
        !modlimit[4] = !limits[4] + !value
        !modlimit[5] = !limits[5] - !value
        !modlimit[6] = !limits[6] + !value
        !this.volumeView.limits = !modlimit
    endif
    !this.volumeView.clipBoxXlen = !this.clipbox.box.xlength + !value
    !this.volumeView.clipBoxYlen = !this.clipbox.box.ylength + !value
    !this.volumeView.clipBoxZlen = !this.clipbox.box.zlength + !value
    !this.volumeView.refresh()
endmethod

define method .updateRange()
    !range[1] = !this.minslide.val
    !range[2] = !this.maxslide.val
    !range[3] = 100
    !this.slide.range = !range
    !this.slide.refresh()
endmethod

define method .enableClip()
    !drawlist = !gphDrawlists.drawlist(!this.drawlist)
    !connectNozz = ARRAY()
    do !n index !this.nozz.rtext
        !nozz = !this.nozz.rtext[!n]
        if !nozz.dbref().cref.unset().not().and(!nozz.dbref().cref.badref().not()) then
            !connectNozz.append(!nozz.dbref().cref)
        endif
    enddo

    !volume = object volume(!this.equip.selection().dbref())
    !this.clipbox.box = !volume.box()
    if !this.clipBut.val then
        !this.clipBut.tag = |Remove Connected and Disable Clipbox...|
        !this.clipPix.addPixmap(!!pml.getPathName(|clip_on.png|))
    endif
endmethod

```

```

        !this.limitslide.visible = TRUE
    do !n index !connectNozz
        !drawlist.add(!connectNozz[!n])
    enddo
    !this.limits = !this.volumeView.limits
    !this.clipbox.active = FALSE
    !this.clipbox.set()
    !this.clipbox.active = TRUE
    !this.volumeView.clipping = TRUE
    !this.slide()
else
    !this.clipBut.tag = |Add Connected and Enable Clipbox...|
    !this.clipPix.addPixmap(!pml.getPathName(|clip_off.png|))
    !this.limitslide.visible = FALSE
    !this.limitslide.expanded = FALSE
    do !n index !connectNozz
        !drawlist.remove(!connectNozz[!n])
    enddo
    !this.clipbox.active = FALSE
    !this.volumeView.clipping = FALSE
endif
endmethod
define method .tag(!gad is GADGET)
    if !gad.tag.upcase().eq(|TAG SELECTED NOZZLE|).or(!gad.tag.upcase().eq(|UNTAG SELECTED NOZZLE|))
then
    !start = !this.nozz.val
    !finish = !this.nozz.val
else
    !start = 1
    !finish = !this.nozz.dtext.size()
endif

    if !gad.val.eq(TRUE) then
        do !n from !start to !finish
            if !this.tagged[!n].eq(FALSE) then
                !nozzname = !this.nozz.rtext[!n].dbref().flnn
                !nozzpos = !this.nozz.rtext[!n].dbref().pos
                !num = 1500 + !n
                AID TEXT NUMBER $!num '$!nozzname' AT $!nozzpos
                !this.tagged[!n] = TRUE
            endif
        enddo
    else
        do !n from !start to !finish
            !num = 1500 + !n
            AID CLEAR text $!num
            handle ANY
            endhandle
            !this.tagged[!n] = FALSE
        enddo
    endif
    !testTagged = !this.tagged
    !testTagged.unique()
    if !testtagged.size().eq(1).and(!testtagged[1].eq(TRUE)) then
        !this.tagNozzlesBut.val = TRUE
        !this.tagNozzlesBut.tag = |Untag All Nozzles|
        !this.tagNozzlesPix.addPixmap(!pml.getPathName(|multi_x.png|))
    else
        !this.tagNozzlesBut.val = FALSE
        !this.tagNozzlesBut.tag = |Tag All Nozzles|
        !this.tagNozzlesPix.addPixmap(!pml.getPathName(|multi.png|))
    endif
    !this.check()
endmethod

define method .check()
    !check = !this.tagged[!this.nozz.val]
    if !check.eq(TRUE) then
        !this.tagNozzBut.val = TRUE
        !this.tagNozzBut.tag = |Untag Selected Nozzle|
        !this.tagNozzPix.addPixmap(!pml.getPathName(|single_x.png|))
    else
        !this.tagNozzBut.val = FALSE
        !this.tagNozzBut.tag = |Tag Selected Nozzle|
        !this.tagNozzPix.addPixmap(!pml.getPathName(|single.png|))
    endif
endmethod

```

```

define method .colourpick(!colour is ANY)
!col = !colour.string()
if !col.eq('A') then
!this.col = 1
!this.colourpick.visible = TRUE
elseif !col.eq('B') then
!this.col = 2
!this.colourpick.visible = TRUE
elseif !col.eq('C') then
!this.col = 3
!this.colourpick.visible = TRUE
elseif !col.eq('D') then
!this.col = 4
!this.colourpick.visible = TRUE
else
if !this.col.eq(1) then
!this.butt1.background = !col.real()
elseif !this.col.eq(2) then
!this.butt2.background = !col.real()
elseif !this.col.eq(3) then
!this.butt3.background = !col.real()
elseif !this.col.eq(4) then
!this.butt4.background = !col.real()
endif
!this.colourpick.visible = FALSE
!this.highlight()
endif
endmethod

define method .highlight()
!equip = !this.equip.selection().dbref()
if !this.h1NozzBut.val.eq(TRUE) then
!!gphDrawlists.drawlists[!this.drawlist].highlight(!equip,!this.butt1.background)
!this.h1NozzPix.AddPixmap(!PML.GetPathName('highlight_x.png'))
!this.h1NozzBut.tag = |Unhighlight Nozzles|
!this.colour.visible = TRUE
if !this.nozz.rtext.unset().not() then
!nozz = !this.highlight
do !I index !nozz
!nozzle = !this.nozz.rtext[!I].dbref()
if !nozz[!I][2].eq(|OK|) then
!col = !this.butt2.background
elseif !nozz[!I][2].eq(|Check|) then
!col = !this.butt3.background
endif
if !nozz[!I][3].eq(|Check|) or !nozz[!I][4].eq(|Check|) or !nozz[!I][5].eq(|Check|) then
!col = !this.butt4.background
endif
!!gphDrawlists.drawlists[!this.drawlist].highlight(!nozzle,!col)
enddo
endif
else
!this.h1NozzPix.AddPixmap(!PML.GetPathName('highlight.png'))
!this.h1NozzBut.tag = |Highlight Nozzles|
!this.colour.visible = FALSE
!this.colour.expanded = FALSE
!!gphDrawlists.drawlists[!this.drawlist].unhighlight(!equip)
if !this.nozz.rtext.unset().not() then
do !i index !this.nozz.rtext
!!gphDrawlists.drawlists[!this.drawlist].unhighlight(!this.nozz.rtext[!i].dbref())
enddo
endif
endif
endmethod

define method .clear()
!equip = !this.equip.selection().dbref()
!!gphDrawlists.drawlists[!this.drawlist].unhighlight(!equip)
do !n index !this.nozz.rtext
!num = 1500 + !n
AID CLEAR text $!num
handle ANY
endhandle
enddo
!this.tagNozzBut.val = FALSE
!this.tagNozzBut.tag = |Tag Selected Nozzle|
!this.tagNozzPix.addPixmap(!pml.getPathName(|single.png|))
!this.tagNozzlesBut.val = FALSE

```

```
!this.tagNozzlesBut.tag = |Tag All Nozzles|
!this.tagNozzlesPix.addPixmap(!pml.getPathName(|multi.png|))
endmethod
```

Appendix B12 - Example ex10.pmlfrm – part 1

```
setup form !!ex10
!this.formTitle      = |Equipment Checker|
!this.initcall       = |!this.init()|
!this.newMenu(|pop1|)
!this.pop1.add('CALLBACK', 'Go to Nozzle', '!!CE = !this.nozz.selection().dbref()')
para      .title text |Available Equipment|
line      .titleLine at xmin.title ymax.title horiz wid 48
button    .pick tooltip 'Identify Equipment' pixmap at xmin.title + 0.5 ymax.titleLine + 0.2 call
|!this.pick()| width 16 height 16
option .equip at xmax.pick ymin.pick call |!this.collNozz()| width 10
list      .nozz at xmin.pick ymax.equip width 45 length 5
button    .site linklabel |Update| at xmax.equip + 0.5 ymin.equip call |!this.init()|
exit

define method .pick()
!packet = object EDGPACKET()
!packet.elementPick(|Identify Equipment|)
!packet.description = |Identify Equipment|
!packet.action = |!ex10.identify(!this.return)|
!!edgCntrl.add(!packet)
endmethod

define method .identify(!pickInfo is ANY)
!item = !pickInfo[1].item
!!edgCntrl.remove(|Identify Equipment|)
-- Loop to find the EQUI element
do
break if (!item.type eq |EQUI|) or (!item.type eq |WORL|)
!item = !item.owner
enddo
if !item.type eq |WORL| then
!!Alert.Warning(|Choose a piece of equipment|)
!this.pick()
else
-- Find the EQUI in the OPTION gadget
!this.equip.val = !this.equip.dtext.FindFirst(!item.flnn)
handle ANY
!!Alert.Warning(|The picked equipment is not available in the current OPTION gadget|)
endhandle
!this.collNozz()
endif
endmethod
```

Appendix B13 - Example ex10.pmlfrm – part 2

```
define method .pick()
!packet = object EDGPACKET()
!packet.elementPick(|Identify EQUI or NOZZ - Escape to cancel|)
!packet.description = |Identify Element|
!packet.action = |!ex10a.identify(!this.return)|
!!edgCntrl.add(!packet)
endmethod

define method .identify(!pickInfo is ANY)
!item = !pickInfo[1].item
!!edgCntrl.remove(|Identify Element|)
-- Loop to find the EQUI element
if !item.type.neq(|NOZZ|) then
do
break if (!item.type.eq(|EQUI|).or(!item.type.eq(|WORL|)))
!item = !item.owner
enddo
endif
if !item.type.eq(|WORL|) then
!!Alert.Warning(|Choose either a nozzle or piece of equipment|)
!this.pick()
elseif !item.type.eq(|EQUI|) then
-- Find the EQUI in the OPTION gadget
!this.equip.val = !this.equip.rtext.findFirst(!item.string())
!this.collNozz()
endif
endmethod
```

```
elseif !item.type.eq(|NOZZ|) then
  -- Pick the chosen NOZZ
  if !this.nozz.rtext.findFirst(!item.string()).unset() then
    !equi = !item
    do
      break if (!equi.type.eq(|EQUI|))
      !equi = !equi.owner
    enddo
    if !this.equip.rtext.findFirst(!equi.string()).set() then
      !this.equip.val = !this.equip.rtext.findFirst(!equi.string())
      !this.collNozz()
      !this.nozz.val = !this.nozz.rtext.findFirst(!item.string())
    endif
  else
    !this.nozz.val = !this.nozz.rtext.findFirst(!item.string())
  endif
endif
endmethod
```

Appendix B14 - Example ex10.pmlfrm – part 3

```
setup form !!ex10b
!this.formTitle      = |Equipment Checker|
!this.initcall       = |!this.init()|
!this.firstShownCall = |!this.firstShown()|
!this.killingCall    = |!this.close()|
!this.newMenu(|pop1|)
!this.pop1.add('CALLBACK', 'Go to Nozzle', '!!CE = !this.nozz.selection().dbref()', 'NOZZ')
para .prompt text |Navigate : | width 46 lines 1
frame .view panel at x 0.5 ymax.prompt wid 1 hei 1
  view .volumeView at x 0.5 ymax.prompt call '!!edgCntrl.canvasPick(' prompt .prompt VOLUME
  width 50 aspect 0.707
  border off
  shading on
  isometric 3
  inmode create _default type |DES_NAVIGATE|
  inmode create _pick type |DES_PICK|
exit
exit
para .title at xmax.view + 0.5 ymin.prompt text |Available Equipment|
line .titleLine at xmin.title ymax.title horiz wid 48
combo .equip at xmin.title + 0.5 ymax.titleLine + 0.2 call |!this.validate(| width 10
list .nozz at xmin.equip ymax.equip call |!this.check()| width 45 length 5
button .site linklabel |Update| at xmax.equip + 0.5 ymin.equip call |!this.init()|
button .refreshNozz at xmax.nozz - size ymax.nozz tooltip |Refresh Nozzles| call
|!this.checkNozz()| pixmap wid 16 hei 16
button .pick tooltip 'Identify Nozzle' pixmap at xmin.refreshNozz - size ymax.nozz call
|!this.pick()| width 16 height 16
textpane .atta |Equipment Attributes| at xmin.equip ymax.refreshNozz width 47 height 3.5
button .updateAtta linklabel |Update Atts| at xmax.nozz - size ymax.atta + 0.2 call
|!this.updateAtt(2)| wid 7
member .drawlist is REAL
exit

define method .pick()
  !packet = object EDGPACKET()
  !packet.elementPick(|Identify a nozzle - Escape to cancel|)
  !packet.description = |Identify Element|
  !packet.action = |!ex10b.identify(!this.return)|
  !!edgCntrl.add(!packet)
  !!edgCntrl.addView(!ex10b.volumeView)
endmethod

define method .identify(!pickInfo is ANY)
  !item = !pickInfo[1].item
  !!edgCntrl.remove(|Identify Element|)
  !!edgCntrl.removeView(!ex10b.volumeView)
  -- Loop to find the EQUI element
  if !item.type.eq(|NOZZ|) then
    !this.nozz.val = !this.nozz.rtext.findFirst(!item.string())
  else
    !this.pick()
  endif
endmethod
```

Appendix B15 - Example apppml.obj

PML addin file

```
name:      PMLTraining
title:     PML Training
showOnMenu: FALSE
object:    appPML
```

apppml.obj

```
define object appPML
endobject

define method .modifyMenus()
    !this.utilitiesMenu()
endmethod

define method .modifyForm()
endmethod

define method .utilitiesMenu()
    !menu = object APPMENU('SYSUTIL')
    !menu.add('SEPARATOR')
    !menu.add('FORM', |Calculator|, |ex5|)
    !menu.add('MENU', |Equipment Checker|, 'EquipCheck')
    !!appMenuCntrl.addMenu(!menu, 'EQUI')

    !menu= object APPMENU('EquipCheck')
    !menu.add('FORM', |Exercise 8...|, |ex8|)
    !menu.add('FORM', |Exercise 9...|, |ex9|)
    !!appMenuCntrl.addMenu(!menu, 'EQUI')
endmethod
```